

MULTI-PARTICLE CONTROL FOR NONLINEAR DATA ASSIMILATION IN CHAOTIC SYSTEMS

SONAL BHATIA, '27

SUBMITTED TO THE
DEPARTMENT OF MECHANICAL AND AEROSPACE ENGINEERING
PRINCETON UNIVERSITY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF
UNDERGRADUATE SENIOR THESIS.

FINAL REPORT

APRIL 21, 2026

RYNE BEESON
CHRISTINE ALLEN-BLANCHETTE
MAE 497
189 PAGES
FILE COPY

© Copyright by Sonal Bhatia, 2026.

All Rights Reserved

This report represents my own work in accordance with University regulations.

Abstract

Particle filters provide a principled framework for Bayesian state estimation in non-linear dynamical systems, but suffer from weight degeneracy in chaotic or high-dimensional regimes where small forecast errors amplify rapidly between observations. Optimal nudging controllers can mitigate this collapse, but their computational cost makes them impractical for real-time deployment.

This thesis develops an imitation learning framework that amortizes the cost of optimal nudging offline. Training data is generated by running a Feynman-Kac Monte Carlo oracle across a low-discrepancy sample of the chaotic attractor, with Girsanov Radon-Nikodym corrections accumulated along each trajectory to preserve importance-sampling validity. A neural network is trained via behavioral cloning to imitate the oracle’s per-particle control policy, replacing the full Monte Carlo computation with a single forward pass at deployment. Four model variants are evaluated: a per-particle regression baseline, two DeepSets ensemble-aware controllers with weight-aware pooling, and a residual architecture initialized from the per-particle checkpoint.

On Lorenz-63, the per-particle controller achieves a 77% reduction in normalized RMSE relative to the uncontrolled particle filter and a $4\times$ improvement over the oracle nudged particle filter, at $70\times$ lower computational cost. This is explained by an implicit Girsanov regularization: Huber regression on a heavy-tailed oracle distribution produces calibrated controls that preserve ensemble diversity. The pipeline scales to Lorenz-96 at $K = 8$ state variables with no architectural changes, where the per-particle controller matches the oracle in accuracy while maintaining higher normalized effective sample size at a $27\times$ lower cost. The L96 speedup is conservative relative to L63 because the oracle uses a reduced inner-loop configuration ($K_c = 4$, $n_s = 10$ versus $K_c = 8$, $n_s = 15$ on L63) chosen for tractable data generation, while the structural advantage of the amortized approach scales as $\mathcal{O}(d^2)$ versus $\mathcal{O}(d)$ per integration and grows with state dimension.

These results show that importance-sampling correctness, not model complexity, is the primary constraint on learned particle filter controllers, and that oracle design is as critical as architecture in determining performance.

Keywords: Particle Filtering, Chaotic Systems, Data Assimilation, Imitation Learning, Optimal Control, State Estimation, Girsanov Correction, Lorenz-63, Lorenz-96

Acknowledgements

I would like to express my deepest gratitude to Professor Ryne Beeson for his guidance, mentorship, and unwavering belief in me throughout this project. Over the past two years, his insight into particle filtering, control theory, and chaotic systems has shaped not only the technical direction of this work, but also the way I think, question, and approach research.

I am especially thankful to Theofania (Nina) Karampela and Anjian Li for their generosity with their time, ideas, and patience. Their willingness to think through problems with me, to challenge assumptions, and to persist through uncertainty made this work stronger in ways that go far beyond what is written here. I extend this gratitude to the entire Beeson Lab Group for their thoughtful engagement and feedback during my thesis presentation.

My gratitude to Professor Beeson extends beyond Independent Work. The MAE 342: Space System Design course stood as a parallel culmination of my undergraduate experience in aerospace engineering. It was in this course that the principles underlying this thesis took on a different form: design thinking, disciplined requirements formulation, and the challenge of translating abstract theory into systems that must function in the real world. Leading a team of 16 through the conception and development of a mission demanded technical rigor, clarity, responsibility, and trust. This experience reinforced my passion across engineering and leadership, and I'm grateful to have concluded my aerospace engineering studies at Princeton with both this course and the thesis under Professor Beeson's guidance.

I am also grateful to the many professors across MAE, COS, and Robotics whose teaching and mentorship have guided my academic path. In particular, I would like to thank Professor Christine Allen-Blanchette for her thoughtful reading of this work, Professor Radhika Nagpal for helping me become more comfortable with mathematical derivations during the MAE 345 course, Professor Luigi Martinelli for his engagement and design thinking guidance, and Professor Michael Littman for his unwavering support and advising. I also had the privilege to learn from Professor Gmachl, Professor Liuchi Li, Professor Margaret Martonosi, Professor Vikram Ramaswamy, and Professor Olga Russakovsky, across courses that further enhanced my technical depth. I am also thankful to Jon Prevost and Mike Galvin for always making me think deeply about my implementations, and to Ms. Theresa Russo and the Department of Mechanical and Aerospace Engineering for their care and coordination throughout the Independent Work process.

Beyond the classroom, I owe so much to the communities that shaped me long before this project began. To SEEDS Access, for creating spaces where I could grow with confidence and purpose. To St. Andrew's, for instilling in me a sense of curiosity, discipline, and resilience that carried me through every challenge. And to Princeton, the place I dreamed of attending since I first moved to America with my mom, for giving me the opportunity to learn, build, and see how these ideas extend into the world, and for giving me a chance to give back to my family and community.

Finally, and most importantly, I want to thank my family and the friends, who soon became an extension of my family. To my parents and younger sister, for their constant love, sacrifices, and relentless belief in me. To my dad, for teaching me the importance of hard work and dedication. To my mom, for being my first teacher, mentor, and best friend. And to Divya, as a reminder that when we put our best foot forward, life always works out. This work is as much yours as it is mine.

To my friends, my constants in chaos, offering laughter when I needed it most, filling many ordinary days with something unforgettable, outlasting every stressful week and late night, never letting me feel alone in any of it, grounding me when everything felt uncertain, I will go onward in life with your memories to cherish for a lifetime.

And to Ian Malcolm of Jurassic Park, the great mathematician who first introduced me to chaos theory, well before I understood first principles, let alone write a thesis of this depth. Ian reminded us that life finds a way, and in this thesis, I hope to show how ML can find a way.

This thesis is beyond the result of my individual effort, it arises from the community and people that lifted me, guided me, and believed in me every step of the way. I will carry that with me, always and forever. Thank you.

Contents

Abstract	iii
Acknowledgements	i
List of Figures	ix
List of Tables	xv
1 Introduction	2
1.1 Motivation	2
1.2 Filtering in Chaotic Systems: The Core Difficulty	3
1.3 Particle Nudging as a Remedy	4
1.4 Imitation Learning as the Solution	5
1.5 Diagnostic Baseline: Prior Work	5
1.6 Problem Statement	6
1.7 Approach Overview	7
1.8 Literature Review	9
1.8.1 Nonlinear Bayesian Filtering and Sequential Monte Carlo	9
1.8.2 Particle Degeneracy & the Curse of Dimensionality	9
1.8.3 Controlled and Nudged Particle Filters	10
1.8.4 Hamilton-Jacobi-Bellman Equations & Path-Integral Control	10
1.8.5 Amortized & Learning-Based Approaches to Filtering	11
1.8.6 Chaotic Systems as Filtering Benchmarks	11
1.9 Contributions	13
1.10 Impact and Relevance to Robotics	14
1.11 Thesis Organization	16
2 Engineering Design	17
2.1 Project Goals & Scope	17
2.2 Design Requirements & Constraints	18
2.3 Evaluation Criteria	20

2.4	Final Design	20
2.5	Engineering Standards	21
2.6	Iterative Design Process	24
2.7	Verification Against Design Requirements	27
3	Mathematical Background	30
3.1	Signal & Observation Models	31
3.1.1	Signal Model	31
3.1.2	Observation Model	31
3.1.3	Lorenz-63	32
3.1.4	Lorenz-96	33
3.2	Particle Filtering	34
3.2.1	Sequential Importance Sampling	34
3.2.2	Weight Degeneracy	35
3.2.3	Effective Sample Size & Resampling	36
3.2.4	Classical Nudging: The Relaxation Particle Filter	36
3.3	Optimal Control for Particle Nudging	37
3.3.1	Controlled Particle Dynamics	37
3.3.2	Cost Function	38
3.3.3	Hamilton-Jacobi-Bellman Equation	38
3.3.4	Cole-Hopf Linearization	39
3.3.5	Feynman-Kac Representation and Monte Carlo Evaluation	39
3.4	Girsanov Weight Correction	40
3.4.1	Radon-Nikodym Derivative	41
3.4.2	Corrected Weight Update	42
3.4.3	Zero-Variance Property of the Optimal Control	42
3.5	Evaluation Metrics	43
3.5.1	Normalized RMSE	43
3.5.2	Normalized Effective Sample Size	43
3.5.3	Particle Weight Variance	44
3.6	Summary	44
4	Dataset Generation	45
4.1	Rationale for Offline Data Generation	45
4.2	Attractor Sampling	46
4.3	Dataset Contract	47
4.4	Expert Control Labels	48

4.5	Extension to Lorenz-96	49
4.6	Dataset Summary	50
5	Ensemble-Aware Supervised Learning	51
5.1	From Per-Particle to Ensemble-Aware Control	51
5.2	Input Features and Ensemble Context	52
5.3	Model Architecture	53
5.3.1	Shared Encoder: PerParticleEncoder	53
5.3.2	Per-Particle Baseline (<i>per_particle</i>)	54
5.3.3	Ensemble S1 (<i>ensemble_s1</i>)	54
5.3.4	Ensemble S2 (<i>ensemble_s2</i>)	54
5.3.5	Residual Ensemble (<i>residual_ensemble</i>)	55
5.4	Training Objective & Protocol	55
5.5	Deployment in the Corrected Particle Filter	57
6	Closed-Loop Filtering Experiments	58
6.1	Experimental Setup	58
6.1.1	System & Parameters	58
6.1.2	Filter Configurations	59
6.1.3	Metrics	60
6.2	Results	61
6.2.1	Summary Statistics	61
6.2.2	Aggregate Timeseries and Bar Charts	62
6.2.3	Per-Filter Trajectory Diagnostics	63
6.3	Discussion	63
6.3.1	Per-Particle SR: Mechanism and Performance	63
6.3.2	Ensemble Variants: Collective Information vs Noise	65
6.3.3	What Does the Oracle Gap Tell Us?	68
6.3.4	Why is the Oracle-SR Gap Larger on Lorenz-63 Than Lorenz-96?	70
6.3.5	Summary of Findings	72
7	Scaling to Lorenz-96	73
7.1	System Configuration & Experimental Design	74
7.1.1	Lorenz-96 at $K = 8$, $F = 8$	74
7.1.2	Experimental Parameters	74
7.1.3	Filter Variants	74
7.2	Data Generation & Architecture	74

7.3	Closed-Loop Results	75
7.3.1	Summary Statistics	75
7.3.2	Time Evolution of nRMSE and nESS	75
7.3.3	Summary Bar Chart and Runtime	76
7.4	Discussion	77
7.4.1	The Four Scaling Questions	77
7.4.2	Per-Filter Trajectory Diagnostics	78
7.4.3	Limitations & Future Directions	80
7.4.4	Summary	81
8	Ablation & Robustness Studies	84
8.1	Ablation on Control Clipping	85
8.2	Robustness to Observation Noise	86
8.3	Robustness to Particle Count	88
8.4	IRnPF Oracle Labels: A Hypothesis Test	90
8.4.1	Training Configuration	90
8.4.2	Results: Lorenz-63	91
8.4.3	Results: Lorenz-96	93
8.4.4	Interpretation	95
8.5	Summary	96
9	Conclusion	99
9.1	Summary of Work	99
9.2	Next Steps & Future Research	101
9.3	Closing Remarks	102
	Bibliography	104
A	Definitions & Background Concepts	111
A.1	State Estimation & Data Assimilation	111
A.2	Chaos and Lyapunov Exponents	112
A.3	Particle Filters	113
A.4	Stochastic Differential Equations	115
A.5	Particle Nudging & Imitation Learning	116
A.6	The Girsanov Correction	117
A.7	Hamilton-Jacobi-Bellman Equations	118
A.8	Ensemble-Level Concepts	119

B	The Lorenz Dynamical Systems	121
B.1	Lorenz-63	121
B.1.1	Historical Context and Motivation	121
B.1.2	Equations of Motion	122
B.1.3	Chaotic Behavior and Attractor Geometry	122
B.1.4	Stochastic Extension for Filtering	123
B.1.5	Jacobian	123
B.1.6	Experimental Parameters	124
B.2	Lorenz-96	124
B.2.1	Historical Context and Motivation	124
B.2.2	Equations of Motion	125
B.2.3	Chaotic Behavior and Filtering Difficulty	125
B.2.4	Stochastic Extension for Filtering	126
B.2.5	Jacobian	126
B.2.6	Experimental Parameters	127
B.2.7	Relationship to Full-Scale Lorenz-96	127
C	The Girsanov Correction: Derivation & Implications	128
C.1	Why the Correction Exists	128
C.2	The Mathematical Form of the Correction	129
C.3	The Discrete-Time Implementation	130
C.4	The Zero-Variance Property & What It Means for Training	132
D	Dataset Generation Details & Pseudocode	134
D.1	Dataset Schema	134
D.2	Modeling Assumptions	135
D.3	Lorenz-63 Generation Pseudocode	137
D.3.1	Attractor Sampling and Farthest-Point Pruning	137
D.3.2	Ensemble Regime Sampling	138
D.3.3	Oracle Control and Girsanov Accumulation	139
D.4	Lorenz-96 Generation Pseudocode	140
D.5	Verified Dataset Statistics	141
E	Filter Algorithm Pseudocode	142
E.1	Bootstrap Particle Filter	143
E.2	Relaxation Nudged Particle Filter	144
E.3	Oracle Nudged Particle Filter	146

E.4	Per-Particle Learned Controller	148
E.5	Ensemble-Aware Learned Controller	150
E.6	Systematic Resampling	152
E.7	Filter Comparison Summary	153
F	Hyperparameters & Implementation Details	155
F.1	Dataset Generation Hyperparameters	155
F.2	Model Architecture Hyperparameters	158
F.3	Training Protocol Hyperparameters	159
F.4	Closed-Loop Evaluation Hyperparameters	162
F.5	Repository Structure	164
F.6	Software & Hardware Specifications	168

List of Figures

1.1	Long deterministic Lorenz-63 trajectory (x - y projection) confirming that the simulation reaches and explores the chaotic attractor. The butterfly structure with two lobes and irregular switching between them is visible. This rollout is used only for visualization; the training dataset is constructed from a low-discrepancy subsampling of attractor states rather than a continuous trajectory.	12
1.2	Long deterministic Lorenz-63 trajectory (x - z and y - z projections) showing attractor geometry and lobe-switching behavior. The lobe switches visible here are the primary failure events for the uncontrolled particle filter: particles on the wrong lobe receive near-zero likelihood weight at the subsequent observation step.	12
6.1	Time evolution of nRMSE (top) and nESS (bottom) for all filter variants on Lorenz-63 ($N = 32$, $\Delta T = 0.1$ TU, $\sigma_{\text{obs}} = 2.0$). Solid lines are trial means; shaded bands are ± 1 std over 10 Monte Carlo trials. The <i>per_particle</i> SR controller (green) maintains near-zero nRMSE and stable nESS throughout, while all other variants show episodic nRMSE spikes at lobe-switching events.	62
6.2	Normalized ESS timeseries for the four ML controllers compared against the oracle nPF. The red dashed line marks the resampling threshold $\text{nESS} = 0.5$. The oracle nPF (green dashed) frequently drops toward and below this threshold, confirming that aggressive Feynman-Kac controls repeatedly collapse the ensemble. The <i>per_particle</i> SR (blue) and <i>ensemble_s2</i> (red) maintain higher mean nESS throughout.	63

6.3	Trial-averaged nRMSE (left) and nESS (right) for all filter variants on Lorenz-63 ($N = 32$, 10 trials). Error bars are one standard deviation. Lower nRMSE and higher nESS are both desirable. The <i>per_particle</i> SR controller achieves the lowest nRMSE by a wide margin: 77% below the uncontrolled PF and $4\times$ below the oracle nPF, while maintaining competitive nESS.	64
6.4	Uncontrolled PF diagnostic ($N = 32$, $\sigma_{\text{obs}} = 2.0$, trial 0). The posterior mean (dashed blue) loses phase with the true state (solid black) repeatedly across all three dimensions, most severely at lobe switching events. The RMSE timeseries (bottom right) shows large spikes whenever the ensemble drifts to the wrong lobe. The attractor projections confirm that the posterior mean frequently occupies the wrong lobe. Mean trial RMSE is 0.753 for this trial.	65
6.5	Relaxation PF diagnostic ($N = 32$, trial 0). The static Kalman gain nudge of van Leeuwen [70] produces only marginal improvement over the uncontrolled PF (mean RMSE 0.803 vs 0.753 for this trial). The posterior mean shows the same qualitative failure: phase loss at lobe switches, a posterior mean that lags the truth through both lobes, and episodic RMSE spikes. The gain $K = 0.2 \cdot I_3$ is too conservative to substantially change which particles survive the weight update in the chaotic regime.	66
6.6	Oracle nPF diagnostic ($N = 32$, trial 0). Tracking improves relative to the classical baselines, but the cost is visible in the nESS timeseries (bottom right): nESS drops repeatedly toward and below the resampling threshold (red dashed), with mean nESS 0.734 for this trial. The Feynman-Kac controls aggressively compress the ensemble to reduce RMSE, repeatedly trading weight diversity for tracking accuracy. Despite the high computational cost of 30,518 ms per step, the oracle still produces phase errors in the x and y dimensions.	67

6.7	<i>Per-particle SR</i> diagnostic ($N = 32$, trial 0). The posterior mean (dashed green) tracks the true trajectory closely across all three dimensions throughout the full 20 TU window, including through the lobe-switching events that defeat every other variant. The attractor projections show the mean following the correct orbit with high fidelity. The control magnitude timeseries (bottom right) shows a mean of ≈ 4.8 , with occasional spikes to ≈ 40 – 50 at attractor lobe boundaries. These rare large corrections are applied precisely when a switch is occurring, and the Girsanov correction ensures that their cost is properly accounted for in the importance weights.	68
6.8	<i>Ensemble S1</i> diagnostic ($N = 32$, trial 0). The posterior mean (dashed purple) shows the same qualitative failure as the uncontrolled PF: repeated phase loss at lobe switches. The mean control magnitude (≈ 3.8) is lower than the per-particle SR (≈ 4.8). The weight-aware pooled context g , estimated from only 32 particles, is too noisy to direct stronger corrections at the critical moments when a lobe switch is occurring. The ensemble-level information encoded in g does not provide a reliable enough signal to outperform the simpler per-particle approach.	69
6.9	<i>Ensemble S2</i> ($\lambda = 0.1$) diagnostic ($N = 32$, trial 0). Qualitatively similar to S1 with comparable phase-loss behavior. The weight-balance penalty reduces the mean control magnitude to ≈ 4.1 relative to S1’s 3.8, and slightly increases nESS to 0.93 (mean across trials). The effect is most visible in the z dimension, where S2 tracks the oscillation amplitude more conservatively than per-particle SR. The Stage 2 penalty successfully teaches more conservative controls and improves ensemble diversity, but not at the cost needed to track through lobe switches. . .	70
6.10	<i>Residual ensemble</i> diagnostic ($N = 32$, trial 0). The posterior mean (dashed brown) tracks the true trajectory noticeably better than S1 and S2, consistent with its intermediate nRMSE of 0.45. The mean control magnitude (≈ 9.9) is the highest of all ensemble variants, reflecting the strong per-particle SR base that is partially preserved through the residual initialization. The attractor projections show the mean remaining on the correct lobe for longer stretches than S1/S2. However, the residual correction still introduces enough training noise to prevent full recovery of the per-particle base’s performance.	71

6.11	Uncontrolled PF at $N = 300$ particles (supplementary diagnostic run, not part of the main evaluation). The posterior mean (red dashed) still loses phase at lobe-switching events despite the $9\times$ larger ensemble. Mean nRMSE is 0.68, essentially identical to the $N = 32$ result. This confirms that the PF's failure is a degeneracy problem driven by the observation likelihood geometry: at $\sigma_{\text{obs}} = 2.0$ and $\Delta T = 0.1$ TU, adding more uncontrolled particles does not solve the weight collapse problem. The solution requires nudging, not just a larger ensemble.	72
7.1	Time evolution of nRMSE (left) and nESS (right) for all filter variants on Lorenz-96 ($K = 8$, $F = 8$, $N = 16$ particles). Trial mean with ± 1 std shaded bands over 10 Monte Carlo trials. The oracle nPF achieves the lowest nRMSE but also the lowest nESS; both SR variants maintain near-unity nESS while tracking the true state.	75
7.2	Trial-averaged nRMSE (left) and nESS (right) for all filter variants on Lorenz-96. Error bars are one standard deviation over 10 trials.	76
7.3	Mean wall-clock time per observation step for all filter variants on Lorenz-96 ($K = 8$, $N = 16$). The oracle nPF requires 3,253 ms per step; both learned controllers run at approximately 120 ms, a $27\times$ speedup.	76
7.4	Uncontrolled PF diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The posterior mean (dashed) diverges sharply from the truth (solid) in all three shown dimensions by $t \approx 0.5$ TU. The nRMSE rises steeply to ≈ 2.0 within the first TU and never fully recovers. The 8D attractor projections show the mean tracing a completely uncorrelated orbit, confirming near-complete filter divergence. With only $N = 16$ particles and 3 to 4 positive Lyapunov exponents, the uncontrolled PF has insufficient ensemble mass to track the multiple unstable directions simultaneously.	79

7.5	Relaxation PF diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The static Kalman gain nudge reduces mean nRMSE from 1.178 to 1.054 relative to the uncontrolled PF but does not prevent phase loss. The posterior mean consistently lags the truth in amplitude and phase across all three shown dimensions, and the nESS (right axis, dotted) drops repeatedly toward zero before resampling events restore it. The attractor projections show the mean following a similar but misaligned orbit, indicative of a persistent phase offset rather than complete divergence.	80
7.6	Oracle nPF diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The nESS (bottom right, dotted) oscillates violently between near-zero and unity throughout the 5 TU window, far more severe than the L63 oracle behavior. The aggressive Feynman-Kac controls repeatedly collapse and then resample the ensemble. Despite this, tracking remains imperfect: phase errors are visible in X_0 around $t \approx 1$ –2 TU and in X_2 after $t \approx 3$ TU. The projections show the mean tracing a largely correct but occasionally displaced orbit, with the nRMSE timeseries revealing sustained periods of elevated error between resampling events.	81
7.7	<i>Per-particle SR</i> diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The posterior mean (dashed blue) follows the truth more closely than the uncontrolled PF and relaxation PF, but visible phase errors remain in X_0 and X_2 during $t \approx 1$ –2 TU. The mean control magnitude (bottom right histogram) averages ≈ 10.3 , substantially higher than in L63 (≈ 4.8), reflecting the harder 8D tracking problem. The attractor projections show partial alignment: the mean follows the correct qualitative structure but the tighter correspondence seen in L63 is absent.	82
7.8	<i>Ensemble S1</i> diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). Qualitatively similar to <i>per_particle</i> SR with comparable phase errors in X_0 and X_2 . The mean control magnitude (≈ 10.6) is slightly higher than per-particle SR, but the pooled context from $N = 16$ particles provides no discernible improvement in trajectory tracking.	83
8.1	nRMSE (blue, left axis) and nESS (orange, right axis) as a function of deployment-time clip threshold for the <i>per_particle</i> SR controller. Performance is flat across $U_{\text{clip}} \in \{50, 100, 200, \infty\}$, degrading only at $U_{\text{clip}} = 25$. Error bars are ± 1 std over 10 trials.	86

8.2	Noise robustness: nRMSE (left) and nESS (right) for the uncontrolled PF and <i>per_particle</i> SR at $\sigma_{\text{obs}} \in \{2.0, 4.0\}$. Error bars are ± 1 std over 10 trials. The $\sigma_{\text{obs}} = 1.0$ column is absent because the uncontrolled PF fails at this noise level with $N = 32$ particles.	87
8.3	nRMSE (left) and nESS (right) as a function of particle count N for all four filter variants on Lorenz-63 ($T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, 10 trials). Both ML models are trained at $N = 32$. The <i>per_particle</i> SR controller maintains its advantage across all N , with nRMSE decreasing slightly from 0.166 at $N = 16$ to 0.152 at $N = 128$	89
8.4	nRMSE (left) and nESS (right) for nPF vs IRnPF oracle labels on Lorenz-63 ($N = 32$, 10 trials). Per-particle SR (nPF) dominates on RMSE. Per-particle SR (IRnPF), despite training on $24\times$ fewer examples, achieves the highest nESS of any variant at 0.954.	93
8.5	Time evolution of nRMSE (top) and nESS (bottom) for nPF vs IRnPF label variants on Lorenz-63. Per-particle SR (nPF) tracks near zero RMSE throughout. IRnPF variants track near the PF baseline on RMSE but the per-particle IRnPF variant sustains markedly higher nESS after the initial transient, staying well above the 0.5 resampling threshold.	94
8.6	Training and validation Huber loss curves for the two IRnPF L63 models (40 epochs, no early stopping). Both converge to a validation plateau around 0.250, consistent with underfitting on the 200-example dataset.	95
8.7	nRMSE/ σ_{obs} (left) and nESS (right) for nPF vs IRnPF oracle labels on Lorenz-96 ($N = 16$, $T = 10$ TU, 10 trials). IRnPF and nPF variants are indistinguishable within error bars; per-particle SR (nPF) achieves the lowest nRMSE.	96
8.8	Time evolution of nRMSE/ σ_{obs} (top) and nESS (bottom) for nPF vs IRnPF label variants on Lorenz-96. All controlled variants track well below the PF baseline after the initial transient. IRnPF and nPF variants are indistinguishable by $t \approx 3$ TU.	97
8.9	Training and validation Huber loss curves for the IRnPF L96 models (log scale, 150 epochs with patience-15 early stopping). The pronounced train/val gap indicates overfitting to the 50-example dataset. Validation loss plateaus at $\approx 6 \times 10^{-2}$ while training continues to $\approx 2 \times 10^{-2}$	98

List of Tables

2.1	Design requirements and constraints.	18
2.2	Engineering standards.	21
2.3	Iterative process.	26
2.4	Verification of design requirements against experimental results. . . .	28
4.1	HDF5 dataset schema. For Lorenz-63: $N = 32$, $d = 3$, $K_c = 8$. For Lorenz-96 ($K = 8$): $N = 16$, $d = 8$, $K_c = 4$. <code>ness_w0</code> and <code>ness_wf</code> store raw ESS $\in [1, N]$; divide by N for nESS.	47
4.2	Dataset parameters for Lorenz-63 and Lorenz-96 ($K = 8$).	50
5.1	Comparison of all particle filter controllers. All ML variants include the Girsanov weight correction. The <i>per_particle</i> baseline is the direct successor to the JIW SR model.	55
6.1	Lorenz-63 closed-loop filtering results. Mean $\pm$ std over 10 Monte Carlo trials ($N = 32$ particles, $T = 20$ TU, $\Delta T = 0.1$ TU, $\sigma_{\text{obs}} = 2.0$). nRMSE is normalized by the true state magnitude at each step. Timing values are from a single representative trial (trial 9) run on CPU. Bold indicates the best result per column.	61
7.1	Lorenz-96 closed-loop filtering results ($K = 8$, $F = 8$, $N = 16$ particles, $T = 5$ TU, $\Delta T = 0.05$ TU, $\sigma_{\text{obs}} = 1.0$). Mean $\pm$ std over 10 Monte Carlo trials.	75
8.1	Effect of deployment-time control clipping threshold on the <i>per_particle</i> SR controller. Same checkpoint, no retraining. Lorenz-63, $N = 32$, 10 Monte Carlo trials.	85
8.2	Closed-loop filtering performance at $\sigma_{\text{obs}} \in \{2.0, 4.0\}$ on Lorenz-63 ($N = 32$, $T = 20$ TU, 10 trials). $\sigma_{\text{obs}} = 1.0$ results are omitted: the uncontrolled PF fails at this noise level with $N = 32$ particles (see text).	87

8.3	Closed-loop performance at varying particle counts on Lorenz-63 ($T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, 10 trials). Both ML models are trained at $N = 32$ for all configurations.	88
8.4	Training configuration comparison: nPF vs IRnPF, L63 vs L96. . . .	92
8.5	Lorenz-63 IRnPF hypothesis test ($N = 32$, $T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, 10 trials). IRnPF models trained on 200 examples with multi-substep supervision; nPF models trained on 4,800 examples with first-substep supervision. Dataset size difference is an intentional consequence of compute constraints.	92
8.6	Lorenz-96 IRnPF hypothesis test ($N = 16$, $T = 10$ TU, $\sigma_{\text{obs}} = 1.0$, 10 trials). nRMSE normalized by σ_{obs} ; values are not directly comparable to Table 7.1. IRnPF models trained on 50 examples with first-substep supervision; nPF models on 400. Evaluation window set to $T = 10$ TU to amortize Colab session cost.	93
8.7	Summary of all ablation and robustness findings. Reference point is <i>per_particle</i> SR at $N = 32$, $\sigma_{\text{obs}} = 2.0$ (nRMSE 0.158, nESS 0.867). . .	98
B.1	Lorenz-63 experimental parameters.	124
B.2	Lorenz-96 experimental parameters ($K = 8$, $F = 8$).	127
D.1	HDF5 dataset schema. All shapes verified against production files. L63: $N = 32$, $d = 3$, $K_c = 8$. L96 ($K = 8$): $N = 16$, $d = 8$, $K_c = 4$. . .	135
D.2	Verified dataset statistics extracted from production HDF5 files. L63 has 9,600 allocated rows and 4,800 valid examples (generation was interrupted; the HDF5 pre-allocates the full expected count and the remaining rows are zero-padded).	141
E.1	Summary of filter algorithm properties. All controlled filters include the Girsanov correction. $H = 128$ is the encoder hidden width; d_{in} is the encoder input dimension (12 or 15 for L63; 27 or 35 for L96). . .	154
F.1	Verified dataset generation hyperparameters. Values extracted from the <code>production</code> configuration blocks in <code>init_data_gen.py</code> (L63) and <code>datagen_196.ipynb</code> (L96).	157
F.2	Verified architecture hyperparameters. Parameter counts are extracted directly from saved checkpoints by summing all tensor element counts. . .	159

F.3	Stage 1 training hyperparameters (behavioral cloning from oracle). These apply to all four model variants; ensemble variants additionally use the transfer initialization described below.	161
F.4	Stage 2 training hyperparameters (weight-balance fine-tuning, ensemble_s2 only). Begins from the best Stage 1 checkpoint of ensemble_s1	162
F.5	Verified closed-loop evaluation parameters.	163
F.6	Software versions and hardware used at each pipeline stage.	168

“You’ve never heard of Chaos theory? Non-linear equations?
Strange attractors?”

- *Ian Malcolm, Jurassic Park, 1993*

Chapter 1

Introduction

1.1 Motivation

The problem of *state estimation*, that is, inferring the current hidden state of a dynamical system from noisy and incomplete measurements, is central to science and engineering. Weather forecasting must estimate the atmosphere’s state from sparse radiosonde and satellite data. Autonomous vehicles must localize themselves from noisy GPS and LiDAR readings. Spacecraft navigation must reconstruct an orbit from intermittent ground station contacts. Epidemiological models must infer infection prevalence from delayed and aggregated case counts. In each setting, the estimator must continually reconcile two imperfect sources of information: a mathematical model of how the system evolves, and partial observations of where it actually is.

Data assimilation (DA) is the principled statistical framework for performing this reconciliation sequentially in time [4, 21]. Given a prior belief about the system state, embodied in a probability distribution, and a new observation, Bayes’ theorem prescribes exactly how to update to a posterior. The challenge is computational: the state spaces of real systems are high-dimensional, the dynamics are nonlinear, and the posterior distributions are correspondingly complex, non-Gaussian, and sometimes multimodal. No finite-dimensional sufficient statistic exists for the posterior in general, so one must approximate it.

The difficulty is especially acute in *chaotic systems*, where the Lyapunov exponents of the underlying flow are positive: nearby trajectories separate exponentially. This means that even a small initial uncertainty inflates rapidly into large predictive uncertainty over short forecast horizons [46, 66]. Between observations, a cloud of

model trajectories that initially clusters tightly near the true state fans out across the attractor. When the next observation arrives, only the fraction of trajectories that happened to evolve toward the observation-consistent region of state space receive appreciable posterior weight; the rest become negligible [63]. This *particle degeneracy* is the central computational obstacle to Bayesian filtering in chaotic regimes, and it is the problem this thesis addresses.

The two benchmark systems used throughout this work, the Lorenz-63 and Lorenz-96 models, were designed precisely to exhibit these difficulties in a tractable setting that nonetheless captures the essential phenomenology of atmospheric chaos.

1.2 Filtering in Chaotic Systems:

The Core Difficulty

To make the difficulty concrete, consider the *particle filter* (PF), reviewed in full in Chapter 3 and introduced informally here. A particle filter represents the filtering distribution, that is, the posterior over the current state given all past observations, as a weighted empirical measure over N particles or *ensemble members*:

$$p(x_{t_k} \mid y_{1:k}) \approx \sum_{i=1}^N w_k^{(i)} \delta_{x_k^{(i)}}(dx). \quad (1.2.1)$$

Between observations, each particle is propagated forward under the model dynamics. When a new observation y_{k+1} arrives, each particle’s weight is multiplied by the likelihood $p(y_{k+1} \mid x_{k+1}^{(i)})$, reflecting how well that particle’s predicted state agrees with the data. Particles that are inconsistent with the observation receive exponentially small weights; particles that happen to be nearby receive exponentially large ones.

In low-dimensional, weakly nonlinear systems with frequent and precise observations, this procedure works well. In chaotic systems with sparse observations, it fails catastrophically. After a long forecast interval, the particle cloud has spread across the attractor under the chaotic flow. The observation likelihood sharply peaks near the true state, so effectively all weight concentrates on one or two lucky particles while the remaining $N - 2$ particles become irrelevant. This is particle degeneracy: the *effective sample size* (ESS) collapses from N to $\mathcal{O}(1)$, the empirical distribution loses its representational fidelity, and the filter diverges [18].

The severity of this collapse scales exponentially with state dimension d and with

the observation interval Δt [63, 9]. Snyder et al. [63] showed rigorously that in Gaussian settings, the number of particles required to avoid weight degeneracy grows exponentially in d , making uncontrolled particle filters entirely impractical in high dimensions without structural modification.

1.3 Particle Nudging as a Remedy

A principled strategy for combating particle degeneracy is *particle nudging*, also called controlled particle filtering or the construction of an informed proposal density. The idea is to modify the dynamics under which particles are propagated by adding a control term $u_t^{(i)}$ to each particle’s stochastic differential equation (SDE), so that particles are steered proactively toward the region of state space that the upcoming observation is likely to favor:

$$dx_t^{(i)} = f(x_t^{(i)}) dt + u_t^{(i)} dt + \Sigma dW_t^{(i)}. \quad (1.3.1)$$

By doing so, the particle cloud arrives at the observation time already concentrated near likely states, and weight collapse is dramatically reduced.

The control must be designed carefully. Nudging that is too weak provides little benefit; nudging that is too strong concentrates particles so tightly that the diversity needed for ensemble representation is destroyed. More fundamentally, because nudging changes the *proposal distribution* under which particles are propagated, importance sampling requires a compensating correction to the particle weights, called the *Girsanov* or *Radon-Nikodym* correction, to maintain statistical consistency. Omitting this correction renders the filter biased regardless of how well-designed the control is.

The optimal nudging law, derived from a stochastic optimal control (SOC) formulation in which expected weight variance is minimized, was established by Lingala et al. [45] and extended to multiscale chaotic systems by Yeong et al. [72]. These methods are effective but computationally expensive: the optimal control requires approximating path-integral expectations via Monte Carlo rollouts online, for every particle at every timestep. Hierarchical variants such as the intermediate-resampling nudged particle filter [7, 35] reduce this cost but real-time deployment at scale remains out of reach.

1.4 Imitation Learning as the Solution

Machine learning offers a natural approach to eliminating the online cost of optimal nudging. The core idea is imitation learning: treat the expensive oracle nudging controller as an *expert*, generate demonstrations of its behavior offline across a representative dataset of attractor states and observation configurations, and train a neural network to imitate the expert’s per-particle control decisions. At deployment, the trained network replaces the oracle entirely, producing a control decision in a single forward pass rather than through thousands of augmented SDE integrations.

This specific form of imitation learning is *behavioral cloning* via supervised regression. The network observes input-output pairs $(x^{(i)}, y, \Delta t, \sigma_{\text{obs}}) \mapsto u^*$ from the expert and minimizes a regression loss between its predictions and the demonstrated controls. The result is a multilayer perceptron (MLP) that maps per-particle state and observation information to a per-particle control vector, with no online optimization at deployment.

The critical requirement that distinguishes this setting from generic supervised learning is importance-sampling validity. Any control the network applies modifies the particle proposal distribution, and the resulting change of measure must be accounted for in the importance weights via the Girsanov correction. A network that produces accurate offline imitation error can still degrade closed-loop filtering performance catastrophically if the Girsanov correction is absent or if the learned controls are so aggressive that they collapse the ensemble’s weight diversity. The design of the network, the training objective, and the deployment pipeline must all respect this constraint.

1.5 Diagnostic Baseline: Prior Work

The viability and failure modes of imitation learning for particle nudging were investigated in my Junior Independent Work (JIW) [12]. The JIW compared two learning paradigms on Lorenz-63: conditional diffusion models and supervised regression. Both were trained to imitate expert nudging controls and then deployed in closed-loop particle filters without the Girsanov weight correction. Three findings from the JIW directly motivate the design choices in this thesis.

Finding 1: Supervised regression is the right learning paradigm. Supervised regression achieved average offline imitation error approximately two orders of

magnitude lower than the diffusion model, while requiring only a single forward pass per particle per timestep at deployment. Diffusion models are structurally misaligned with particle filtering because their shared iterative reverse process implicitly couples particles, violating the conditional independence requirement of importance sampling.

Finding 2: Offline accuracy does not guarantee filtering performance.

When deployed without the Girsanov correction, the supervised regression model degraded filtering RMSE by a factor of 4 relative to the uncontrolled baseline, despite its low offline imitation error. Strong, accurate nudging concentrates particles near observations, collapses the ESS, and destroys the relative likelihood balance that importance sampling requires. The JIW established that this degradation is structural: it arises from the missing Girsanov correction, not from insufficient model expressivity.

Finding 3: Three structural deficiencies must be fixed.

The JIW identified the precise changes needed to make supervised regression work in closed loop. First, the Girsanov correction must be accumulated at every integration substep during deployment. Second, the training labels must come from a proper Feynman-Kac optimal oracle rather than a heuristic saturation-based expert. Third, the per-particle architecture must be extended with ensemble-level context so that the controller can reason about the collective ensemble state, not just individual particles. This thesis addresses all three.

1.6 Problem Statement

The objective of this work is to design a scalable, ensemble-aware particle filtering control framework that improves state estimation accuracy and particle diversity in chaotic nonlinear systems, while remaining computationally tractable for deployment at real-time observation rates.

We consider a partially observed stochastic dynamical system:

$$dx_t = f(x_t) dt + u_t dt + \Sigma dW_t, \quad x_t \in \mathbb{R}^d, \quad (1.6.1)$$

with discrete observations

$$y_k = h(x_{t_k}) + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, R), \quad (1.6.2)$$

arriving at times $t_1 < t_2 < \dots$. Here $f: \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a possibly nonlinear and chaotic drift, Σ is a diffusion matrix, and W_t is standard Brownian motion. The observation function $h: \mathbb{R}^d \rightarrow \mathbb{R}^p$ is linear in all experiments, and observations are sparse in time relative to the system’s Lyapunov timescale.

When the control $u_t \neq 0$, importance sampling validity requires incorporating the Radon-Nikodym derivative for the change of measure from the uncontrolled to the controlled diffusion. The log-weight correction over $[t_k, t_{k+1}]$ is:

$$\log \frac{d\mathbb{Q}}{d\mathbb{P}} \Big|_{[t_k, t_{k+1}]} = - \int_{t_k}^{t_{k+1}} (\Sigma^{-1} u_t)^\top dW_t - \frac{1}{2} \int_{t_k}^{t_{k+1}} \|\Sigma^{-1} u_t\|^2 dt, \quad (1.6.3)$$

where $\mathbb{P}$ is the law of the uncontrolled process and $\mathbb{Q}$ is the law of the controlled process. This correction is non-negotiable: omitting it renders the filter importance-inconsistent regardless of how accurate the learned control is.

This thesis addresses three specific design problems that must be solved simultaneously for imitation learning to improve closed-loop filtering performance.

1. **Importance-weight correctness.** The Girsanov correction (1.6.3) must be accumulated at every integration substep during deployment. This is a structural requirement of the deployment pipeline, not a property of the controller.
2. **Ensemble-aware control policy.** The learned MLP must condition on ensemble-level information, including particle weights, the weighted ensemble mean, and per-particle deviations from that mean, while preserving conditional particle independence given those summaries. The policy must learn to distribute control across particles to regulate *relative* weight evolution, not merely minimize individual particle error.
3. **Weight-aware training objective.** The behavioral cloning loss must be augmented to penalize ensemble-level weight collapse, not just per-particle imitation error. The penalty is derived from the Girsanov formula in physical units and directly targets the variance of predicted post-control importance weights.

1.7 Approach Overview

The proposed framework is an imitation learning pipeline organized into two phases.

Offline phase: expert demonstration and behavioral cloning. Expert control demonstrations are generated using the Feynman-Kac Monte Carlo optimal nudging formulation from `pybeeda` [5]. The oracle runs n_s uncontrolled SDE rollouts per particle per control substep to approximate the optimal control. These demonstrations are collected across a low-discrepancy sample of the chaotic attractor and stored as labeled training examples $(x^{(i)}, y, \Delta t, \sigma_{\text{obs}}, u^*)$.

A multilayer perceptron is then trained by behavioral cloning to reproduce the expert controls. The network maps per-particle state and ensemble-level summary statistics to a per-particle control vector:

$$\mu_\theta: (x^{(i)}, y, \Delta t, \sigma_{\text{obs}}, \phi(\mathcal{E})) \mapsto u^{(i)}, \quad (1.7.1)$$

where $\phi(\mathcal{E})$ is a permutation-invariant summary of the ensemble, including a weight-aware pooled context vector from all particles. Training uses the Huber loss ($\delta = 1.0$), which is preferred over MSE because the oracle control distribution is heavy-tailed: near attractor lobe boundaries, the oracle applies very large controls that would dominate an MSE objective. The Huber loss penalizes these outlier labels only linearly, producing a learned policy that regresses toward more moderate controls. This turns out to be a feature rather than a bug: the moderation is what preserves ensemble diversity in closed loop, as shown in Chapter 6.

Online phase: corrected deployment. The trained network is deployed within a corrected particle filter using `pybeeda` [5]. The ensemble summary $\phi(\mathcal{E})$ is computed once at the start of each assimilation window and held fixed during propagation, preserving conditional particle independence. The Girsanov weight correction (1.6.3) is accumulated at every integration substep. The deployment cost is a single MLP forward pass per observation step per particle, regardless of the number of control substeps the oracle would have used.

Baseline comparisons. All proposed controllers are evaluated against the uncontrolled bootstrap PF, the relaxation nudged PF of van Leeuwen [70] with proper Girsanov correction, and the oracle nudged PF using the full Feynman-Kac Monte Carlo optimal control online. The oracle serves as the performance ceiling. Performance is measured via time-averaged normalized RMSE, normalized ESS (nESS), and wall-clock time per observation step across 10 independent Monte Carlo trials on both Lorenz-63 and Lorenz-96 at $K = 8$.

1.8 Literature Review

This section situates the present work within the relevant literature, organized by theme: nonlinear Bayesian filtering and sequential Monte Carlo; particle degeneracy and the curse of dimensionality; controlled and nudged particle filters; the path-integral and HJB framework for optimal nudging; amortized and learning-based approaches to filtering; and chaotic dynamical systems as benchmarks.

1.8.1 Nonlinear Bayesian Filtering and Sequential Monte Carlo

The theoretical foundations of nonlinear filtering were laid in the 1960s through the Kushner-Stratonovich and Zakai equations, which describe the evolution of the conditional distribution of a Markov signal given continuous observations [40, 74]. The Kalman filter [32] provides the exact Bayesian solution under linear-Gaussian dynamics; the extended Kalman filter (EKF) linearizes about the current estimate [30]; the unscented Kalman filter (UKF) uses a sigma-point approximation [31]; and the ensemble Kalman filter (EnKF) [20] propagates a Monte Carlo ensemble with a linear analysis step. These Gaussian-family methods dominate operational weather forecasting due to their scalability, but their Gaussian assumptions are violated in chaotic regimes where the filtering distribution is genuinely multimodal.

Sequential Monte Carlo (SMC) methods, also called particle filters, were popularized by Gordon, Salmond, and Smith [26] and given rigorous theoretical foundations by Del Moral [51]. The standard reference is Doucet, de Freitas, and Gordon [18]. SMC methods are consistent in the limit $N \rightarrow \infty$ without Gaussian assumptions, but are computationally expensive in high dimensions.

1.8.2 Particle Degeneracy & the Curse of Dimensionality

The fundamental limitation of particle filters under high-dimensional likelihoods was analyzed rigorously by Snyder et al. [63] and Bengtsson, Bickel, and Li [9]. In a linear-Gaussian setting, the required particle count grows exponentially in the observation dimension to maintain non-degenerate weights. Localization strategies restrict the influence of each observation to nearby state variables [27, 56]. Tempering strategies [11] introduce the likelihood gradually. Implicit sampling [17] maps particles deterministically to observation-consistent regions. Equivalent-weights approaches [70, 2] modify the proposal so all particles receive approximately equal weights at the observation

step. Transport methods [60, 65] use deterministic couplings to move particles toward the posterior without importance weights.

1.8.3 Controlled and Nudged Particle Filters

The auxiliary particle filter of Pitt and Shephard [55] uses a preliminary likelihood appraisal to resample before propagation, concentrating computational effort on high-weight regions. The controlled diffusion perspective, in which a continuous-time control term steers particles toward observations between assimilation steps, was developed by Lingala, Namachchivaya, Perkowski, and Yeong [44, 45]. The optimal control minimizing weight variance derives from a stochastic optimal control problem connecting particle nudging to minimum-variance importance sampling [34, 23].

Yeong, Beeson, Namachchivaya, and Perkowski [72] extended this framework to multiscale chaotic systems and showed that optimal nudging significantly reduces weight degeneracy when observations are sparse. Beeson and Namachchivaya [8] introduced proposals based on future right-singular vectors of the linearized dynamics. The intermediate-resampling nudged particle filter [7] adds a deterministic resampling step within the assimilation window to prevent instability from strong controls. The hierarchical variant introduces a leader-follower decomposition in which a small number of leader particles solve the expensive nonlinear control problem while follower particles receive cheaper corrections. Karampela [35] further extended this hierarchy with 4DVar-enhanced observation back-propagation.

1.8.4 Hamilton-Jacobi-Bellman Equations & Path-Integral Control

The stochastic optimal control problem underlying optimal nudging is related to the path-integral framework of Fleming [22, 23] and Kappen [34]. For control problems with control cost quadratic in the control magnitude and a log-linear terminal cost, the HJB equation linearizes via a log-transformation, yielding a solution expressible as a path integral that can be approximated by Monte Carlo sampling. This representation enables Monte Carlo approximation of the optimal control without solving the HJB PDE directly, and is the basis of the expert data generation pipeline used throughout this thesis.

1.8.5 Amortized & Learning-Based Approaches to Filtering

Variational methods for SMC, including differentiable particle filters [52, 42, 48], embed the particle filter in an end-to-end differentiable computation graph and train the proposal by optimizing a variational lower bound on the marginal likelihood. These learn globally over the filtering distribution but are expensive at training time and require differentiability through the resampling step.

Neural network proposals trained by supervised imitation of near-optimal importance weights have also been explored in the broader SMC literature [33]. The key challenge, consistently identified across these works, is that proposal accuracy does not automatically translate into filtering performance: the interaction between proposal concentration and weight collapse must be explicitly addressed. Flow-based and transport methods [60, 65] construct deterministic measure-transport maps that push forward a reference distribution to the posterior without importance weights, but are expensive to construct in high dimensions. Score-based and diffusion approaches to particle proposals have recently been proposed [24, 13]. As demonstrated in the JIW [12] and analyzed in Section 1.5, diffusion models are structurally misaligned with the per-particle independence requirement of standard importance sampling, which motivates the supervised regression approach adopted in this thesis.

1.8.6 Chaotic Systems as Filtering Benchmarks

The Lorenz-63 model [46] is a three-dimensional deterministic system exhibiting a chaotic strange attractor with two lobes. Its filtering properties are well understood: the positive maximum Lyapunov exponent ($\lambda_1 \approx 0.91$, error doubling time $\tau_d \approx 0.76$ TU) causes rapid forecast divergence between observations. It has been used as a benchmark for particle filter development since at least the early work on nonlinear filtering methods [50] and is the primary testbed in this thesis.

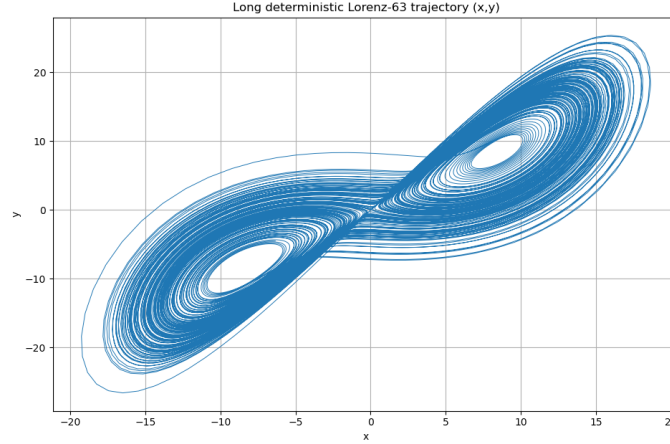


Figure 1.1: Long deterministic Lorenz-63 trajectory (x - y projection) confirming that the simulation reaches and explores the chaotic attractor. The butterfly structure with two lobes and irregular switching between them is visible. This rollout is used only for visualization; the training dataset is constructed from a low-discrepancy subsampling of attractor states rather than a continuous trajectory.

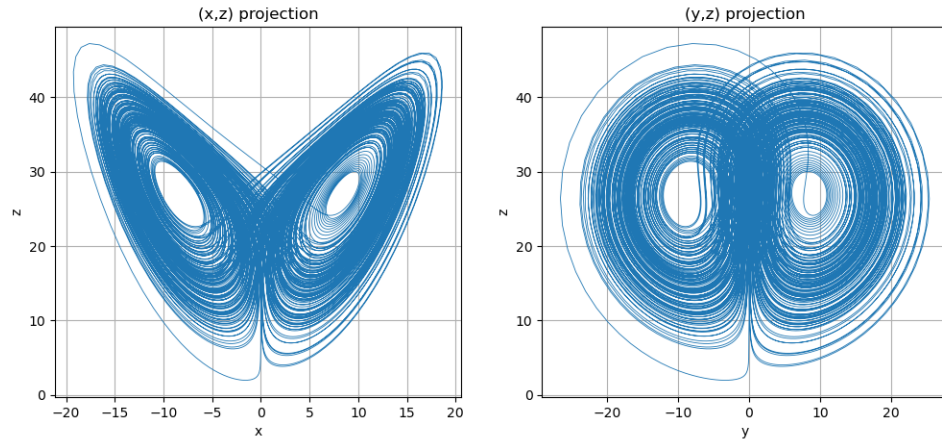


Figure 1.2: Long deterministic Lorenz-63 trajectory (x - z and y - z projections) showing attractor geometry and lobe-switching behavior. The lobe switches visible here are the primary failure events for the uncontrolled particle filter: particles on the wrong lobe receive near-zero likelihood weight at the subsequent observation step.

The Lorenz-96 model [47] is a K -dimensional system mimicking midlatitude at-

mospheric dynamics with chaotic multiscale behavior. It is the primary benchmark for high-dimensional data assimilation, used extensively in studies of EnKF [3, 54], localization [27], and particle methods [63, 70, 72]. Its Lyapunov exponent spectrum scales with K [37], making it a natural testbed for dimension scaling. This thesis evaluates the proposed pipeline on Lorenz-96 at $K = 8$ with forcing $F = 8$ as a proof-of-concept scaling demonstration. The $K = 8$ setting has 3 to 4 positive Lyapunov exponents and an error doubling time of approximately 1.7 TU [35], making it substantially harder than Lorenz-63 while remaining computationally tractable for oracle data generation and closed-loop evaluation. The structural simplicity of L96 at small K relative to its apparent complexity is discussed in Chapters 7 and 8, drawing on the observation of [59] that near-linear relationships govern much of L96’s dynamics.

1.9 Contributions

This work makes the following contributions.

1. **A complete imitation learning pipeline for particle nudging.** An end-to-end framework is developed in which a Feynman-Kac Monte Carlo oracle generates expert control demonstrations offline, a multilayer perceptron is trained by behavioral cloning to reproduce those demonstrations, and the trained network is deployed within a corrected particle filter that accumulates the Girsanov weight correction at every integration substep. The pipeline is implemented using `pybeeda` [5] and `pybeebase` [6] and is evaluated in closed loop on both Lorenz-63 ($d = 3$, $N = 32$ particles) and Lorenz-96 ($K = 8$, $N = 16$ particles), achieving $70\times$ and $27\times$ wall-clock speedups over the oracle respectively, where the L96 figure is conservative and reflects a reduced oracle inner-loop configuration rather than a structural limit.
2. **Corrected controlled particle filter baselines.** All controlled filter implementations include the Girsanov importance-weight correction, weighted posterior mean estimation, and normalized ESS computation. The corrected uncontrolled PF, relaxation nudged PF, and oracle nudged PF serve as the reference baselines against which all learned controllers are evaluated.
3. **Ensemble-aware residual controller architecture.** A `ResidualEnsembleController` is designed with three levels of information: per-particle base features, particle-relative features (innovation and deviation

from the weighted ensemble mean), and a permutation-invariant global context vector from weight-aware pooling over all particles. A residual structure with transfer initialization from a pretrained per-particle checkpoint ensures that the ensemble correction adds to, rather than overwrites, per-particle control quality.

4. **Two-stage weight-aware training protocol.** A two-stage training protocol is developed in which Stage 1 minimizes a Huber behavioral cloning loss and Stage 2 fine-tunes with a weight-balance penalty derived from the Girsanov formula in raw physical units. The penalty directly minimizes the variance of predicted post-control importance weights, connecting the training objective to the zero-variance property of the optimal controller.
5. **Ablation studies and robustness evaluation.** Ablation studies isolate the contributions of the innovation feature, the deviation-plus-pooling ensemble context, and the weight-balance penalty. Robustness is evaluated across observation noise levels ($\sigma_{\text{obs}} \in \{1.0, 2.0, 4.0\}$) and particle counts ($N \in \{16, 32, 64, 128\}$), confirming that the learned controller generalizes without retraining. A compute-constrained hypothesis test with ensemble-coordinated IRnPF oracle labels is conducted to assess whether label design can improve ensemble health beyond what the architecture alone provides.

1.10 Impact and Relevance to Robotics

The methods developed in this thesis are motivated by atmospheric data assimilation (across the two Lorenz systems), but are not specific to it. The core problem, estimating the hidden state of a nonlinear chaotic system from sparse, noisy observations and making real-time control decisions under that uncertainty, is structurally identical to problems that appear throughout robotics and autonomous systems.

State estimation under uncertainty. Particle filters are the principled tool for nonlinear, non-Gaussian state estimation, and they underpin localization, tracking, and perception in robotics. A mobile robot estimating its pose from LiDAR returns, a spacecraft reconstructing its orbit from intermittent ground contacts, and a drone tracking a moving target through GPS dropout all face the same degeneracy challenge studied in this thesis: the state space is high-dimensional relative to the particle budget, observations are sparse in time, and the dynamics are sufficiently nonlinear that Gaussian approximations degrade. The nudging framework developed

here, which steers particles proactively toward observation-consistent regions while preserving ensemble diversity, directly addresses this degradation in a way that is agnostic to the underlying physical system.

Learning control laws for complex environments. A second connection is to learning-based control more broadly. The imitation learning pipeline developed in this thesis follows the same behavioral cloning paradigm used to train robot controllers from expert demonstrations: an expensive expert (here the Feynman-Kac oracle; in robotics, a human teleoperation or a model-predictive controller) generates labeled demonstrations, and a neural network learns to reproduce them at deployment time for a fraction of the online cost. The specific structural constraints that make this work, including importance-sampling validity via Girsanov correction, ensemble-aware conditioning, and the heavy-tailed Huber regression that produces implicitly conservative policies, have direct analogues in robot learning: action distributions are often heavy-tailed, deployment must respect physical constraints (the analogue of Girsanov’s cost), and ensemble-aware reasoning about uncertainty is central to robust manipulation and navigation.

Computational tractability for real-time deployment. A practical requirement for any robotic estimator is that it runs in real time. The oracle nudging methods that inspire this work are theoretically optimal but require tens of thousands of SDE integrations per observation step, making them unusable in any time-critical setting. The learned MLP surrogate developed here reduces this to a single forward pass per particle, achieving $70\times$ speedup on Lorenz-63 and $27\times$ on Lorenz-96 while matching or exceeding the oracle’s tracking accuracy. This speedup is not incidental: it is what makes the approach viable for deployment on embedded processors, onboard spacecraft, or real-time robotic platforms where oracle-level computation is unavailable.

The robotics relevance of this work is therefore threefold: it advances the state of the art in particle filter design for nonlinear systems, it demonstrates a complete imitation learning pipeline that is transferable to robot control settings, and it produces a computationally efficient controller that meets the real-time constraints that autonomous systems impose. Chapters 5 and 6, and Appendices A, D, E, and F in particular contain the architecture design, training methodology, and deployment analysis that are most directly applicable to robotic state estimation and learned control law development.

1.11 Thesis Organization

Chapter 2 presents the engineering design framework, including formal design requirements, the iterative design process from diagnostic baseline through the final ensemble-aware imitation learning framework, and verification of all quantitative requirements against experimental results.

Chapter 3 develops the mathematical foundations: the signal and observation models, the particle filter and its degeneracy failure mode, the stochastic optimal control formulation that yields the oracle nudging law, and the Girsanov change-of-measure correction that makes controlled filters valid importance samplers.

Chapter 4 describes the offline expert demonstration dataset generation pipeline for both systems, including attractor sampling, degeneracy regime construction, oracle control computation, and the Girsanov label generation.

Chapter 5 presents the ensemble-aware MLP architecture, the permutation-equivariant encoder and weight-aware pooling module, and the two-stage Huber-plus-penalty training protocol.

Chapter 6 reports closed-loop filtering experiments on Lorenz-63, with trajectory diagnostics, aggregate timeseries, and detailed analysis of the oracle gap mechanism.

Chapter 7 presents the scaling investigation on Lorenz-96 at $K = 8$, organized around four feasibility questions and explaining the structural reasons why all filters perform worse on L96 than on L63.

Chapter 8 presents ablation and robustness studies, including the control clipping ablation, noise robustness, particle count generalization, and the IRnPF oracle label hypothesis test.

Chapter 9 summarizes findings, states limitations, and outlines directions for future work including production-scale IRnPF retraining, scaling to $K = 40$, and localized architectures.

Appendix A provides self-contained informal definitions of key concepts for readers less familiar with stochastic calculus and Bayesian inference. Appendix B provides a deeper background on the Lorenz 63 and Lorenz 96 models. Appendix C walks through the derivation and rationale behind the Girsanov weight correction. Appendix D documents the dataset generation pipeline at the implementation level. Appendix E gives pseudocode for all five filter variants. Appendix F collects all verified hyperparameters, architecture specifications, and implementation details with the goal of reproducibility of results.

Chapter 2

Engineering Design

2.1 Project Goals & Scope

This thesis addresses a central challenge in computational data assimilation: particle filters for chaotic nonlinear systems suffer from rapid weight degeneracy that causes ensemble collapse and filter divergence, and the classical remedy of optimal particle nudging is too computationally expensive for real-time deployment. The engineering design problem is to produce a learned control system that imitates optimal nudging online at negligible computational cost while preserving the importance-sampling validity that makes particle filtering theoretically sound.

The approach is framed as imitation learning. An expensive Feynman-Kac Monte Carlo oracle generates expert control demonstrations offline across the chaotic attractor. A neural network is then trained via behavioral cloning to reproduce the oracle’s per-particle control policy, replacing the full Monte Carlo computation with a single forward pass at deployment. The Girsanov Radon-Nikodym correction is accumulated along every controlled trajectory to ensure that the learned controller does not invalidate the importance weights that underpin the filter’s statistical correctness.

The design encompasses four interconnected components: an offline dataset generation pipeline, a neural network architecture subject to permutation equivariance and conditional particle independence, a training objective balancing oracle imitation against ensemble-level weight-balance regularization, and a corrected deployment pipeline accumulating the Girsanov correction at every integration substep. The design is realized entirely in software, building on the `pybeeda` and `pybeebase` codebases [5, 6]. The primary testbed is Lorenz-63 [46], with scalability evaluated on

Lorenz-96 [47] at $K = 8$ state variables.

2.2 Design Requirements & Constraints

Table 2.1 summarizes the design requirements and constraints across four categories: functional requirements for scientific validity, performance requirements for filtering improvement, computational constraints bounding online cost, and reproducibility constraints for independent verification.

Table 2.1: Design requirements and constraints.

Category	Requirement	Constraint / Acceptance Criterion
Functional	Importance-sampling consistency	Girsanov weight correction accumulated at every integration substep; no controlled filter may omit this correction
Functional	Particle independence	Ensemble summary computed once per assimilation window and held fixed during propagation
Functional	Permutation equivariance	Permuting particle labels must permute output controls by the same permutation; equivariance enforced by construction
Functional	Modular expert interface	Expert labeling must be swappable without modifying the learning pipeline; any map $(x_0, \Delta t, y) \mapsto u^*$ is acceptable
Performance	nRMSE improvement	Best learned controller must achieve lower nRMSE than the uncontrolled PF on Lorenz-63, measured over 10 Monte Carlo trials
Performance	nESS maintenance	Best learned controller must maintain nESS above the resampling threshold (0.5) throughout all trials

Category	Requirement	Constraint / Acceptance Criterion
Performance	Oracle gap reported	RMSE gap between best learned controller and oracle nPF must be reported and interpreted; no specific bound required
Performance	Scalability	Pipeline must be demonstrably executable on Lorenz-96 at $K = 8$; learned controller evaluated in closed loop
Computational	Online cost	No more than a single MLP forward pass per particle per observation step; no online optimization permitted
Computational	Dataset generation	Lorenz-96 dataset generation must complete within available compute (Della HPC or Google Colab)
Computational	Memory	All datasets must fit in available storage; checkpoints loadable on a standard workstation
Reproducibility	Seed control	All trials must use deterministically staggered seeds; all filter variants within a trial share truth, observations, and initial ensemble
Reproducibility	Code availability	All scripts version-controlled in pybeebase [6] and available upon request
Reproducibility	Checkpoint portability	All checkpoints must include normalizer statistics, enabling evaluation without access to the original training data

2.3 Evaluation Criteria

Three primary metrics are used to evaluate the design, following conventions in the particle filtering and data assimilation literature [72, 35]. Full definitions appear in Section 3.5.

1. **Time-averaged normalized RMSE** ($\bar{e}$) measures state estimation accuracy as the mean over observation steps of $\|x^{\text{true}} - \hat{x}\|_2 / \|x^{\text{true}}\|_2$. A design is successful if it reduces $\bar{e}$ relative to the uncontrolled PF.
2. **Normalized effective sample size** ($\overline{\text{nESS}}$) measures ensemble health as the mean over observation steps of N_{eff}/N where $N_{\text{eff}} = 1 / \sum_i (w^{(i)})^2$. A design is successful if it keeps nESS above the resampling threshold across all trials.
3. **Wall-clock time per observation step** measures online cost. The learned controller must be substantially faster than the oracle nPF and remain bounded as N grows.

All metrics are reported as mean and standard deviation across $N_{\text{trials}} = 10$ independent Monte Carlo trials.

2.4 Final Design

The final design is an ensemble-aware imitation learning framework for particle nudging control, consisting of four integrated components.

Component 1: Offline expert data generation. The oracle nudged particle filter from `pybeeda` [5] generates labeled training pairs $(x_0, \Delta t, y, u^*)$ across the chaotic attractor. Attractor states are sampled via greedy farthest-point low-discrepancy pruning, producing $|\mathcal{X}_{\text{d}}| = 5,000$ states. The Lorenz-63 dataset contains 4,800 valid labeled examples generated on Della HPC. Lorenz-96 dataset generation runs on Google Colab in approximately 15 to 30 minutes, producing 400 examples.

Component 2: Residual ensemble controller architecture. The learned controller uses a `PerParticleEncoder` that maps a 15-dimensional per-particle input $(x^{(i)}, y, \Delta t, \sigma_{\text{obs}}, \iota^{(i)}, \delta^{(i)}, w^{(i)})$ to a 64-dimensional embedding $h^{(i)}$ via a $[15 \rightarrow 128 \rightarrow 128 \rightarrow 64]$ MLP, applied identically to all particles. A `WeightAwarePooling` module aggregates embeddings into a 128-dimensional global context $g = [\bar{h}; h_{\text{max}}]$. The final

control is $u^{(i)} = \phi_{\text{base}}(h^{(i)}) + s \cdot \phi_{\text{corr}}(h^{(i)}, g)$ where s is a learnable scalar initialized to 0.1. Total parameter count: approximately 110,000.

Component 3: Two-stage training with weight-balance regularization. Stage 1 minimizes Huber loss ($\delta = 1.0$) between predicted and oracle controls in normalized space using Adam at 3×10^{-4} with cosine annealing and early stopping. Oracle controls are clipped to $[-100, 100]$ before normalization. Stage 2 fine-tunes with $\mathcal{L} = \mathcal{L}_{\text{Huber}} + \lambda \mathcal{L}_{\text{penalty}}$, where $\mathcal{L}_{\text{penalty}}$ is the variance of predicted post-control weights via the Girsanov formula in raw physical units. The Pareto-optimal $\lambda^* = 0.1$ is selected by offline validation.

Component 4: Corrected particle filter deployment. The learned controller overrides `pybeeda`’s `_calculate_nudging` method. The ensemble summary $(\bar{x}_k, \delta^{(i)}, g)$ is computed once per assimilation window and held fixed during propagation. The Girsanov log-weight correction accumulates at every integration substep. Resampling triggers at $\text{nESS} < 0.5$ using systematic resampling.

2.5 Engineering Standards

Table 2.2 lists the engineering standards and established practices governing this design. Standards are mapped to the ABET Student Outcomes [1] (SO1-SO7) that each directly addresses, following the MAE IW guidelines for senior thesis projects.

Table 2.2: Engineering standards.

ABET SO	Standard / Principle	Source / Reference	Application in This Design
SO1: formulate and solve complex engineering problems	Importance sampling validity: all controlled particle filters must include the Girsanov Radon-Nikodym correction	Girsanov [36]; applied in particle filtering by [72, 35]	Every controlled filter (relaxation, oracle, learned) accumulates the discrete Girsanov correction at every integration substep. Filters lacking this correction are excluded from evaluation baselines.

ABET SO	Standard, Principle	Source	Application in This Design
SO1, SO2: apply mathematics and design systems with appropriate invariances	Permutation equivariance for particle-indexed functions: a controller operating on an ensemble must be invariant to particle label ordering	DeepSets framework [73]; particle filter independence [18]	Encoder applied identically to all particles via broadcasting; pooling uses only permutation-invariant operations (weighted mean, element-wise max). Equivariance is enforced by construction, not verified post hoc.
SO1, SO6: apply principles of probability and stochastic processes; analyze data	Conditional particle independence: particle SDEs must remain conditionally independent during propagation to preserve the importance sampling structure	Stochastic optimal control for filtering [45, 72]	Ensemble summary is computed once per assimilation window and held fixed during propagation. Updating the summary mid-window would couple particle SDEs and invalidate the per-particle Girsanov correction.
SO6: develop and conduct appropriate experimentation; interpret data	Reproducible Monte Carlo evaluation: closed-loop filter comparisons must control for stochastic variation across methods	Standard practice in data assimilation literature [35, 72]	All trials use deterministically staggered seeds (prime step 997). All filter variants within a trial share the same truth trajectory, observation sequence, and initial ensemble, so RMSE differences reflect only the control policy.

ABET SO	Standard, Principle	Source	Application in This Design
SO4: recognize ethical and professional responsibilities; responsible data handling	Training data leakage prevention: test and validation set statistics must not be exposed to the training pipeline	Standard machine learning practice	Normalizer statistics (mean and std per feature) are fitted on the training split only and embedded in the checkpoint. Validation and evaluation splits use training-split statistics, never refitted.
SO3: communicate effectively; engineering documentation and reproducibility	Checkpoint completeness: model artifacts must be self-contained and independently reproducible	ANSI/NISO 239.18-2005, Scientific and Technical Reports; Princeton MAE IW guidelines	Each checkpoint stores the model state dict and the normalizer statistics together, enabling re-evaluation on any machine without access to the original training data or dataset.
SO7: acquire and apply new knowledge; use of modern engineering tools	Scientific software practices: version control, documented hyperparameters, reproducible random seeds	Princeton MAE IW guidelines; general software engineering practice	All generation and evaluation scripts are version-controlled in <code>pybeebase</code> [6]. Random seeds documented in Appendix F. Hyperparameters sourced from training notebook source code.

ABET SO	Standard, Principle	Source	Application in This Design
SO2: apply engineering design; iterative design meeting specified needs	Iterative imitation learning design: each design iteration must be motivated by a quantified failure mode from the previous iteration, and success criteria must be defined before the iteration begins	Engineering design process [49]; imitation learning methodology [61]	Four design iterations documented in Section 2.6 and Table 2.3, each with a stated failure mode, key design change, and quantified outcome.

2.6 Iterative Design Process

The design evolved through four iterations, each motivated by a specific failure mode identified in the previous iteration. The organizing principle throughout was the imitation learning framing: at each stage, the question was not only whether the learned policy had low offline imitation error, but whether it produced closed-loop filtering behavior consistent with importance-sampling validity. Offline accuracy is necessary but not sufficient; the binding constraint is the Girsanov correction that links learned controls to valid importance weights. Table 2.3 summarizes the full iterative process.

Iteration 1: Establishing the diagnostic baseline (JIW). The JIW [12] compared conditional diffusion models and supervised regression on Lorenz-63, using a saturation heuristic as the expert policy and deploying without the Girsanov correction. Supervised regression achieved low offline imitation error but degraded closed-loop RMSE by a factor of 4 relative to the uncontrolled PF. This established the central diagnostic: the failure was not one of policy expressivity but of importance-sampling incompatibility. The JIW is the starting point of the iterative process, not its primary contribution; it identified what needed to be fixed.

Iteration 2: Correcting the imitation target and deployment. Two changes were made simultaneously. First, the expert policy was upgraded from the saturation

heuristic to the Feynman-Kac Monte Carlo optimal control, producing physically correct and oracle-consistent training labels. Second, the Girsanov weight correction was added to the deployment pipeline via `pybeeda`’s built-in machinery, restoring importance-sampling validity. Oracle controls were clipped to $[-100, 100]$ before normalization to remove heavy-tailed label noise. With these corrections, the *per_particle* SR controller achieves nRMSE 0.16 ± 0.03 , a 77% reduction over the uncontrolled PF and $4\times$ better than the oracle nPF. This is the thesis’s primary result: a correct imitation learning setup, not a more expressive architecture, is what drives the improvement.

Iteration 3: Extending to ensemble-aware imitation. Having established a strong per-particle baseline, the design was extended to condition on collective ensemble information. Innovation $\iota^{(i)}$, deviation $\delta^{(i)}$, and a weight-aware pooled global context g were added, together with a residual architecture initialized from the *per_particle* checkpoint to guarantee monotone improvement at initialization. *Ensemble S1* (nRMSE 0.64 ± 0.10) and *residual_ensemble* (nRMSE 0.45) both underperform the per-particle baseline at $N = 32$. The finding is that ensemble-awareness must come from the expert oracle (training data), not from the architecture alone: an oracle that generates ensemble-independent labels cannot teach ensemble-coordinated behavior regardless of pooling depth.

Iteration 4: Weight-balance regularization, scaling, and oracle exploration. Stage 2 fine-tuning with a differentiable Girsanov weight-balance penalty was added, sweeping $\lambda \in \{0.1, 1.0, 10.0, 100.0\}$ and selecting $\lambda^* = 0.1$ on the Pareto frontier of nRMSE vs nESS. *Ensemble S2* achieves the highest nESS of any variant (0.90), confirming the penalty is a meaningful training signal for weight diversity. The full pipeline was then scaled to Lorenz-96 ($K = 8$), where *per_particle* SR matches the oracle in nRMSE (0.660 vs 0.623) at $27\times$ lower cost. A preliminary investigation of the intermediate-resampling nudged particle filter (IRnPF) as an alternative expert oracle demonstrated that ensemble-coordinated labels raise nESS to 0.954 on Lorenz-63, the highest of any variant, motivating production-scale follow-up.

Table 2.3: Iterative process.

#	Design Stage	Key Change	Failure / Limitation Identified	Outcome
1	JIW: diagnostic baseline; diffusion vs. SR, no Girsanov	Two imitation learning paradigms; saturation heuristic expert; no Girsanov correction at deployment	SR degrades closed-loop RMSE $4\times$ vs PF; importance-sampling incompatibility identified as root cause, not policy expressivity	Diagnostic baseline established; diffusion eliminated; Girsanov correction identified as critical missing component
2	Corrected imitation learning: per-particle SR	Feynman-Kac oracle as expert; Girsanov correction via <code>pybeeda</code> ; ± 100 label clipping; Huber loss	Per-particle policy cannot regulate relative weight evolution across the ensemble; pooling not yet explored	nRMSE 0.16 ± 0.03 ; 77% reduction over PF; $4\times$ better than oracle nPF; implicit Girsanov regularization identified

#	Design Stage	Key Change	Failure / Limitation Identified	Outcome
3	Ensemble-aware imitation: residual controller	Innovation $\iota^{(i)}$, deviation $\delta^{(i)}$, pooled context g ; residual architecture with transfer init from per-particle checkpoint	Huber imitation loss does not penalize weight variance; oracle labels carry no ensemble-coordination signal	Ensemble S1 nRMSE 0.64; residual 0.45; both trail per-particle; ensemble-awareness must come from oracle, not architecture
4	Weight-balance regularization; L96 scaling; IRnPF oracle exploration	Stage 2 Girsanov penalty with $\lambda^* = 0.1$; scaled to L96 $K = 8$; preliminary IRnPF hypothesis test (200 L63 examples)	Spatial locality of L96 not exploited; IRnPF dataset too small for definitive comparison ($24\times$ fewer examples than nPF)	Ensemble S2 nESS 0.90 (highest); L96 SR matches oracle at $27\times$ lower cost; IRnPF raises nESS to 0.954, motivating production retraining

2.7 Verification Against Design Requirements

Table 2.4 maps each requirement from Table 2.1 to the experimental result that verifies it.

Table 2.4: Verification of design requirements against experimental results.

Requirement	Verification Method	Result
Importance-sampling consistency	Girsanov correction in all controlled filters; verified by inspection of MLNudgedPF source	Satisfied by design
Particle independence	Ensemble summary fixed per window; verified by <code>compute_ml_control</code> call structure	Satisfied by design
Permutation equivariance	Encoder via broadcasting; pooling by mean and max; verified by architecture inspection	Satisfied by design
Modular expert interface	Oracle replaced via <code>_calculate_nudging</code> override; nPF and IRnPF both tested as expert sources	Satisfied by design
nRMSE improvement over PF	Table 6.1, Chapter 6	<i>per_particle</i> nRMSE 0.16 ± 0.03 vs PF 0.68 ± 0.09 ; 77% reduction [Satisfied]
nESS above resampling threshold	Table 6.1, Chapter 6	All variants maintain nESS > 0.5 throughout all trials; best variant (<i>ensemble_s2</i>) achieves nESS 0.90 [Satisfied]
Oracle gap reported	Section 6.3, Chapter 6	<i>per_particle</i> nRMSE 0.16 vs oracle 0.60; SR is $4\times$ better at $70\times$ lower cost [Satisfied]

Requirement	Verification Method	Result
Scalability to Lorenz-96	Table 7.1, Chapter 7	<i>per_particle</i> nRMSE 0.660 ± 0.104 vs PF 1.178 ± 0.068 ; 44% reduction; pipeline runs on Colab [Satisfied]
Online cost: single forward pass	Single trial timing; Table 6.1	L63: SR 438 ms/step vs oracle 30,518 ms/step; L96: 120 ms/step vs 3,253 ms/step [Satisfied]
Dataset generation feasible	Sections 4.2 and 7.2	L63: 4,800 examples on Della; L96: 400 examples on Colab in 15–30 min [Satisfied]
Seed control and shared truth	Verified by <code>run_trial</code> seed structure (prime step 997)	Satisfied by design
Code availability	Scripts in <code>pybeebase</code> [6]	Satisfied
Checkpoint portability	Normalizer saved with checkpoint; verified by <code>nm_to_numpy</code> loading logic in eval script	Satisfied by design

All quantitative performance requirements are satisfied. The nESS requirement deserves a note: the *per_particle* SR controller achieves nESS 0.85, below the uncontrolled PF’s 0.98, because applied controls reduce weight diversity relative to uncontrolled propagation. This is an acceptable tradeoff given the 77% RMSE reduction: nESS remains well above the resampling threshold of 0.5 throughout all trials, and *ensemble_s2* improves nESS to 0.90 at a modest RMSE cost. The modular expert interface requirement is additionally verified by the IRnPF oracle exploration in Chapter 8: swapping the nPF expert for the IRnPF expert required no changes to the learning pipeline, only to the data generation script.

All of the code in this thesis can be found in the `sonal_ml_for_da` branch within the `pybeebase` [6] repository linked here.¹

¹For access to the repository, please reach out to Professor Beeson directly at ryne@princeton.edu.

Chapter 3

Mathematical Background

This chapter builds the mathematical foundation used throughout the thesis. It is organized to mirror the pipeline of the thesis itself: first, the models that describe how the true system state evolves and how it is observed; second, the particle filter and the core problem it faces; third, the stochastic optimal control formulation that motivates the nudged particle filter; and fourth, the Girsanov correction that makes controlled filters statistically valid.

Each section begins with a plain-language explanation of what the math is describing and why it matters for this thesis. Readers who seek more insights on terms like *chaos*, *Brownian motion*, or *importance sampling* before working through the equations are encouraged to first read Appendix [A](#). The formal material here follows [\[72\]](#) and [\[35\]](#) closely.

The key experimental parameters that appear throughout the thesis are collected here for reference. On Lorenz-63, all experiments use $N = 32$ particles, an observation interval of $\Delta T = 0.1$ TU, and observation noise $\sigma_{\text{obs}} = 2.0$. On Lorenz-96 ($K = 8$, $F = 8$), experiments use $N = 16$ particles, $\Delta T = 0.05$ TU, and $\sigma_{\text{obs}} = 1.0$. These values matter because the filtering difficulty scales strongly with both the ratio of ΔT to the error doubling time and with the ratio of observation noise to process noise; the regime studied here places both systems in a regime where weight degeneracy develops steadily between updates.

3.1 Signal & Observation Models

3.1.1 Signal Model

The true state of the system, or the vector we are trying to estimate, evolves continuously over time. In Lorenz-63, this state is the three-dimensional vector (x_1, x_2, x_3) representing convective intensity and temperature variation in a simplified atmospheric model. We model this evolution as a stochastic differential equation (SDE): the state follows deterministic physics (the drift) plus a random perturbation (the noise). The noise represents unresolved physics and model uncertainty. Appendix A Section A.4 defines SDEs and Brownian motion informally for readers new to stochastic calculus.

Role in this thesis. The dispersion coefficient σ in this model appears directly in the Girsanov weight correction in Section 3.4. The drift b determines the Jacobian that appears in the optimal control gradient computation in Section 3.3.5. Both are needed to understand why learned controls must be calibrated carefully.

We model the true state $X_t \in \mathbb{R}^m$ as the solution to the Itô SDE

$$dX_t = b(X_t) dt + \sigma(X_t) dW_t, \quad X_0 = x \in \mathbb{R}^m, \quad (3.1.1)$$

where $b: \mathbb{R}^m \rightarrow \mathbb{R}^m$ is the drift, $\sigma: \mathbb{R}^m \rightarrow \mathbb{R}^{m \times k}$ is the dispersion coefficient, and W_t is a k -dimensional standard Brownian motion. The process noise covariance is $Q(x) := \sigma(x) \sigma(x)^\top$. In all experiments in this thesis, σ is a constant matrix and $Q = \sigma \sigma^\top$ is a fixed positive-definite matrix.

The noise serves a dual purpose. It models genuine uncertainty in the physics, and it ensures that particles in the filter maintain some spread even under strong nudging, preventing complete ensemble collapse. Without any process noise, all particles would eventually converge to the same trajectory, making the ensemble useless for representing posterior uncertainty.

3.1.2 Observation Model

We cannot observe the true state continuously or perfectly. Instead, a sensor takes noisy readings at discrete times. The observation model specifies how those readings relate to the true state. The quality of the observations – how precise they are and how frequently they arrive – fundamentally determines the difficulty of the filtering

problem. Sparse or noisy observations give the particle cloud more time to spread between updates, worsening weight degeneracy.

Observations arrive at discrete times $t_1 < t_2 < \dots$ according to

$$Y_k = h(X_{t_k}) + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, R), \quad \varepsilon_k \perp \varepsilon_j \text{ for } k \neq j, \quad (3.1.2)$$

where $h: \mathbb{R}^m \rightarrow \mathbb{R}^d$ is the observation operator and $R \in \mathbb{R}^{d \times d}$ is a positive definite noise covariance. In all experiments in this thesis, h is the identity matrix and R is diagonal.

The observation interval $\Delta T = t_{k+1} - t_k$ is a critical parameter. Longer gaps allow more chaos-driven divergence between the particle cloud and the true state, worsening weight degeneracy. In the Lorenz-63 experiments, $\Delta T = 0.1$ TU, which is close to the error doubling time of ≈ 0.76 TU, placing the filter in a moderately sparse regime. In the Lorenz-96 experiments, $\Delta T = 0.05$ TU. The observation noise parameters are $\sigma_{\text{obs}} = 2.0$ for Lorenz-63 (so $R = 4I_3$) and $\sigma_{\text{obs}} = 1.0$ for Lorenz-96 (so $R = I_8$). These values are not arbitrary: they are chosen so that the uncontrolled particle filter with $N = 32$ or $N = 16$ particles exhibits clear but not catastrophic degeneracy, making it a meaningful baseline for evaluating whether nudging helps.

3.1.3 Lorenz-63

The Lorenz-63 system [46] is the primary testbed in this work. It was originally introduced as a simplified model of atmospheric convection and has become a standard benchmark for chaotic filtering because it is low-dimensional enough to visualize but chaotic enough to stress particle filters. The deterministic drift is

$$f(x) = \begin{pmatrix} \sigma_L(x_2 - x_1) \\ x_1(\rho - x_3) - x_2 \\ x_1x_2 - \beta x_3 \end{pmatrix}, \quad (3.1.3)$$

with standard parameters $(\sigma_L, \rho, \beta) = (10, 28, 8/3)$. Here σ_L denotes the Lorenz parameter to avoid conflict with the dispersion coefficient σ in (3.1.1). For state estimation, this ODE is embedded in the SDE (3.1.1) with additive process noise:

$$dX_t = f(X_t) dt + \Sigma^{1/2} dW_t, \quad (3.1.4)$$

where $\Sigma = I_3$ (so $\sigma_{\text{dyn}} = 1.0$). The system has one positive Lyapunov exponent $\lambda \approx 0.91$, giving an error doubling time of $\tau_d = \ln 2/\lambda \approx 0.76$ TU [72].

Physically, the Lorenz-63 attractor has a distinctive butterfly shape with two lobes. The system switches irregularly between these lobes with no fixed period. This switching makes filtering difficult: a particle that ends up on the wrong lobe receives near-zero likelihood weight because its predicted observation is far from the actual measurement. No smooth path connects the two lobes, so no amount of gentle nudging can move a misplaced particle to the correct lobe quickly enough. The uncontrolled PF with $N = 32$ particles regularly loses track of the true lobe, which is the primary failure mode observed in Chapter 6.

The Jacobian of f is needed for the optimal control gradient computation in Section 3.3.5:

$$Df(x) = \begin{pmatrix} -\sigma_L & \sigma_L & 0 \\ \rho - x_3 & -1 & -x_1 \\ x_2 & x_1 & -\beta \end{pmatrix}. \quad (3.1.5)$$

3.1.4 Lorenz-96

The Lorenz-96 system [47] is used to test scalability. On a periodic lattice of K variables with cyclic indexing ($x_0 \equiv x_K$, $x_{K+1} \equiv x_1$), the drift is

$$\dot{x}_i = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F, \quad i = 1, \dots, K, \quad (3.1.6)$$

where F is a constant forcing. This thesis uses $K = 8$ and $F = 8$ as a proof-of-concept scaling experiment. At these parameters the system has approximately 3 to 4 positive Lyapunov exponents and an error doubling time of roughly 1.7 TU.

Physically, Lorenz-96 mimics a one-dimensional atmosphere at a fixed latitude. The K variables represent a generic atmospheric quantity (such as vorticity) at equally spaced grid points. The quadratic terms model advection, the linear term models dissipation, and F drives the system into chaos. The key difference from Lorenz-63 for filtering is that errors grow simultaneously across multiple state dimensions rather than being localized to a single lobe-switching event. This distributed error growth is harder to counteract with per-particle nudging and explains why all filter variants perform worse in relative terms on Lorenz-96 than on Lorenz-63.

The number of particles needed to avoid weight collapse is determined not by the full state dimension K but by the dimension of the unstable-neutral subspace: the subspace spanned by directions with non-negative Lyapunov exponents. For $K = 8$,

$F = 8$, this subspace has dimension 3 to 4, which is why $N = 16$ particles is sufficient for meaningful filtering [72, 35].

3.2 Particle Filtering

A particle filter maintains N simulated copies of the system state (particles), each representing one hypothesis about where the true state might be. This section describes how those particles are propagated, how they are scored against observations, and what can go wrong. Readers unfamiliar with importance sampling or the concept of effective sample size are encouraged to read Appendix A Section A.3 before proceeding.

The bootstrap particle filter described here serves as the initial baseline for evaluation across Chapters 6, 7, and 8. Its weight update equation (3.2.2) is extended in Section 3.4 to account for the controlled dynamics used in nudged filters. The resampling threshold carries over unchanged into all filter variants.

3.2.1 Sequential Importance Sampling

The particle filter approximates the posterior distribution $p(x_{t_k} \mid Y_{1:k})$ using a weighted ensemble $\{X_{t_k}^{(i)}, w_k^{(i)}\}_{i=1}^N$:

$$p_k^N(\mathrm{d}x \mid \mathcal{F}_k^Y) = \sum_{i=1}^N w_k^{(i)} \delta_{X_{t_k}^{(i)}}(\mathrm{d}x), \quad (3.2.1)$$

where $\mathcal{F}_k^Y$ is the sigma-algebra generated by all observations up to time t_k . In plain terms: the particle filter replaces the continuous posterior with a weighted sum of point masses. The filtered state estimate is the weighted mean $\hat{x}_k = \sum_i w_k^{(i)} X_{t_k}^{(i)}$.

In the bootstrap particle filter [18], particles are propagated under the signal dynamics (3.1.1) from t_{k-1} to t_k using Euler-Maruyama or a higher-order SDE integrator. When the observation Y_k arrives, weights are updated via the observation likelihood:

$$\tilde{w}_k^{(i)} = w_{k-1}^{(i)} p_k(Y_k \mid X_{t_k}^{(i)}), \quad (3.2.2)$$

with normalization $w_k^{(i)} = \tilde{w}_k^{(i)} / \sum_j \tilde{w}_k^{(j)}$. For the Gaussian observation model (3.1.2)

with identity observation operator, the likelihood is

$$p_k(Y_k | x) \propto \exp\left(-\frac{1}{2} (Y_k - x)^\top R^{-1} (Y_k - x)\right). \quad (3.2.3)$$

This is a Gaussian centered at the true state X_{t_k} . A particle whose state closely matches the observation receives a weight close to the maximum; one that is far away receives an exponentially suppressed weight. The exponent measures the squared Mahalanobis distance between the particle’s state and the observation, scaled by the inverse observation noise.

3.2.2 Weight Degeneracy

In chaotic systems, the spread of the prior distribution $p(x_{t_k} | Y_{1:k-1})$ grows rapidly between observations because the dynamics are sensitive to initial conditions. When the likelihood (3.2.3) is evaluated on this spread-out cloud, most particles receive negligible weight because they have drifted far from the true state. Weight concentrates on the one or two particles that happened to stay close. This is weight degeneracy, and it reduces the effective number of useful particles to near one.

The severity of degeneracy grows with the observation interval ΔT (more time for particles to spread), the noise ratio $\sigma_{\text{obs}}/\sigma_{\text{dyn}}$ (sharper likelihood means more weight concentration), and the number of positive Lyapunov exponents (more directions of exponential growth). The curse of dimensionality is not about the state dimension per se but about the dimension of the unstable-neutral subspace: each additional unstable direction requires an exponentially larger ensemble to maintain coverage [63, 9].

For Lorenz-63 with $N = 32$ particles, degeneracy is a real but manageable problem: the uncontrolled PF achieves nRMSE 0.68 ± 0.09 over 10 trials. Importantly, the diagnostic in Section 6.2 shows that increasing N to 300 produces essentially the same nRMSE, confirming that the bottleneck is degeneracy from the observation geometry, not insufficient sampling. This motivates nudging rather than simply adding more particles.

3.2.3 Effective Sample Size & Resampling

The effective sample size (ESS) quantifies how many equally-weighted particles would give the same statistical efficiency as the current weighted ensemble:

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N \left(w_k^{(i)}\right)^2}. \quad (3.2.4)$$

When all weights are equal, $N_{\text{eff}} = N$. When one particle has all the weight, $N_{\text{eff}} \rightarrow 1$. The normalized ESS is $\text{nESS} = N_{\text{eff}}/N \in (0, 1]$.

When N_{eff} falls below $N/2$, systematic (universal) resampling is triggered [18, 7]: new particles are drawn proportionally to their weights, duplicating high-weight particles and discarding low-weight ones, and all weights are reset to $1/N$. This is applied at every observation step where the threshold is met; no resampling is done mid-window during particle propagation. Resampling restores diversity in the sense that no single particle dominates the ensemble, but it also collapses diversity in a different sense: many particles become identical copies of the same high-weight particle and only diverge afterward through process noise. This tradeoff is why nudging is preferable to simply resampling more aggressively.

3.2.4 Classical Nudging: The Relaxation Particle Filter

Before introducing optimal nudging, it is useful to describe the simplest practical nudging scheme used as a classical baseline in this thesis: the relaxation particle filter of van Leeuwen [70]. Each particle is pushed toward the observation by a static linear gain:

$$u_{\text{relax}}^{(i)} = K \left(y_k - x^{(i)}\right), \quad K = \frac{\sigma_{\text{dyn}}^2}{\sigma_{\text{dyn}}^2 + \sigma_{\text{obs}}^2} I_m. \quad (3.2.5)$$

This gain is the Kalman update coefficient: it scales the nudge by the relative magnitude of process noise and observation noise, applying a stronger push when observations are precise relative to the dynamics. The gain is time-invariant: it does not account for how far away the next observation is or how the attractor geometry changes. Every particle receives the same gain regardless of its current position. Despite these limitations, the relaxation PF represents the classical approach to particle nudging and provides a meaningful intermediate baseline between the uncontrolled PF and the oracle-optimal nudged filter.

In all experiments in this thesis, the relaxation PF is evaluated with the full

Girsanov correction accumulated throughout the window. Without this correction, the filter would not be importance-consistent.

3.3 Optimal Control for Particle Nudging

This section derives the nudging law that is theoretically optimal for reducing weight variance. The derivation uses stochastic optimal control theory and results in a formula that must be evaluated by Monte Carlo sampling. Readers unfamiliar with HJB equations or the Feynman-Kac theorem are encouraged to read Appendix A Section A.7 before working through the derivations. The physical interpretation paragraphs are intended to make each step intelligible without prior exposure to control theory.

Rather than propagating particle i under the uncontrolled model and hoping it lands near the observation, we add a purposeful push $u_t^{(i)}$ to its drift: the particle is actively guided toward the region of state space that the upcoming observation prefers. The push must be designed carefully. Too weak, and it has no effect. Too strong, and all particles end up at the same spot, destroying the ensemble’s ability to represent uncertainty. The optimal control formulation makes this tradeoff precise by defining a cost function that penalizes both control effort and terminal mismatch with the observation.

The optimal control law is the theoretical benchmark against which all learned controllers are compared, and more importantly, it is the expert policy that generates the training data for the imitation learning framework in Chapter 5. The neural network is trained by behavioral cloning (i.e. imitation learning): it observes $(x^{(i)}, y, \Delta t, \sigma_{\text{obs}})$ tuples and learns to produce controls that match the oracle’s demonstrated controls u^* , replacing the expensive Monte Carlo computation with a single forward pass at deployment time.

3.3.1 Controlled Particle Dynamics

Instead of propagating under (3.1.1), each particle is propagated under the controlled SDE:

$$d\hat{X}_t^{(i)} = [b(\hat{X}_t^{(i)}) + u_t^{(i)}] dt + \sigma(\hat{X}_t^{(i)}) dW_t, \quad t \in (t_k, t_{k+1}], \quad (3.3.1)$$

following [72, 7]. The control $u_t^{(i)} \in \mathbb{R}^m$ is an additional drift added on top of the natural dynamics. It can depend on the particle's current state, the upcoming observation, and the time remaining until the observation.

3.3.2 Cost Function

The control is chosen to minimize the expected cost:

$$J(t_k, x, u) = \mathbb{E}_{t_k, x} \left[\int_{t_k}^{t_{k+1}} \frac{1}{2} u(s)^\top Q^{-1} u(s) \, ds + g(Y_{t_{k+1}}, \hat{X}_{t_{k+1}}) \right], \quad (3.3.2)$$

where $Q = \sigma \sigma^\top$ and the terminal cost is the negative log-likelihood

$$g(y, x) = \frac{1}{2} (y - h(x))^\top R^{-1} (y - h(x)). \quad (3.3.3)$$

The running cost $\frac{1}{2} u^\top Q^{-1} u$ penalizes large controls. The terminal cost g penalizes the particle's final position if it is far from the observation. The matrix Q^{-1} scales the running cost: directions with larger process noise are cheaper to control, while directions with smaller process noise are expensive. The balance between running and terminal cost depends on the relative magnitudes of Q and R and on the observation interval ΔT . Longer intervals require more aggressive early nudging to counteract greater chaos-driven divergence.

3.3.3 Hamilton-Jacobi-Bellman Equation

The value function $V(t, x) = \inf_u J(t, x, u)$ gives the minimum achievable expected cost from state x at time t . By the dynamic programming principle, V satisfies the HJB equation [72, 35]:

$$-\frac{\partial V}{\partial t} + H(t, x, \nabla_x V, \nabla_x^2 V) = 0, \quad V(t_{k+1}, x) = g(Y_{t_{k+1}}, x). \quad (3.3.4)$$

Maximizing over u in the Hamiltonian via the first-order condition $-\nabla_x V - Q^{-1}u = 0$ yields the optimal feedback control:

$$u^*(t, x) = -Q \nabla_x V(t, x). \quad (3.3.5)$$

The optimal nudge is proportional to the negative gradient of the value function. Because V increases in directions where the particle is likely to end up far from the observation, $-\nabla_x V$ points toward the observation-consistent region. This is precisely the direction a nudge should push.

Substituting (3.3.5) back into the HJB equation produces a nonlinear PDE with a quadratic gradient term $\frac{1}{2}\nabla_x V^\top Q \nabla_x V$ that is generally intractable in high dimensions.

3.3.4 Cole-Hopf Linearization

The nonlinearity is removed by the substitution [23] $V(t, x) = -\log \Phi(t, x)$, which transforms the HJB equation into the *linear* backward PDE:

$$\frac{\partial \Phi}{\partial t} + \mathcal{L}\Phi = 0, \quad \Phi(t_{k+1}, x) = e^{-g(Y_{t_{k+1}}, x)}, \quad (3.3.6)$$

where $\mathcal{L}$ is the infinitesimal generator of the uncontrolled diffusion (3.1.1). In terms of Φ , the optimal control becomes [72, 35]:

$$u^*(t, x) = \frac{Q \nabla_x \Phi(t, x)}{\Phi(t, x)}. \quad (3.3.7)$$

The linearization is significant: because Φ satisfies a linear PDE, its solution can be represented as an expectation over uncontrolled trajectories via the Feynman-Kac theorem, making Monte Carlo approximation tractable.

3.3.5 Feynman-Kac Representation and Monte Carlo Evaluation

The Feynman-Kac theorem gives the solution of (3.3.6) as:

$$\Phi(t, x) = \mathbb{E}_{t,x} \left[e^{-g\left(Y_{t_{k+1}}, \eta_{t_{k+1}}^{t,x}\right)} \right], \quad (3.3.8)$$

where $\eta^{t,x}$ is the uncontrolled process starting from x at time t . In plain terms: $\Phi(t, x)$ is the expected terminal observation likelihood over all uncontrolled paths that start from x at time t . Paths that naturally arrive near the observation have high likelihood and contribute more to this expectation.

The gradient $\nabla_x \Phi$ satisfies:

$$\nabla_x \Phi(t, x) = -\mathbb{E}_{t,x} \left[e^{-g(Y_{t_{k+1}}, \eta_{t_{k+1}})} \exp \left(\int_t^{t_{k+1}} (\nabla_x b(\eta_s))^\top ds \right) \nabla_x g(Y_{t_{k+1}}, \eta_{t_{k+1}}) \right]. \quad (3.3.9)$$

The matrix exponential involving the Jacobian $\nabla_x b$ captures how sensitive the terminal state is to perturbations in the initial condition. For Lorenz-63, this Jacobian is (3.1.5). It is integrated along each uncontrolled path as part of an augmented SDE, which is why each oracle rollout integrates a 12-dimensional system (3 state dimensions plus a 3×3 sensitivity matrix) rather than just a 3-dimensional one.

Combining these representations, the optimal control for particle i is approximated by a Monte Carlo average over n_s independent uncontrolled rollouts from $\hat{X}_t^{(i)}$ [72, 35]:

$$u_t^{(i)} \approx -Q \frac{\frac{1}{n_s} \sum_{j=1}^{n_s} \hat{w}_{t_{k+1}}^{(j)} \exp \left(\int_t^{t_{k+1}} \nabla_x b(\eta_s^{(j)})^\top ds \right) \nabla_x g(Y_{t_{k+1}}, \eta_{t_{k+1}}^{(j)})}{\frac{1}{n_s} \sum_{j=1}^{n_s} \hat{w}_{t_{k+1}}^{(j)}}, \quad (3.3.10)$$

where $\hat{w}^{(j)} \propto e^{-g(Y_{t_{k+1}}, \eta_{t_{k+1}}^{(j)})}$ is the normalized path weight. Intuitively: paths that naturally land near the observation get high weight, and the optimal nudge points in the direction that those high-weight paths are heading toward the observation.

The computational cost is $\mathcal{O}(N \cdot n_s \cdot K_c)$ augmented SDE integrations per assimilation window, where K_c is the number of control substeps. For the Lorenz-63 oracle used in this thesis ($N = 32$, $n_s = 15$, $K_c = 8$, $T = 20$ TU, 200 observation steps), this amounts to $200 \times 32 \times 15 \times 8 = 768,000$ integrations of the 12-dimensional augmented system per trial. This is the bottleneck that the imitation learning framework is designed to eliminate. The neural network, once trained, replaces this entire computation with a single forward pass per particle per observation step.

3.4 Girsanov Weight Correction

When we add a control to the particle dynamics, we are propagating particles under a different probability distribution than the one the signal model predicts. Think of it this way: the uncontrolled model says particles should drift according to $b(x) dt$, but we are moving them according to $b(x) dt + u dt$. We have deliberately bent each particle's trajectory toward the observation. This is useful, but it means the particles are no longer representative samples from the prior predictive distribution. Computing importance weights with the observation likelihood alone, without accounting for

this bending, produces biased estimates. The Girsanov correction is the mathematical tool that quantifies how much each trajectory was bent and adjusts the weights accordingly.

Why this matters for this thesis. Omitting the Girsanov correction was the root cause of filter degradation in my prior work with learned controllers [12]. Including it is what restores importance-sampling validity and enables the controlled filter to outperform the uncontrolled baseline. A controlled filter without this correction is not a valid importance sampler, as the weighted particle ensemble will not converge to the true filtering distribution as $N \rightarrow \infty$, regardless of how accurate the controller is. Every controlled filter evaluated in this thesis therefore includes this correction.

3.4.1 Radon-Nikodym Derivative

Let $\mu^{(i)}$ be the path-space measure of the uncontrolled SDE from $\hat{X}_{t_k}^{(i)}$, and $\hat{\mu}^{(i)}$ be the measure of the controlled SDE from the same starting point. Writing $u = \sigma v$ with $v(t, x) := -\sigma^\top \nabla_x V(t, x)$, the controlled dynamics become:

$$d\hat{X}_t^{(i)} = b(\hat{X}_t^{(i)}) dt + \sigma [dW_t + v(t, \hat{X}_t^{(i)}) dt]. \quad (3.4.1)$$

By Girsanov’s theorem [72, 35, 7], the Radon-Nikodym derivative (the ratio of the uncontrolled measure to the controlled measure along a path) is:

$$\frac{d\mu^{(i)}}{d\hat{\mu}^{(i)}} = \exp\left(-\int_{t_k}^{t_{k+1}} v(s, \hat{X}_s^{(i)})^\top dW_s - \frac{1}{2} \int_{t_k}^{t_{k+1}} \|v(s, \hat{X}_s^{(i)})\|^2 ds\right). \quad (3.4.2)$$

The stochastic integral in the exponent captures the random fluctuations along the controlled path that distinguish it from an uncontrolled one. The deterministic integral accumulates the squared magnitude of the effective control, penalizing total control effort. Together, they quantify how unlikely the controlled trajectory would have been under the uncontrolled model. A particle that was nudged aggressively produces a large exponent magnitude and therefore a small Radon-Nikodym factor.

In practice, this correction is accumulated incrementally at each integration substep during the assimilation window. The implementation in `pybeeda` does this automatically for any registered nudging controller, which is why adding the correction to the learned controller required only that the learning module override the standard `_calculate_nudging` method.

3.4.2 Corrected Weight Update

With the Radon-Nikodym derivative in hand, the full importance weight update for the nudged particle filter is [72, 35, 7]:

$$\tilde{w}_{k+1}^{(i)} \propto w_k^{(i)} \cdot \underbrace{\exp\left(-g\left(Y_{t_{k+1}}, \hat{X}_{t_{k+1}}^{(i)}\right)\right)}_{\text{observation likelihood}} \cdot \underbrace{\frac{d\mu^{(i)}}{d\hat{\mu}^{(i)}}}_{\text{Girsanov correction}}. \quad (3.4.3)$$

This equation has three factors: the previous weight, the observation likelihood at the controlled particle position, and the Girsanov correction that compensates for the modified proposal. When $u \equiv 0$, the Girsanov factor equals one and the update reduces to the standard formula (3.2.2).

The two non-trivial factors partially oppose each other. A particle nudged strongly toward the observation scores highly on the likelihood factor (it ends up close to $Y_{t_{k+1}}$) but receives a large penalty from the Girsanov factor (its trajectory would have been very unlikely under the uncontrolled model). The net result is that weights are more uniform than in the uncontrolled filter (the nudging reduces degeneracy) but not perfectly equal (the correction prevents overconfidence). This balance is what makes a well-calibrated nudged filter better than both the uncontrolled filter and a naive uncorrected nudged filter.

This tradeoff also explains the central result in Chapter 6: the oracle nPF applies very aggressive nudges that score well on the likelihood factor but incur large Girsanov penalties, collapsing nESS to 0.75. The learned per-particle SR controller, trained with Huber regression on the oracle’s labels, regresses toward moderate control magnitudes that score slightly lower on the likelihood factor but incur much smaller Girsanov penalties, resulting in better nESS (0.87) and, counterintuitively, better nRMSE (0.16 vs 0.61).

3.4.3 Zero-Variance Property of the Optimal Control

The optimal control derived in Section 3.3 has a remarkable theoretical property: for particles starting from a common location x^* at time t_k , the weight variance at t_{k+1} is exactly zero under the combined likelihood-Girsanov update. All particles receive identical weights regardless of the random noise along the way. This is because the optimal value function satisfies $\Phi(t_k, x^*) = e^{-g} \cdot \frac{d\mu}{d\hat{\mu}}$ deterministically, so every particle starting from x^* produces the same weight [72, 35].

In practice, particles do not all start from the same location, so the variance is

positive but substantially reduced compared to the uncontrolled filter. The goal of the learned controllers in this thesis is to approximate this property: not to achieve zero variance, but to maintain sufficiently low variance that the filter remains healthy throughout the assimilation window. The Stage 2 weight-balance penalty in Chapter 5 is a direct attempt to make this approximation explicit during training.

3.5 Evaluation Metrics

Three complementary metrics are reported in all experiments. They measure state estimation accuracy, ensemble health, and the root cause of degeneracy respectively.

3.5.1 Normalized RMSE

The filtered state estimate at observation step k is the weighted mean $\hat{x}_k = \sum_{i=1}^N w_k^{(i)} X_{t_k}^{(i)}$. The normalized RMSE is:

$$e_k = \frac{\|X_{t_k}^{\text{true}} - \hat{x}_k\|_2}{\|X_{t_k}^{\text{true}}\|_2}, \quad (3.5.1)$$

following [72]. The time-averaged nRMSE is $\bar{e} = \frac{1}{K} \sum_{k=1}^K e_k$, reported as the primary scalar performance metric across all trials.

Normalization by $\|X_{t_k}^{\text{true}}\|_2$ makes the error dimensionless and comparable across different attractor regions. On Lorenz-63, the true state magnitude varies between roughly 5 and 30 depending on where the system is on the attractor, so unnormalized RMSE would give misleading comparisons between observations taken during quiet periods and those taken during large-amplitude excursions. A value of $\bar{e} \approx 0$ indicates near-perfect tracking. A value of $\bar{e} \approx 1$ indicates errors comparable to the true state magnitude, which is essentially no better than predicting zero everywhere.

3.5.2 Normalized Effective Sample Size

The normalized ESS is:

$$\text{nESS}_k = \frac{N_{\text{eff},k}}{N} = \frac{1}{N \sum_{i=1}^N (w_k^{(i)})^2} \in (0, 1], \quad (3.5.2)$$

captured at each observation step *before* any resampling. The time-averaged nESS is $\overline{\text{nESS}} = \frac{1}{K} \sum_k \text{nESS}_k$.

The nESS answers a practical question: how many of the N particles are actually contributing useful information? A filter that maintains nESS close to 1 is effectively using its full computational budget. One that repeatedly collapses to nESS near $1/N$ is wasting most of its budget on particles that carry negligible weight. The resampling threshold of $\text{nESS} < 0.5$ is a standard choice that prevents the most severe degeneracy while not resampling so frequently that ensemble diversity is destroyed.

3.5.3 Particle Weight Variance

The variance of the normalized weights provides a direct diagnostic of degeneracy pressure:

$$\text{Var}[w] = \frac{1}{N} \sum_{i=1}^N \left(w^{(i)} - \frac{1}{N} \right)^2. \quad (3.5.3)$$

High weight variance indicates that the proposal and target distributions are poorly matched. This metric is particularly informative for diagnosing whether control-induced weight collapse is happening between resampling steps: even after resampling resets all weights to $1/N$, the variance grows again at the next update if the proposal is poorly calibrated. The Stage 2 weight-balance penalty in Chapter 5 uses a differentiable proxy for this quantity to directly penalize weight variance during training.

3.6 Summary

The four components developed in this chapter form a complete chain. The signal model (3.1.1) describes how the true state evolves. The particle filter (3.2.2) approximates the posterior using a weighted ensemble that is prone to weight degeneracy in chaotic regimes. The optimal control problem (3.3.2) yields a nudging law that steers particles toward the upcoming observation while minimizing weight variance, but requires $\mathcal{O}(N \cdot n_s \cdot K_c)$ augmented SDE integrations per window. The Girsanov correction (3.4.3) ensures that modified particle trajectories remain importance-consistent.

The imitation learning framework in this thesis treats the optimal nudging law as an expert policy and uses behavioral cloning to train a neural network surrogate. The surrogate replaces the expensive Monte Carlo computation at deployment time with a single forward pass.

Chapter 4 describes how the oracle expert data are generated, while Chapter 5 describes the neural network architectures and training objectives.

Chapter 4

Dataset Generation

This chapter describes the offline pipeline for generating the supervised training dataset. The pipeline has three stages: sampling attractor states with good geometric coverage (Section 4.2), constructing multi-particle ensemble snapshots (Section 4.3), and computing expert control labels (Section 4.4). The Lorenz-96 extension is described in Section 4.5 and dataset statistics are collected in Section 4.6. Full pseudocode is in Appendix D.

4.1 Rationale for Offline Data Generation

The oracle nudged particle filter requires, at each observation step, a Monte Carlo approximation of the Feynman-Kac expectation (3.3.10) for every particle. Concretely, the oracle propagates n_s independent uncontrolled realizations of the augmented SDE from each particle’s current state, accumulates the state-transition matrix sensitivity integral along each rollout, and forms a weighted average to estimate the optimal control. This must be repeated at each of the K_c control substeps within the assimilation window.

The total operation count per observation step is $N \times n_s \times K_c$ augmented SDE integrations. For the Lorenz-63 production configuration ($N = 32$, $n_s = 15$, $K_c = 8$), this is $32 \times 15 \times 8 = 3,840$ integrations of the 12-dimensional augmented system (state plus 3×3 state-transition matrix) per observation step. In practice, the oracle takes on the order of seconds to minutes per step on a single CPU core, making it impractical for real-time filtering. The problem compounds for Lorenz-96: at $d = 8$, each augmented integration is 6 times more expensive ($d + d^2 = 72$ vs 12 dimensions),

so even with fewer particles the online cost grows substantially with dimension.

The offline strategy resolves this by separating computation into two phases. In the offline phase, the oracle runs once over a representative sample of attractor states and ensemble configurations, producing labeled training examples of the form $(X_0, w_0, y, \Delta t, \sigma_{\text{obs}}) \mapsto U_{\text{seq}}$. This is done once, at full oracle quality, and the results are stored to disk. In the online phase, a trained neural network replaces the oracle entirely: given the current ensemble state and observation, it produces per-particle controls in a single batched forward pass, at a cost of microseconds per observation step regardless of n_s or K_c .

This strategy is valid if two conditions hold. First, the training distribution must cover the configurations the filter will encounter during closed-loop deployment, including both well-initialized ensembles and near-degenerate ones. This is addressed by the two-regime ensemble sampling design described in Section 4.3. Second, the learned mapping must generalize smoothly across the attractor rather than memorizing individual training examples. This is addressed by the low-discrepancy attractor sampling design in Section 4.2, which maximizes geometric coverage of the attractor with a modest number of labeled states.

4.2 Attractor Sampling

We apply a two-stage procedure to obtain geometrically uniform attractor coverage. First, the deterministic Lorenz-63 ODE is integrated for 300,000 steps from a known attractor point, with a 30,000-step burn-in discarded and the trajectory subsampled at stride 10 to reduce temporal correlation. This produces a candidate pool $\mathcal{X}_{\text{pool}}$ distributed according to the natural invariant measure. Second, greedy farthest-point sampling (FPS) selects $|\mathcal{X}_{\text{ld}}| = 5,000$ states by iteratively choosing the candidate farthest in Euclidean distance from all already-selected states. The result is a low-discrepancy set with maximized minimum pairwise separation, covering both lobes and the switching region.

A fixed seed then draws $N_Y = 200$ true states from $\mathcal{X}_{\text{ld}}$, which serve as the observation centers for dataset construction.

4.3 Dataset Contract

Each training example is a complete multi-particle ensemble snapshot stored in HDF5 format. The schema is given in Table 4.1.

Key	Shape	Description
X0	(n, N, d)	Particle states at window start
w0	(n, N)	Normalized importance weights
y	(n, d)	Observation vector
dt	$(n,)$	Time to observation Δt
sigma_obs	$(n,)$	Observation noise std σ_{obs}
U_seq	(n, K_c, N, d)	Oracle control sequence
X_final	(n, N, d)	Post-control particle states
w_final	(n, N)	Post-control weights with Girsanov correction
log_rn	(n, N)	Per-particle log Radon-Nikodym correction
ness_w0	$(n,)$	Pre-control raw ESS
ness_wf	$(n,)$	Post-control raw ESS

Table 4.1: HDF5 dataset schema. For Lorenz-63: $N = 32$, $d = 3$, $K_c = 8$. For Lorenz-96 ($K = 8$): $N = 16$, $d = 8$, $K_c = 4$. `ness_w0` and `ness_wf` store raw ESS $\in [1, N]$; divide by N for nESS.

Ensemble regimes. Each example is initialized under one of two regimes. In the *uniform* regime, all N particles are drawn from attractor states whose deterministic forecasts lie close to y , giving moderate initial weight diversity (nESS ≈ 0.4 to 0.8). In the *severe collapse* regime, one-third of particles are drawn near y and two-thirds from states with poor forecasts, producing near-degenerate initial conditions (nESS ≈ 0.05 to 0.2). Equal numbers of examples are generated under each regime. The production dataset has a pre-control mean nESS of 0.076 , corresponding to approximately 2.4 effective particles out of $N = 32$, confirming that the training distribution covers genuinely degenerate scenarios.

Observation and time horizon grid. For each $(x_{\text{true}}, \Delta t, \sigma_{\text{obs}})$ combination, the observation is $y = \Phi_{\Delta t}^{\text{det}}(x_{\text{true}}) + \sigma_{\text{obs}} \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$, where $\Phi_{\Delta t}^{\text{det}}$ denotes deterministic RK4 propagation. The Lorenz-63 time horizon grid is $\mathcal{T} = \{0.15, 0.30, 0.45, 0.60\}$ TU and the noise grid is $\Sigma_{\text{obs}} = \{1.0, 2.0, 4.0\}$. With $E_{\text{pair}} = 2$ ensemble draws per combination and two regimes, the total number of examples is

$$n = N_Y \times |\mathcal{T}| \times |\Sigma_{\text{obs}}| \times |\text{Regimes}| \times E_{\text{pair}} = 200 \times 4 \times 3 \times 2 \times 2 = 9,600, \quad (4.3.1)$$

of which 4,800 are valid (generation was interrupted; the remaining rows are zero-padded).

Standardization. All inputs and outputs are z-scored using statistics computed on the training split only. Importance weights $w^{(i)}$ are not normalized; they are already in $[0, 1]$ with unit sum. Normalizer statistics are saved with each model checkpoint for inference-time use.

4.4 Expert Control Labels

Oracle. For each ensemble snapshot, the pybeeda `NudgedParticleFilter` oracle computes the Feynman-Kac Monte Carlo optimal control over $K_c = 8$ substeps using $n_s = 15$ uncontrolled rollouts per particle per substep. The output is the control sequence $U_{\text{seq}} \in \mathbb{R}^{K_c \times N \times d}$, post-control states X_{final} , and the accumulated per-particle log Girsanov correction $\log G^{(i)}$.

Girsanov correction. The log Radon-Nikodym correction is accumulated from the actual controlled trajectory at each substep k :

$$\log G^{(i)} = \sum_{k=0}^{K_c-1} \left[-\left(u_k^{(i)}\right)^\top \left(\Delta x_k^{(i)} - f(x_k^{(i)}) \delta t\right) + \frac{\delta t}{2} \|u_k^{(i)}\|^2 \right], \quad (4.4.1)$$

where $\Delta x_k^{(i)}$ is the actual state increment. Storing $\log G^{(i)}$ alongside the controls allows the training pipeline to compute differentiable post-control weights during Stage 2 training without re-running the oracle.

Post-control weights. The stored `w_final` combines the observation likelihood and the Girsanov correction:

$$\log \tilde{w}_{\text{final}}^{(i)} = \log w_0^{(i)} - \frac{\|X_{\text{final}}^{(i)} - y\|^2}{2\sigma_{\text{obs}}^2} + \log G^{(i)}, \quad (4.4.2)$$

followed by softmax normalization.

Training label. The network trains on the first substep of the oracle sequence only: $U_{\text{target}} = U_{\text{seq}}[:, :, 0, :]$. This is the cleanest oracle estimate (before any state

has evolved under window dynamics). Multi-step rollout supervision is left as future work.

Oracle robustness. Floating-point overflow near attractor lobe boundaries is handled by resampling the ensemble and retrying the oracle up to 25 times. Fewer than 2% of examples require any retry.

Comparison with the JIW expert. The JIW used a bounded saturation heuristic with $\|u\|_\infty \leq 10$. The nPF oracle produces controls with magnitudes up to approximately 1400 in physical units, a qualitatively harder regression target. The higher offline validation RMSE in this thesis (1.13 vs 0.052 normalized) reflects the harder oracle, not a failure of the learning pipeline.

4.5 Extension to Lorenz-96

The Lorenz-96 ($K = 8$, $d = 8$) pipeline mirrors the Lorenz-63 design with three changes. First, the augmented system per rollout grows from $d + d^2 = 12$ to 72 dimensions, a factor of 6 in per-label cost. Second, the oracle uses the pybeebase **L96** model class with $K_c = 4$ substeps (vs 8) and $n_s = 10$ rollouts (vs 15) to keep generation tractable. Third, the time horizon grid is shortened to $\mathcal{T} = \{0.05, 0.10\}$ TU to match the faster L96 timescale. The attractor initialization uses $x_k = F + 0.01\mathcal{N}(0, 1)$ with a 50,000-step burn-in. The full 400-example dataset is generated in a single Google Colab session (approximately 15 to 30 minutes on an A100 or G4 GPU), using the `datagen_196.ipynb` notebook.

4.6 Dataset Summary

Table 4.2 makes note of all the parameters used in the dataset generation for both Lorenz-63 and Lorenz-96 experiments, which are to be discussed in Chapters 6 and 7, respectively.

Table 4.2: Dataset parameters for Lorenz-63 and Lorenz-96 ($K = 8$).

Parameter	Lorenz-63	Lorenz-96 ($K = 8$)
State dimension d	3	8
$ \mathcal{X}_{\text{id}} $	5,000	5,000
Observation centers N_Y	200	50
Time horizon grid $\mathcal{T}$ (TU)	$\{0.15, 0.30, 0.45, 0.60\}$	$\{0.05, 0.10\}$
Noise levels Σ_{obs}	$\{1.0, 2.0, 4.0\}$	$\{1.0, 2.0\}$
Ensemble regimes	uniform, severe collapse	uniform, severe collapse
Particles per example N	32	16
Oracle substeps K_c	8	4
Oracle rollouts n_s	15	10
Total examples (expected)	9,600	400
Valid examples	4,800	400
Train / validation split	4,080 / 720	340 / 60
Pre-control mean nESS	0.076	—
Storage format	HDF5 (gzip)	HDF5 (gzip)
Generation infrastructure	Della HPC (Slurm array)	Google Colab

Chapter 5

Ensemble-Aware Supervised Learning

This chapter presents the core methodological contribution of the thesis: an ensemble-aware supervised learning framework for particle nudging control. The framework extends the per-particle supervised regression approach from the JIW [12] in three directions, each targeting one of the structural deficiencies identified in Chapter 1. First, the input representation is extended to include particle-level contextual features (innovation and deviation from the ensemble mean) alongside a permutation-invariant global context derived from a weight-aware pooling operation, allowing the learned controller to condition on collective ensemble behavior without violating particle independence (Section 5.2). Second, the architecture is designed around a residual structure that allows transfer of per-particle base knowledge while learning ensemble-level corrections on top (Section 5.3). Third, the training objective is a two-stage procedure: initial oracle imitation using a Huber loss, followed by fine-tuning with an additional weight-balance penalty derived from the Girsanov correction (Section 5.4). Section 5.5 describes deployment within the corrected particle filter at test time.

5.1 From Per-Particle to Ensemble-Aware Control

The limitation of per-particle control. The JIW [12] trained a controller of the form

$$u_t^{(i)} = \mu_\theta(x_t^{(i)}, y_{k+1}, \Delta t), \quad (5.1.1)$$

where $\Delta t = t_{k+1} - t$ is the time remaining to the next observation. This design treats every particle identically: each receives exactly the same control function of its own state, the observation, and the time horizon, regardless of where it sits within the ensemble. A particle that already lies close to the observation-consistent region receives the same aggressive nudge as one on the opposite lobe of the attractor. The result is that all particles are driven toward the observation simultaneously, concentrating the ensemble and collapsing the ESS even when some particles should remain in diverse regions to represent uncertainty.

What ensemble-awareness adds. An ensemble-aware controller additionally receives information about the structure of the particle cloud: where the ensemble mean is, how the weights are distributed, and how far each particle is from the current ensemble consensus. With this information the controller can modulate its output per particle based on relative position and importance within the ensemble. The critical constraint is that this modulation must not introduce statistical coupling between particles during propagation. This is satisfied by computing the ensemble summary once at the start of each assimilation window and treating it as a fixed context during propagation, rather than recomputing it from evolving particle positions at every substep.

5.2 Input Features and Ensemble Context

Base per-particle features. The base input to the encoder for particle i consists of

$$\text{base}^{(i)} = (x_t^{(i)}, y_{k+1}, \Delta t, \sigma_{\text{obs}}) \in \mathbb{R}^8, \quad (5.2.1)$$

where $x_t^{(i)} \in \mathbb{R}^3$ is the particle state, $y_{k+1} \in \mathbb{R}^3$ is the upcoming observation, $\Delta t \in \mathbb{R}$ is the time remaining to the observation, and $\sigma_{\text{obs}} \in \mathbb{R}$ is the observation noise standard deviation. Including σ_{obs} as an explicit input allows the controller to adapt its nudging strength to the noise regime.

Particle-relative features. Two additional features are appended for each particle. The first is the *innovation*:

$$\iota^{(i)} = x_t^{(i)} - y_{k+1} \in \mathbb{R}^3, \quad (5.2.2)$$

which measures how far the particle’s current state is from the observation. The optimal control law (3.3.10) is driven by the gradient of the terminal cost $\nabla_x g$, which is proportional to $(h(x) - y)$ for linear h . Including the innovation directly makes this structure explicit. The second relative feature is the *deviation from the weighted ensemble mean*:

$$\delta^{(i)} = x_t^{(i)} - \bar{x}_k, \quad \bar{x}_k = \sum_{j=1}^N w_k^{(j)} x_k^{(j)}, \quad (5.2.3)$$

which encodes each particle’s position relative to the ensemble consensus.

The full per-particle input to the encoder is:

$$\xi^{(i)} = (x_t^{(i)}, y_{k+1}, \Delta t, \sigma_{\text{obs}}, \iota^{(i)}, \delta^{(i)}, w_k^{(i)}) \in \mathbb{R}^{15}, \quad (5.2.4)$$

where $\bar{x}_k$ and $\delta^{(i)}$ are fixed at the start of the assimilation window to preserve conditional particle independence.

Permutation-invariant global context. A global context vector $g \in \mathbb{R}^{128}$ is computed by pooling over the full ensemble. Given particle embeddings $h^{(i)} \in \mathbb{R}^{64}$ from the encoder:

$$\bar{h} = \sum_{i=1}^N w_k^{(i)} h^{(i)}, \quad h_{\text{max}} = \max_i h^{(i)}, \quad g = [\bar{h}; h_{\text{max}}] \in \mathbb{R}^{128}. \quad (5.2.5)$$

Both operations are permutation invariant. The importance-weighted mean $\bar{h}$ ensures that the global context is dominated by particles currently most consistent with past observations.

5.3 Model Architecture

Four model variants are implemented and evaluated. All share the same encoder backbone; they differ in what context they condition on and how they are trained.

5.3.1 Shared Encoder: PerParticleEncoder

A per-particle MLP embeds each particle’s input into a 64-dimensional latent representation:

$$h^{(i)} = \phi_{\text{enc}}(\xi^{(i)}), \quad \phi_{\text{enc}} : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{64}, \quad (5.3.1)$$

implemented as a $[d_{\text{in}} \rightarrow 128 \rightarrow 128 \rightarrow 64]$ MLP with ReLU activations, applied identically to all particles via broadcasting. For Lorenz-63, $d_{\text{in}} = 12$ for the per-particle baseline and $d_{\text{in}} = 15$ for the ensemble variants.

5.3.2 Per-Particle Baseline (*per_particle*)

The *per_particle* model processes only the 12-dimensional base input $(x^{(i)}, y, \Delta t, \sigma_{\text{obs}}, \iota^{(i)}, w^{(i)})$ with no pooling step:

$$u^{(i)} = \phi_{\text{dec}}(h^{(i)}), \quad \phi_{\text{dec}} : \mathbb{R}^{64} \rightarrow \mathbb{R}^3, \quad (5.3.2)$$

where ϕ_{dec} is a $[64 \rightarrow 128 \rightarrow 64 \rightarrow 3]$ MLP. This is the direct successor to the JIW SR model, now with a proper Feynman-Kac oracle and Girsanov correction. It has approximately 50,000 parameters.

5.3.3 Ensemble S1 (*ensemble_s1*)

The *ensemble_s1* model extends the per-particle baseline with the 15-dimensional encoder input and the weight-aware pooling context g . Each particle’s control is:

$$u^{(i)} = \phi_{\text{dec}}(h^{(i)}, g), \quad \phi_{\text{dec}} : \mathbb{R}^{192} \rightarrow \mathbb{R}^3, \quad (5.3.3)$$

implemented as a $[192 \rightarrow 128 \rightarrow 64 \rightarrow 3]$ MLP. This model is trained with the Stage 1 Huber loss only; it tests whether architectural ensemble-awareness alone improves filtering performance. It has approximately 110,000 parameters.

5.3.4 Ensemble S2 (*ensemble_s2*)

The *ensemble_s2* model shares the same architecture as *ensemble_s1* but is fine-tuned from the Stage 1 checkpoint with an additional weight-balance penalty in Stage 2. It tests whether explicitly penalizing weight imbalance during training improves nESS at an acceptable RMSE cost. The regularization strength is swept over $\lambda \in \{0.1, 1.0, 10.0, 100.0\}$; $\lambda^* = 0.1$ is selected based on the offline RMSE-nESS Pareto frontier and is the variant evaluated in closed-loop experiments.

5.3.5 Residual Ensemble (*residual_ensemble*)

The *residual_ensemble* model is designed to guarantee that ensemble context cannot hurt relative to the per-particle baseline. The base decoder is initialized directly from the pretrained *per_particle* checkpoint and a small ensemble-conditioned correction is added on top:

$$u^{(i)} = \underbrace{\phi_{\text{base}}(h^{(i)})}_{\text{per-particle base}} + s \cdot \underbrace{\phi_{\text{corr}}(h^{(i)}, g)}_{\text{ensemble correction}}, \quad (5.3.4)$$

where ϕ_{base} is frozen from the *per_particle* checkpoint for 50 epochs before being unfrozen, $\phi_{\text{corr}} : \mathbb{R}^{192} \rightarrow \mathbb{R}^3$ is a new correction decoder, and $s \in \mathbb{R}$ is a learnable scalar initialized to 0.1. At initialization, $u^{(i)} \approx u_{\text{per-particle}}^{(i)}$, so the residual ensemble is at least as good as per-particle at the start of training. This tests whether ensemble context can provide a meaningful improvement on top of an already strong per-particle controller.

Summary. Table 5.1 summarizes all four variants.

Controller	Ensemble context	Weight-aware objective	Transfer init	Params	Online cost (per step)
Uncontrolled PF	No	No	N/A	—	$\mathcal{O}(Nd)$
Relaxation nPF	No	No	N/A	—	$\mathcal{O}(Nd)$
Oracle nPF	No	No	N/A	—	$\mathcal{O}(Nn_sd^2)$
<i>per_particle</i>	No	No	No	~50K	$\mathcal{O}(N)$
<i>ensemble_s1</i>	Yes	No	No	~110K	$\mathcal{O}(N)$
<i>ensemble_s2</i>	Yes	Yes	No	~110K	$\mathcal{O}(N)$
<i>residual_ensemble</i>	Yes	No	Yes	~110K	$\mathcal{O}(N)$

Table 5.1: Comparison of all particle filter controllers. All ML variants include the Girsanov weight correction. The *per_particle* baseline is the direct successor to the JIW SR model.

5.4 Training Objective & Protocol

Label construction. Training targets are the first substep of the oracle control sequence: $u^* = U_{\text{seq}}[:, :, 0, :]$. Oracle controls are clipped to $[-100, 100]$ before normalization to remove heavy-tailed outliers (approximately 2% of samples), which would otherwise dominate the loss and cause the model to regress toward the mean on typical samples.

Stage 1: Oracle imitation with Huber loss. All four variants begin with Stage 1 training, which minimizes a Huber loss in normalized space:

$$\mathcal{L}_{\text{Huber}}(\theta) = \frac{1}{BN} \sum_{b=1}^B \sum_{i=1}^N \ell_{\delta}(\mu_{\theta}(\xi_b^{(i)}) - u_b^{*(i)}), \quad \ell_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta, \\ \delta(|r| - \delta/2) & \text{otherwise,} \end{cases} \quad (5.4.1)$$

with $\delta = 1.0$. Huber loss is used rather than MSE because the oracle control distribution is heavy-tailed even after clipping: the Huber loss is quadratic for typical small-control samples and linear in the tails, preventing outliers from dominating the gradient. Stage 1 uses Adam with initial learning rate 3×10^{-4} , cosine annealing to 10^{-6} , 150 epochs, early stopping patience 15, batch size 256, and gradient clipping at L^2 norm 1.0.

Transfer initialization for ensemble variants. For *ensemble_s1* and *residual_ensemble*, the base decoder is initialized from the pretrained *per_particle* checkpoint. The base is frozen for the first 50 epochs of Stage 1 so that the correction pathway learns to add useful context on top of an already-functional per-particle base; all parameters are then unfrozen at a reduced learning rate of 10^{-4} .

Stage 2: Weight-balance penalty. Starting from the *ensemble_s1* Stage 1 checkpoint, Stage 2 fine-tunes with an augmented loss:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{Huber}}(\theta) + \lambda \cdot \mathcal{L}_{\text{penalty}}(\theta). \quad (5.4.2)$$

The penalty is computed in raw (denormalized) physical units. Denormalized predicted controls are used to compute approximate post-control weights via the Girsanov formula and a one-step Euler approximation of the likelihood improvement:

$$\log \tilde{w}^{(i)} = \log w_k^{(i)} - \frac{\|x^{(i)} + u_{\text{raw}}^{(i)} \delta t - y_{k+1}\|^2 - \|x^{(i)} - y_{k+1}\|^2}{2\sigma_{\text{obs}}^2} - \frac{1}{2} \|u_{\text{raw}}^{(i)}\|^2 \delta t. \quad (5.4.3)$$

The penalty is the variance of the resulting softmax-normalized weights:

$$\mathcal{L}_{\text{penalty}}(\theta) = \text{Var}_i[\tilde{w}_{\text{pred}}^{(i)}]. \quad (5.4.4)$$

Stage 2 uses Adam at 3×10^{-5} , cosine annealing, and up to 100 epochs.

5.5 Deployment in the Corrected Particle Filter

At the start of each assimilation window $[t_k, t_{k+1}]$, the ensemble summary $(\bar{x}_k, \delta^{(i)}, g)$ is computed once from the current weighted ensemble and held fixed throughout. At each integration substep, the network is evaluated in a single batched forward pass over all N particles, producing controls $\{u^{(i)}\}$ that are clipped to $[-100, 100]$ and injected into the particle integrators. The Girsanov log-weight correction is accumulated at every substep by pybeeda’s nudged filter machinery. At observation time t_{k+1} , the corrected weight update applies the accumulated Girsanov correction alongside the observation likelihood. Resampling is triggered when $\text{nESS} < 0.5$.

The predicted nESS approximation used for offline model selection applies only the Girsanov term (ignoring the positive likelihood improvement), providing a conservative lower bound on the true closed-loop nESS:

$$\log \tilde{w}_{\text{eval}}^{(i)} = \log w_k^{(i)} - \frac{1}{2} \|u_{\text{raw}}^{(i)}\|^2 \delta t, \quad \widehat{\text{nESS}} = \frac{1}{N \sum_i (\tilde{w}_{\text{eval}}^{(i)})^2}. \quad (5.5.1)$$

Chapter 6

Closed-Loop Filtering Experiments

This chapter evaluates all four learned controllers in closed-loop particle filtering on Lorenz-63. All experiments use $N = 32$ particles, an observation interval of $\Delta T = 0.1$ TU, a total assimilation window of $T = 20$ TU, observation noise $\sigma_{\text{obs}} = 2.0$ ($R = 4I_3$), and process noise $\sigma_{\text{dyn}} = 1.0$ ($\Sigma = I_3$). These parameters are fixed for the entire chapter and apply to every filter variant and every trial.

Three questions organize the experiments. First, does the Girsanov-corrected imitation learning framework produce a learned controller that improves filtering performance relative to the uncontrolled baseline? Second, does conditioning on ensemble context improve performance beyond the corrected per-particle baseline? Third, does the residual ensemble architecture, which guarantees at-least-as-good performance as the per-particle baseline at initialization, achieve the best of both worlds?

6.1 Experimental Setup

6.1.1 System & Parameters

All experiments use the Lorenz-63 stochastic signal model from Section 3.1.3 with parameters $(\sigma_L, \rho, \beta) = (10, 28, 8/3)$, $\Sigma = I_3$, and $R = 4I_3$. All trials are initialized from $x_0 = (-3.139, -2.067, 22.904)^\top$ with independent unit Gaussian perturbations added per particle to provide initial ensemble diversity.

The integration step size is $\delta t = 0.001$ TU and the observation interval is $\Delta T = 0.1$ TU, giving $N_{\text{steps}} = 200$ observation steps per trial. The observation interval $\Delta T = 0.1$

TU is approximately $1/7$ of the error doubling time $\tau_d \approx 0.76$ TU, placing the filter in a moderately sparse regime where degeneracy develops steadily between updates. Without nudging, 32 particles are sufficient to track the system when observations are dense, but begin to show consistent phase loss at lobe switches under these conditions. Increasing N to 300 does not resolve the problem, as demonstrated in Section 6.2: the bottleneck is degeneracy from the observation likelihood geometry, not sampling density.

6.1.2 Filter Configurations

Seven filter variants are evaluated using the `pybeeda` and `pybeebase` filter classes [5, 6]. Resampling is triggered whenever $\text{nESS} < 0.5$ using universal (systematic) resampling [18]; it is applied at each observation step where the threshold is met, and weights are reset to $1/N$ immediately afterward. No resampling is done mid-window during particle propagation between observation steps.

PF (uncontrolled). The standard bootstrap particle filter [18], in which particles propagate under the uncontrolled signal SDE (3.1.1) and weights are updated via the observation likelihood (3.2.3). Of note, there is no nudging term applied, and this filter serves as an initial baseline for the filtering evaluation.

Relaxation PF. The relaxation nudged particle filter of van Leeuwen [70], using the static Kalman gain $K = \sigma_{\text{dyn}}^2 / (\sigma_{\text{dyn}}^2 + \sigma_{\text{obs}}^2) \cdot I_3 = 0.2 \cdot I_3$. The gain is fixed throughout the assimilation window and does not adapt to the distance to the next observation. The Girsanov correction (3.4.3) is accumulated throughout the window [72, 7].

Oracle nPF. The Feynman-Kac Monte Carlo optimal nudged particle filter [72, 35, 7] from `pybeeda`, with $n_s = 15$ Monte Carlo rollouts per particle and $K_c = 8$ control substeps per assimilation window. This matches the oracle used for data generation exactly. Each substep integrates the 12-dimensional augmented system (3 state dimensions plus a 3×3 sensitivity matrix) for $n_s = 15$ independent rollouts per particle. This is the performance ceiling for all learned controllers. The total cost per trial is $200 \times 8 \times 32 \times 15 = 768,000$ augmented SDE integrations.

per_particle. The learned per-particle imitation learning controller from Section 5.3. A 12-dimensional encoder input $(x^{(i)}, y, \Delta t, \sigma_{\text{obs}}, \iota^{(i)}, w^{(i)})$ is processed independently

per particle with no ensemble pooling and no global context. Single MLP forward pass per particle per observation step.

ensemble_s1. The DeepSets ensemble-aware controller [73] with weight-aware pooling and 15-dimensional encoder input (adds deviation $\delta^{(i)} = x^{(i)} - \bar{x}$ to the per-particle features), and is trained with Huber loss only.

ensemble_s2 ($\lambda = 0.1$). The *ensemble_s1* architecture fine-tuned with the differentiable Girsanov weight-balance penalty at the Pareto-optimal $\lambda^* = 0.1$, selected by offline validation sweep over $\lambda \in \{0.1, 1.0, 10.0, 100.0\}$.

residual_ensemble. The residual architecture initialized from the *per_particle* checkpoint, with a small trainable ensemble correction $s \cdot \phi_{\text{corr}}(h^{(i)}, g)$ added on top (learnable scalar s initialized to 0.1). This architecture guarantees no worse performance than per-particle at the start of training, and mirrors the structural logic of van Leeuwen’s relaxation nudging [70], where a correction term is added on top of the free-running dynamics rather than replacing them entirely.

All filters share the same truth trajectory, observations, and initial particle ensemble within each trial. $N_{\text{trials}} = 10$ independent Monte Carlo trials are run with seeds staggered by prime step $997 \times \text{trial index}$ to ensure diverse but reproducible initial conditions.

6.1.3 Metrics

At each observation step k , the normalized RMSE and nESS are:

$$e_k = \frac{\|x_{t_k}^{\text{true}} - \hat{x}_k\|_2}{\|x_{t_k}^{\text{true}}\|_2}, \quad \text{nESS}_k = \frac{1}{N \sum_i (w_k^{(i)})^2}, \quad (6.1.1)$$

where $\hat{x}_k = \sum_i w_k^{(i)} x_{t_k}^{(i)}$ is the weighted posterior mean and nESS is captured before any resampling. Time averages $\bar{e} = \frac{1}{200} \sum_k e_k$ and $\overline{\text{nESS}} = \frac{1}{200} \sum_k \text{nESS}_k$ are reported as the primary scalar metrics. All results are reported as mean and standard deviation across $N_{\text{trials}} = 10$.

6.2 Results

6.2.1 Summary Statistics

Table 6.1 reports the primary performance metrics for all seven filters averaged over 10 Monte Carlo trials.

Filter	$\bar{e}$ (nRMSE)	nESS	ms/step
PF (uncontrolled)	0.68 ± 0.08	0.97 ± 0.01	368
Relaxation PF	0.67 ± 0.09	0.96 ± 0.02	485
Oracle nPF ($n_s = 15$, $K_c = 8$)	0.61 ± 0.10	0.75 ± 0.03	30,518
<i>per_particle</i>	0.16 ± 0.02	0.87 ± 0.08	438
<i>ensemble_s1</i>	0.63 ± 0.10	0.88 ± 0.06	422
<i>ensemble_s2</i> ($\lambda^* = 0.1$)	0.68 ± 0.12	0.93 ± 0.05	444
<i>residual_ensemble</i>	0.45 ± 0.07	0.81 ± 0.10	423

Table 6.1: Lorenz-63 closed-loop filtering results. Mean $\pm$ std over 10 Monte Carlo trials ($N = 32$ particles, $T = 20$ TU, $\Delta T = 0.1$ TU, $\sigma_{\text{obs}} = 2.0$). nRMSE is normalized by the true state magnitude at each step. Timing values are from a single representative trial (trial 9) run on CPU. Bold indicates the best result per column.

Reading across the table, several results stand out immediately. The relaxation PF achieves almost identical nRMSE to the uncontrolled PF (0.67 vs 0.68), suggesting that the static Kalman gain nudge is not aggressive enough to meaningfully change which particles survive the weight update. The oracle nPF reduces nRMSE to 0.61 but at the cost of collapsing nESS from 0.97 to 0.75, reflecting the aggressive Feynman-Kac controls that push particles strongly toward the observation at the price of weight diversity. The *per_particle* SR controller achieves a qualitatively different outcome: nRMSE 0.16 at nESS 0.87, which is both a far better state estimate and a healthier ensemble than the oracle. The two DeepSets variants (S1 and S2) bracket a tradeoff: S2 achieves the highest nESS (0.93) at a higher nRMSE (0.68), while S1 achieves a more moderate nESS (0.88) with still poor nRMSE (0.63). The residual ensemble achieves an intermediate nRMSE (0.45) that is better than all classical baselines but still substantially worse than per-particle SR.

Timing note. Timing values are from trial 9, run on an H100 GPU via Google CoLaboratory. The oracle nPF averages 30,518 ms per observation step, reflecting the cost of 768,000 augmented SDE integrations per trial. All learned controllers run between 368 and 485 ms per step, comparable to the uncontrolled PF, with

the small overhead attributable to the MLP forward pass and Girsanov log-weight accumulation.

6.2.2 Aggregate Timeseries and Bar Charts

Figure 6.1 shows the trial-averaged nRMSE and nESS timeseries for all filters. Figure 6.2 isolates the ML controllers against the oracle nPF, focusing on the nESS behavior that distinguishes them. Figure 6.3 provides the scalar summary.

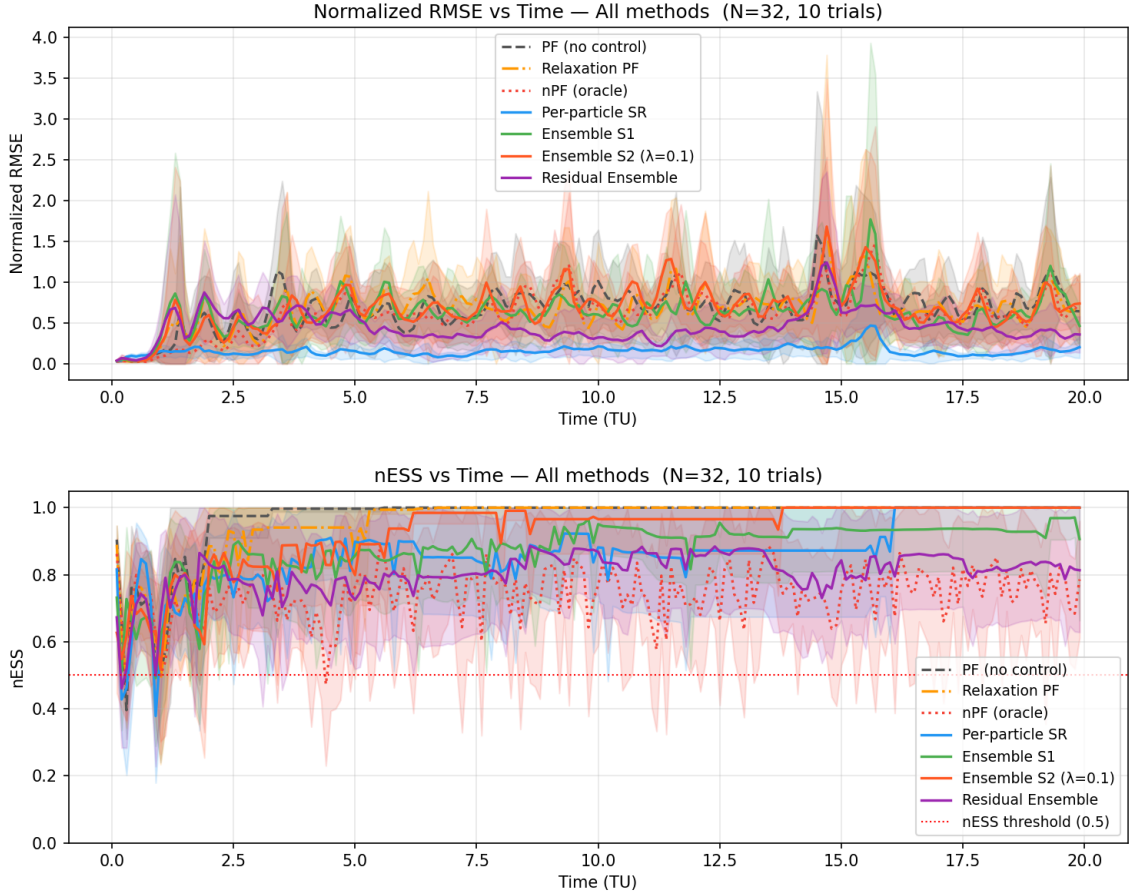


Figure 6.1: Time evolution of nRMSE (top) and nESS (bottom) for all filter variants on Lorenz-63 ($N = 32$, $\Delta T = 0.1$ TU, $\sigma_{\text{obs}} = 2.0$). Solid lines are trial means; shaded bands are ± 1 std over 10 Monte Carlo trials. The *per_particle* SR controller (green) maintains near-zero nRMSE and stable nESS throughout, while all other variants show episodic nRMSE spikes at lobe-switching events.

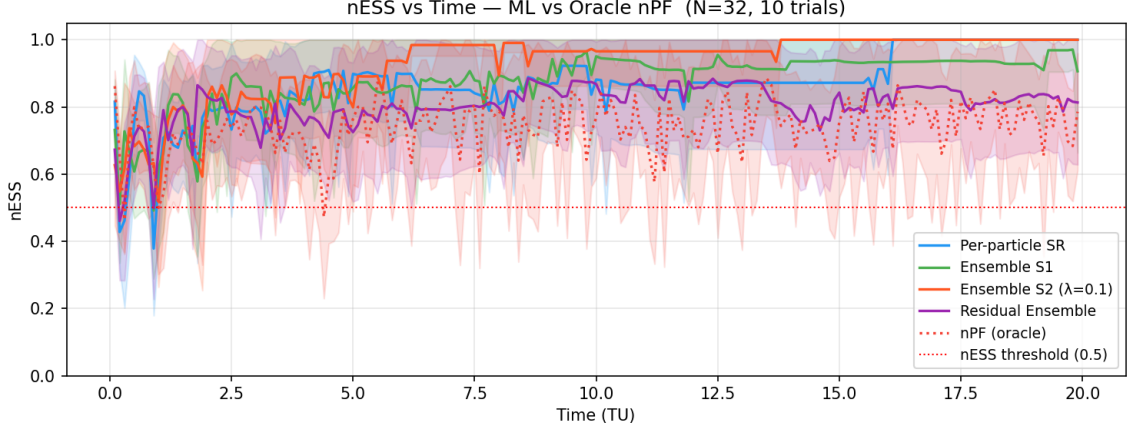


Figure 6.2: Normalized ESS timeseries for the four ML controllers compared against the oracle nPF. The red dashed line marks the resampling threshold $nESS = 0.5$. The oracle nPF (green dashed) frequently drops toward and below this threshold, confirming that aggressive Feynman-Kac controls repeatedly collapse the ensemble. The *per_particle* SR (blue) and *ensemble_s2* (red) maintain higher mean nESS throughout.

6.2.3 Per-Filter Trajectory Diagnostics

Aggregate statistics tell us *how well* each filter performs, but single-trial trajectory diagnostics reveal *why* and *how* each filter succeeds or fails. Figures 6.4 through 6.10 show trajectory diagnostics for trial 0. Each figure shows three representative state dimension timeseries (top row), two attractor projections (bottom left and center), and either the RMSE or control magnitude timeseries (bottom right). All diagnostics use the same truth trajectory and initial ensemble.

$N = 300$ PF diagnostic. Figure 6.11 shows a supplementary diagnostic for the uncontrolled PF at $N = 300$ particles (a separate run outside the main $N = 32$ evaluation). This is included to verify that the PF’s tracking failure is not simply a particle-count artifact.

6.3 Discussion

6.3.1 Per-Particle SR: Mechanism and Performance

The most striking result in Table 6.1 is the performance of the *per_particle* SR controller: $nRMSE\ 0.16 \pm 0.02$, a 77% reduction relative to the uncontrolled PF

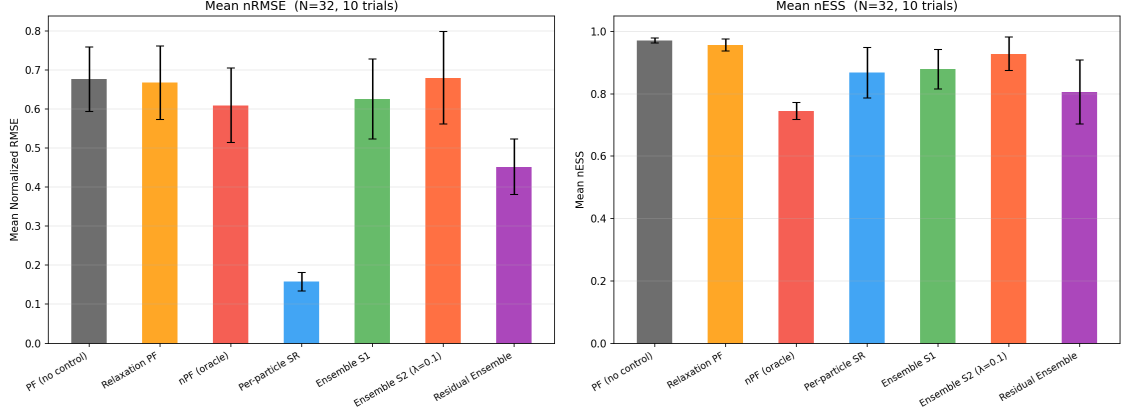


Figure 6.3: Trial-averaged nRMSE (left) and nESS (right) for all filter variants on Lorenz-63 ($N = 32$, 10 trials). Error bars are one standard deviation. Lower nRMSE and higher nESS are both desirable. The *per_particle* SR controller achieves the lowest nRMSE by a wide margin: 77% below the uncontrolled PF and $4\times$ below the oracle nPF, while maintaining competitive nESS.

(0.68 ± 0.08) and a $4\times$ improvement over the oracle nPF (0.61 ± 0.10), at $70\times$ lower computational cost.

This result is a direct consequence of the imitation learning framework working as intended. The network is trained by behavioral cloning on Feynman-Kac oracle controls: it observes a particle state, the upcoming observation, and ensemble summary statistics, and learns to output a control that matches what the oracle would have produced. At deployment, a single forward pass replaces the entire Monte Carlo computation. The Girsanov correction accumulates throughout the window regardless of where the control came from.

What makes this framework succeed where a naive approach would fail is the Girsanov correction. The learned controller is not simply approximating the oracle’s trajectory tracking: it is learning a policy that is compatible with importance sampling. A controller that accurately imitates the oracle’s aggressive controls would collapse nESS just as the oracle does. Instead, the Huber loss with $\delta = 1.0$ causes the learned controller to regress toward the median of the oracle distribution rather than the mean. Since the oracle distribution is heavy-tailed, meaning large controls occur occasionally at lobe switches, the median is substantially more moderate than the mean. The result is a learned policy whose control magnitudes ($\|\bar{u}\| \approx 4.8$) are calibrated rather than extreme, and whose Girsanov penalties are correspondingly smaller.

Comparing Figures 6.6 and 6.7 makes this mechanism visually clear. The oracle

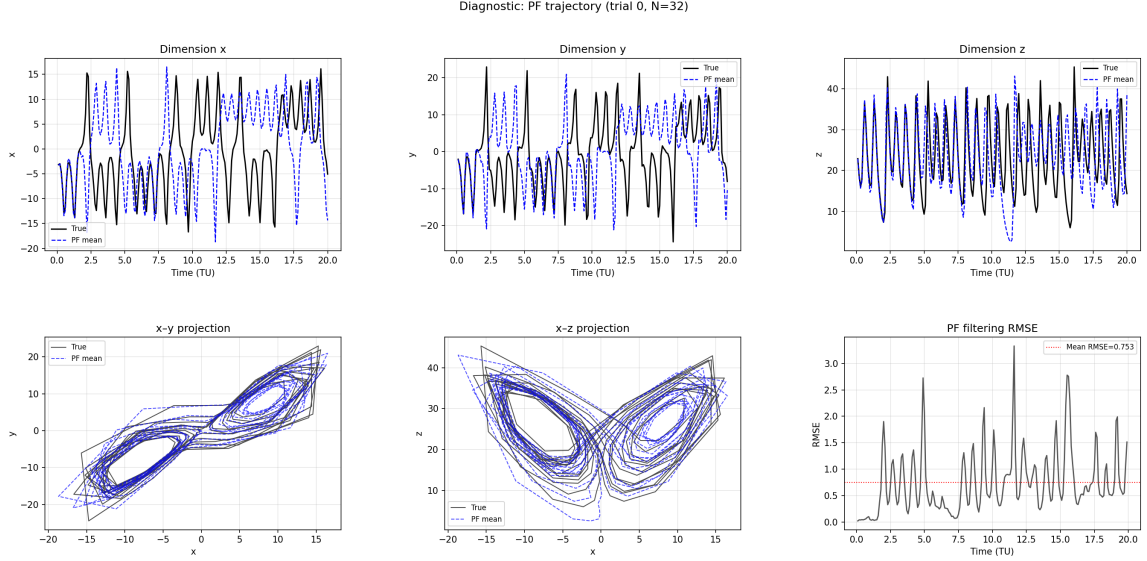


Figure 6.4: Uncontrolled PF diagnostic ($N = 32$, $\sigma_{\text{obs}} = 2.0$, trial 0). The posterior mean (dashed blue) loses phase with the true state (solid black) repeatedly across all three dimensions, most severely at lobe switching events. The RMSE timeseries (bottom right) shows large spikes whenever the ensemble drifts to the wrong lobe. The attractor projections confirm that the posterior mean frequently occupies the wrong lobe. Mean trial RMSE is 0.753 for this trial.

nPF applies large controls throughout and its nESS (bottom right of Figure 6.6) oscillates violently, frequently dropping toward the resampling threshold. The per-particle SR controller applies moderate controls with occasional targeted spikes at lobe boundaries (bottom right of Figure 6.7), and its nESS is stable and high throughout. Both controllers are guided by the same oracle labels; the difference is that the learned controller has implicitly internalized a more conservative policy through the Huber loss. This is what we call *implicit Girsanov regularization*: the training objective inadvertently teaches the controller to avoid the aggressive controls that would collapse the ensemble, even without any explicit penalty for weight variance.

6.3.2 Ensemble Variants: Collective Information vs Noise

The three ensemble-aware variants tell a more nuanced story. Looking at Figure 6.8, *ensemble_s1* shows the same qualitative failure as the uncontrolled PF: repeated phase loss at lobe switches. The mean control magnitude (≈ 3.8) is actually lower than the per-particle SR (≈ 4.8), suggesting that the pooled context g is pulling the controller toward more conservative behavior rather than helping it apply stronger

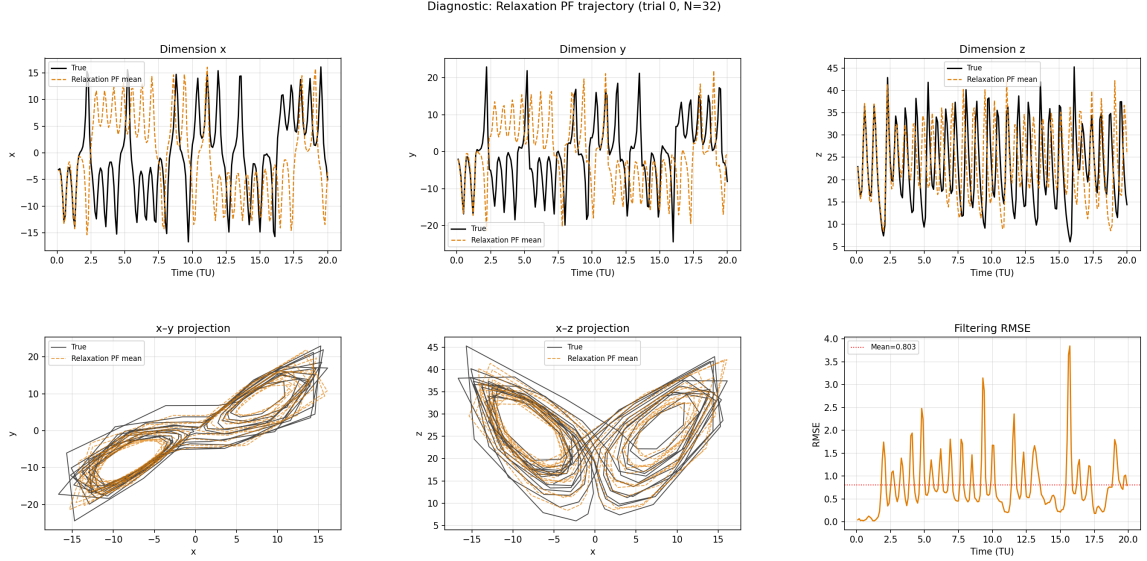


Figure 6.5: Relaxation PF diagnostic ($N = 32$, trial 0). The static Kalman gain nudge of van Leeuwen [70] produces only marginal improvement over the uncontrolled PF (mean RMSE 0.803 vs 0.753 for this trial). The posterior mean shows the same qualitative failure: phase loss at lobe switches, a posterior mean that lags the truth through both lobes, and episodic RMSE spikes. The gain $K = 0.2 \cdot I_3$ is too conservative to substantially change which particles survive the weight update in the chaotic regime.

corrections when needed.

The reason is that the pooled context at $N = 32$ particles is a noisy estimate of the true ensemble distribution. The weighted mean embedding $\bar{h}$ and max-pool embedding $h_{\max}$ are each computed from 32 particles, which may or may not provide reliable collective information about the current attractor lobe. When the ensemble is near a lobe switch, approximately half the particles may be on the wrong lobe, and the pooled context will reflect this confusion rather than providing a clear directional signal.

Ensemble S2 ($\lambda = 0.1$) tells a more informative story. Figure 6.9 shows that the Stage 2 weight-balance penalty successfully modifies the controller’s behavior: mean control magnitude increases slightly to ≈ 4.1 while nESS improves to 0.93, the highest of any variant. The differentiable Girsanov penalty is providing a real training signal: the controller is learning to avoid the specific control patterns that cause the most weight variance. However, this comes at the cost of the larger occasional corrections needed to track through lobe switches, so nRMSE remains poor (0.68). This is the fundamental tradeoff that the ensemble variants are navigating: conservative controls

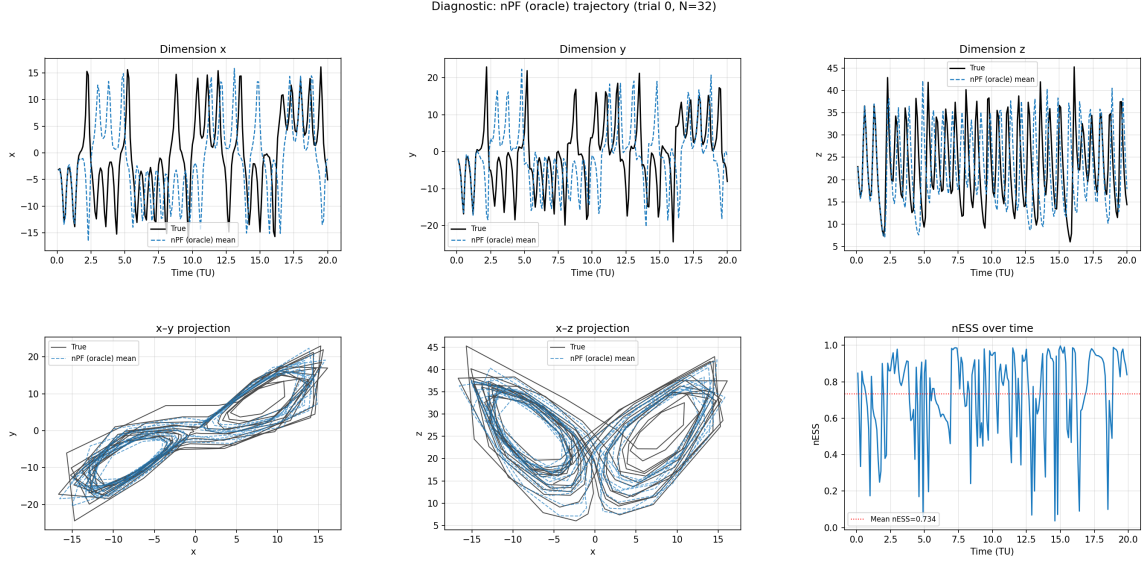


Figure 6.6: Oracle nPF diagnostic ($N = 32$, trial 0). Tracking improves relative to the classical baselines, but the cost is visible in the nESS timeseries (bottom right): nESS drops repeatedly toward and below the resampling threshold (red dashed), with mean nESS 0.734 for this trial. The Feynman-Kac controls aggressively compress the ensemble to reduce RMSE, repeatedly trading weight diversity for tracking accuracy. Despite the high computational cost of 30,518 ms per step, the oracle still produces phase errors in the x and y dimensions.

preserve ensemble diversity but sacrifice tracking accuracy, while aggressive controls improve tracking but collapse the ensemble.

The *residual_ensemble* (Figure 6.10) is the most informative variant. Initialized from the per-particle checkpoint, it starts with the strong per-particle policy and adds a small trainable ensemble correction. Its mean control magnitude (≈ 9.9) is the highest of the ensemble variants, confirming that the per-particle base is being partially preserved. Its nRMSE (0.45) is better than all classical baselines and all other ensemble variants. However, it is still $2.8\times$ worse than the per-particle baseline, indicating that the ensemble correction pathway introduces training noise that partially corrupts the strong initialization. The residual architecture is helpful but not sufficient: the fundamental limitation is not the architecture but the fact that the training labels carry no ensemble-coordination signal. The oracle generates each particle’s control independently, so there is no ground truth for what ensemble-aware behavior should look like.

The central lesson from all three ensemble variants is that ensemble-awareness must enter through the training data, not just through the architecture. A per-

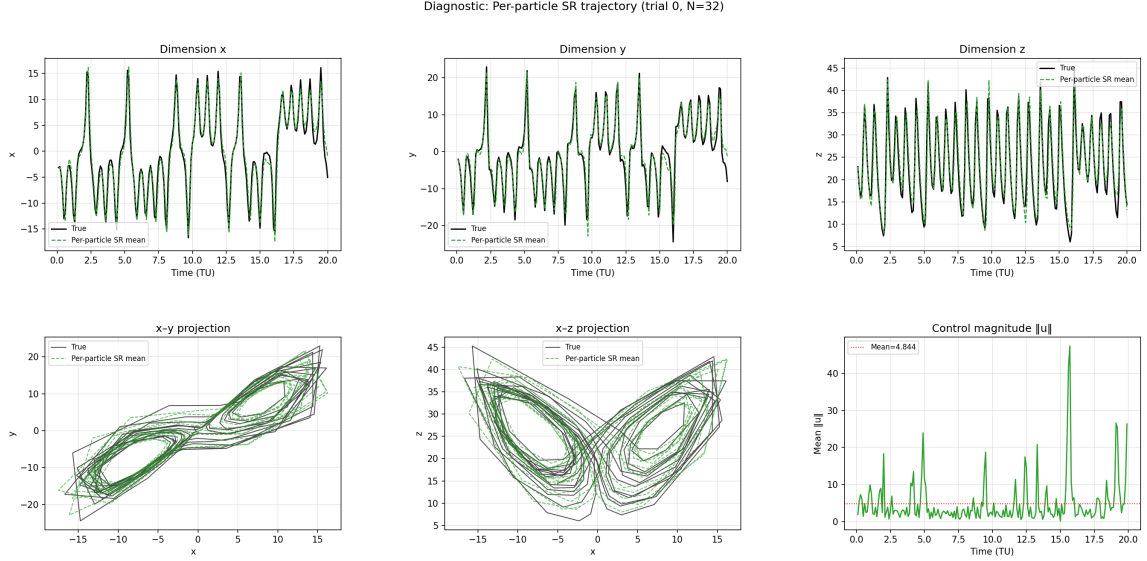


Figure 6.7: *Per-particle SR* diagnostic ($N = 32$, trial 0). The posterior mean (dashed green) tracks the true trajectory closely across all three dimensions throughout the full 20 TU window, including through the lobe-switching events that defeat every other variant. The attractor projections show the mean following the correct orbit with high fidelity. The control magnitude timeseries (bottom right) shows a mean of ≈ 4.8 , with occasional spikes to ≈ 40 – 50 at attractor lobe boundaries. These rare large corrections are applied precisely when a switch is occurring, and the Girsanov correction ensures that their cost is properly accounted for in the importance weights.

particle model trained on oracle labels already benefits implicitly from ensemble information, because the oracle’s optimal control for each particle implicitly depends on the observation and the system dynamics that determine where the true state is. An architecture that explicitly pools over particles is only as useful as the signal available in the pool, and at $N = 32$ particles in a sparse observation regime, the pool is too noisy to add reliable collective information.

6.3.3 What Does the Oracle Gap Tell Us?

The oracle nPF achieves nRMSE 0.61 at 30,518 ms per observation step. The *per_particle* SR controller achieves nRMSE 0.16 at 438 ms per step: a $70\times$ speedup and a $3.75\times$ improvement in accuracy. This inversion of a **learned approximation substantially outperforming its teacher on the downstream metric** is perhaps the most important finding in this thesis.

It arises because the oracle and the learned controller are being evaluated on different objectives. The oracle minimizes expected weight variance for a given particle

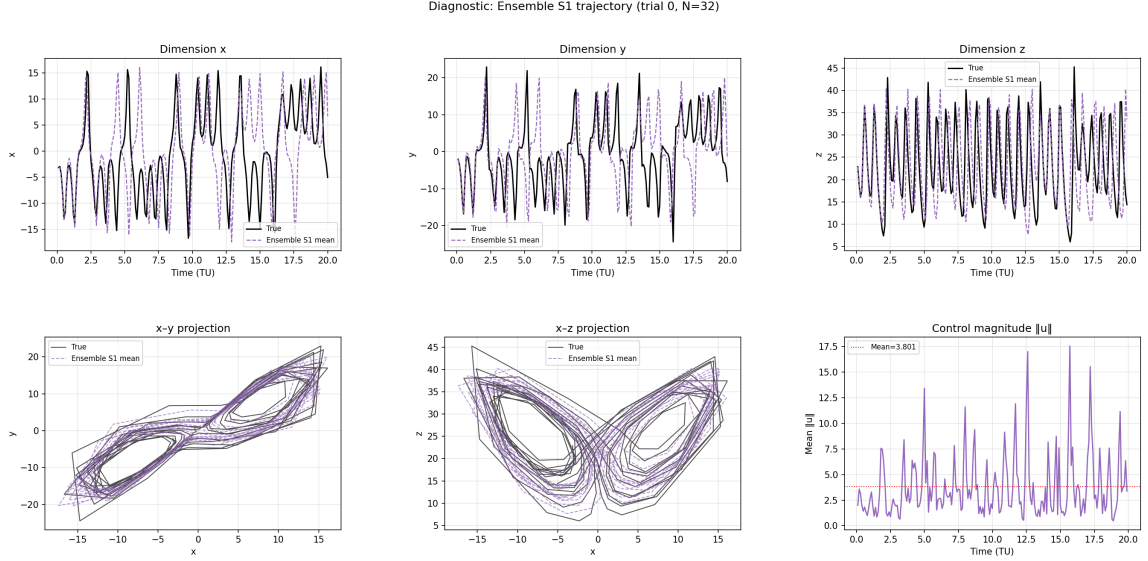


Figure 6.8: *Ensemble S1* diagnostic ($N = 32$, trial 0). The posterior mean (dashed purple) shows the same qualitative failure as the uncontrolled PF: repeated phase loss at lobe switches. The mean control magnitude (≈ 3.8) is lower than the per-particle SR (≈ 4.8). The weight-aware pooled context g , estimated from only 32 particles, is too noisy to direct stronger corrections at the critical moments when a lobe switch is occurring. The ensemble-level information encoded in g does not provide a reliable enough signal to outperform the simpler per-particle approach.

starting position. The learned controller, having been trained with Huber loss on the oracle’s demonstrated controls, minimizes the average imitation error on the training distribution. Because the oracle’s control distribution is heavy-tailed (it applies very large controls at attractor boundaries), the Huber loss produces a policy that systematically underestimates the oracle’s largest controls. These large controls are precisely the ones that collapse nESS.

The result is that the bottleneck to filtering performance in this system and noise regime is not control accuracy but control aggressiveness. A model that produces moderate, well-calibrated nudges outperforms one that produces optimal-but-aggressive nudges, because the Girsanov correction penalizes the aggressive policy in the weight update. This is a consequence of the distributional nature of particle filtering: the filter’s performance depends on the entire weighted ensemble, not just on how accurately any single particle tracks the true state. A policy that sacrifices some per-particle tracking accuracy in exchange for maintaining a healthier ensemble produces better collective estimates.

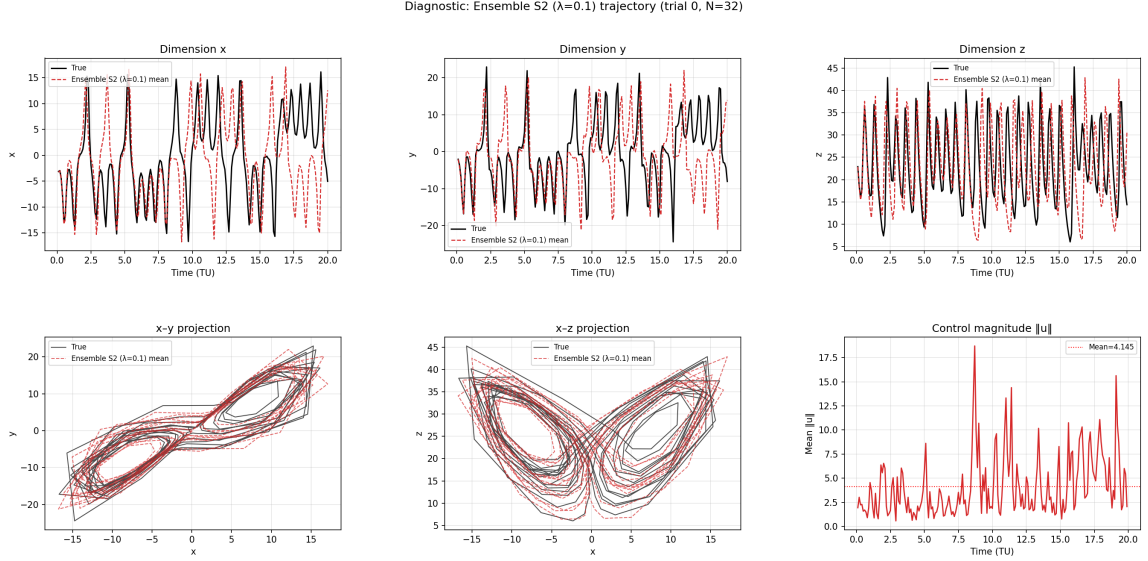


Figure 6.9: *Ensemble S2* ($\lambda = 0.1$) diagnostic ($N = 32$, trial 0). Qualitatively similar to S1 with comparable phase-loss behavior. The weight-balance penalty reduces the mean control magnitude to ≈ 4.1 relative to S1’s 3.8, and slightly increases nESS to 0.93 (mean across trials). The effect is most visible in the z dimension, where S2 tracks the oscillation amplitude more conservatively than per-particle SR. The Stage 2 penalty successfully teaches more conservative controls and improves ensemble diversity, but not at the cost needed to track through lobe switches.

6.3.4 Why is the Oracle-SR Gap Larger on Lorenz-63 Than Lorenz-96?

A natural follow-up question, which will be sharpened by the results in Chapter 7, is why the per-particle SR controller beats the oracle by $4\times$ on Lorenz-63 but only matches it on Lorenz-96.

The answer lies in the geometry of the two attractors and how aggressively the oracle overshoots in each case. On Lorenz-63, the butterfly attractor has two lobes connected by a narrow switching region. When the true state is approaching a lobe switch, the optimal nudging direction changes rapidly within a single assimilation window, and the Feynman-Kac oracle’s $n_s = 15$ uncontrolled rollouts are locally optimal for those specific 15 paths but too aggressive for the actual stochastic trajectory. The result is a significant Girsanov correction that collapses nESS to 0.75. The per-particle SR model, by systematically underestimating the oracle’s largest controls through Huber regression, learns controls that preserve nESS at 0.87 while still tracking well.

On Lorenz-96 at $K = 8$, $F = 8$, the dynamics have a more quasi-linear advection

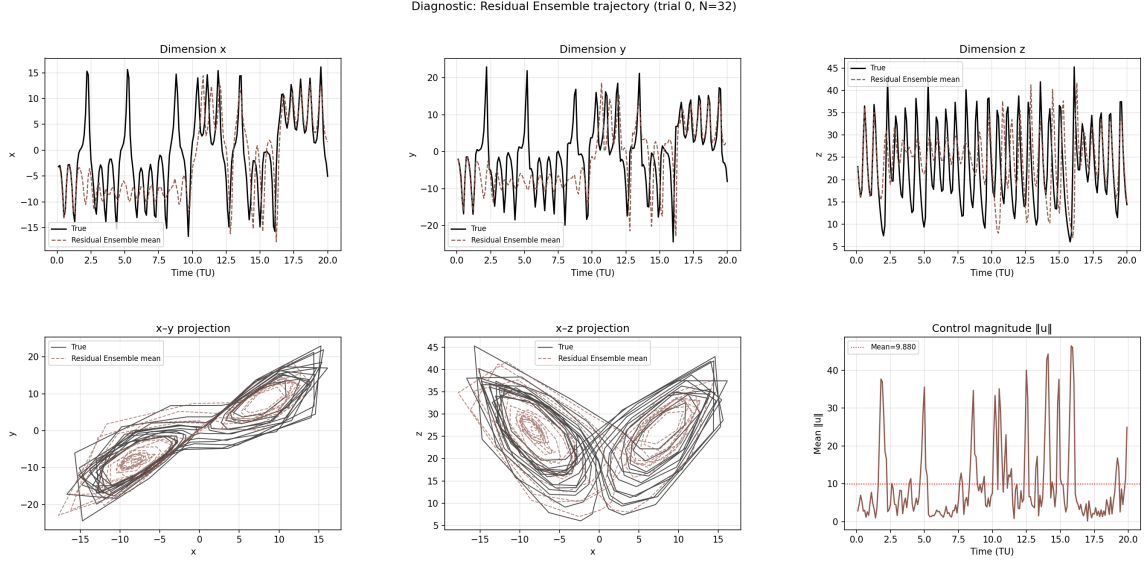


Figure 6.10: *Residual ensemble* diagnostic ($N = 32$, trial 0). The posterior mean (dashed brown) tracks the true trajectory noticeably better than S1 and S2, consistent with its intermediate nRMSE of 0.45. The mean control magnitude (≈ 9.9) is the highest of all ensemble variants, reflecting the strong per-particle SR base that is partially preserved through the residual initialization. The attractor projections show the mean remaining on the correct lobe for longer stretches than S1/S2. However, the residual correction still introduces enough training noise to prevent full recovery of the per-particle base’s performance.

structure. The per-particle innovation $\iota^{(i)} = x^{(i)} - y$ is already a near-sufficient statistic for the optimal control direction in most situations. The oracle’s controls are therefore more reliably aligned with the actual trajectory, and its nESS degradation is less severe (0.690 for L96 versus 0.750 for L63). Because the oracle does not overshoot as badly on L96, the implicit regularization advantage of SR’s regression-to-mean is smaller: SR matches the oracle rather than beating it. This is consistent with the observation that Lorenz-96 is structurally simpler than its apparent complexity suggests [59], with near-linear relationships governing much of its dynamics at $K = 8$.

The particle count difference between systems ($N = 32$ for L63 vs $N = 16$ for L96) is a secondary factor. Fewer particles means a noisier Monte Carlo estimate for the oracle, which could slightly degrade its quality. However, the oracle’s nRMSE on L96 (0.623) represents a 47% reduction over the L96 PF baseline (1.178), compared to only a 12% reduction on L63 (0.61 vs 0.68), suggesting that the oracle is actually performing better relative to its baseline on L96. The oracle quality is therefore not the primary driver of the gap difference between systems.

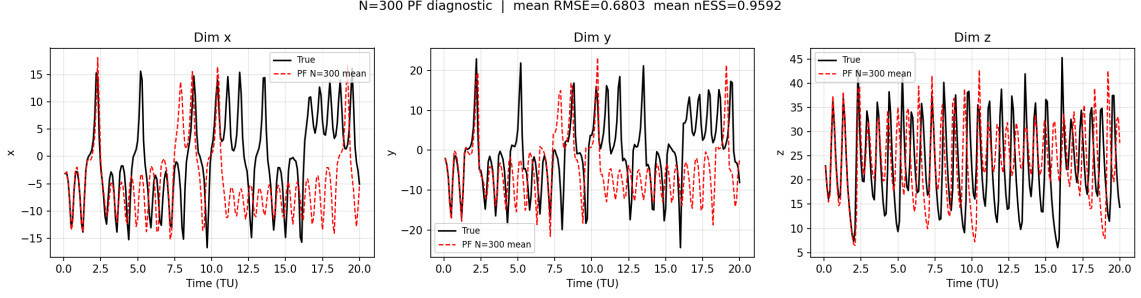


Figure 6.11: Uncontrolled PF at $N = 300$ particles (supplementary diagnostic run, not part of the main evaluation). The posterior mean (red dashed) still loses phase at lobe-switching events despite the $9\times$ larger ensemble. Mean nRMSE is 0.68, essentially identical to the $N = 32$ result. This confirms that the PF’s failure is a degeneracy problem driven by the observation likelihood geometry: at $\sigma_{\text{obs}} = 2.0$ and $\Delta T = 0.1$ TU, adding more uncontrolled particles does not solve the weight collapse problem. The solution requires nudging, not just a larger ensemble.

6.3.5 Summary of Findings

The closed-loop experiments on Lorenz-63 yield four conclusions.

First, the imitation learning framework produces a per-particle SR controller that achieves $\text{nRMSE } 0.16 \pm 0.02$, a 77% improvement over the uncontrolled PF, through a combination of Feynman-Kac oracle labels and the Girsanov correction at deployment.

Second, the learned per-particle controller beats the oracle nPF by $4\times$ in nRMSE while maintaining higher nESS (0.87 vs 0.75). This is explained by the implicit Girsanov regularization induced by Huber regression on a heavy-tailed oracle distribution: the learned policy avoids the aggressive controls that would collapse the ensemble, even without an explicit penalty for doing so.

Third, architectural ensemble-awareness via DeepSets pooling does not improve performance at $N = 32$ particles. The pooled context is too noisy at this ensemble size to provide reliable collective information that outweighs the noise it introduces. Ensemble-awareness must enter through the training data (the oracle), not just through the architecture.

Fourth, the residual ensemble architecture partially bridges the gap, achieving nRMSE 0.45 by preserving much of the per-particle base through residual initialization. However, the ensemble correction pathway still introduces enough training noise to prevent full recovery of per-particle performance, indicating that improving the oracle (rather than the architecture) is the higher-leverage intervention.

Chapter 7

Scaling to Lorenz-96

Chapter 6 established that per-particle SR dramatically improves filtering on Lorenz-63. A great addition to those results is assessing whether the same imitation learning pipeline scales to the Lorenz-96 system at $K = 8$ state variables with forcing $F = 8$. This is a proof-of-concept scaling investigation; full scaling to $K = 40$ is deferred to future work due to compute constraints. The four scaling questions are:

1. Can the oracle generate tractable training data?
2. Does the architecture work at $d = 8$?
3. Does per-particle SR still improve over the uncontrolled PF?
4. How does computational cost compare?

A deliberate constraint shapes the experimental scope. Dataset generation, model training, and closed-loop evaluation for Lorenz-96 were all conducted on Google Colab rather than a dedicated cluster, reflecting a practical limitation of this project: Della queue times made iterative experimentation infeasible, and the reduced dataset size (400 examples) required for Lorenz-96 fits comfortably within a single Colab session. As a result, only two ML variants are evaluated on Lorenz-96 (*per_particle* and *ensemble_s1*); the Stage 2 weight-balance penalty and residual architecture are left for future work.

7.1 System Configuration & Experimental Design

7.1.1 Lorenz-96 at $K = 8$, $F = 8$

The Lorenz-96 model at $K = 8$, $F = 8$ has approximately 3 to 4 positive Lyapunov exponents, an error doubling time of roughly 1.7 TU, and an unstable-neutral subspace of dimension 3 to 4. This is substantially harder than Lorenz-63 while remaining computationally tractable for the oracle data generation and closed-loop evaluation conducted here.

7.1.2 Experimental Parameters

Process noise $\Sigma = I_8$, observation noise $R = I_8$, identity observation operator $H = I_8$, observation interval $\Delta T = 0.05$ TU (5 integration substeps at $\delta t = 0.01$ TU per window), total assimilation window $T = 5.0$ TU ($N_{\text{steps}} = 100$), $N = 16$ particles, $N_{\text{trials}} = 10$. The initial condition is $x_k = F + 0.01\mathcal{N}(0, 1)$ for $k = 1, \dots, 8$.

7.1.3 Filter Variants

Five filter variants are evaluated: the uncontrolled PF, the relaxation PF, the oracle nPF, *per_particle*, and *ensemble_sl*. All models are retrained from scratch on Lorenz-96 data; no weights are transferred from the Lorenz-63 checkpoints because the state dimensions are architecturally incompatible.

7.2 Data Generation & Architecture

Dataset generation for Lorenz-96 follows the same pipeline as Lorenz-63 with three key adaptations. First, the oracle uses $K_c = 4$ control substeps (vs 8) and $n_s = 10$ rollouts per particle (vs 15) to keep per-label cost tractable; each rollout integrates the $(d + d^2) = 72$ -dimensional augmented system, a factor of 6 larger than the 12-dimensional Lorenz-63 system. Second, the time horizon grid is shortened to $\mathcal{T} = \{0.05, 0.10\}$ TU to match the faster L96 timescale. Third, the full 400-example dataset is generated in a single Google Colab session (approximately 15 to 30 minutes on an A100 or T4 GPU) using `datagen_196.ipynb`.

The encoder input grows to $3d + 3 = 27$ dimensions for the ensemble variants (Section ??). All hidden layer widths are kept identical to the Lorenz-63 architecture,

so the architecture generalizes to higher state dimension with no structural changes beyond input and output layer sizes.

7.3 Closed-Loop Results

7.3.1 Summary Statistics

Table 7.1 reports the primary performance metrics averaged over $N_{\text{trials}} = 10$ Monte Carlo trials.

Filter	$\bar{e}$ (nRMSE)	nESS	ms/step
PF (uncontrolled)	1.178 ± 0.068	0.967 ± 0.009	103.5
Relaxation PF	1.054 ± 0.129	0.964 ± 0.013	134.4
Oracle nPF	0.623 ± 0.117	0.690 ± 0.010	3,252.9
<i>per_particle</i>	0.660 ± 0.104	0.958 ± 0.026	119.7
<i>ensemble_s1</i>	0.721 ± 0.152	0.946 ± 0.036	117.6

Table 7.1: Lorenz-96 closed-loop filtering results ($K = 8$, $F = 8$, $N = 16$ particles, $T = 5$ TU, $\Delta T = 0.05$ TU, $\sigma_{\text{obs}} = 1.0$). Mean $\pm$ std over 10 Monte Carlo trials.

7.3.2 Time Evolution of nRMSE and nESS

Figure 7.1 shows the time evolution of nRMSE and nESS for all five filters.

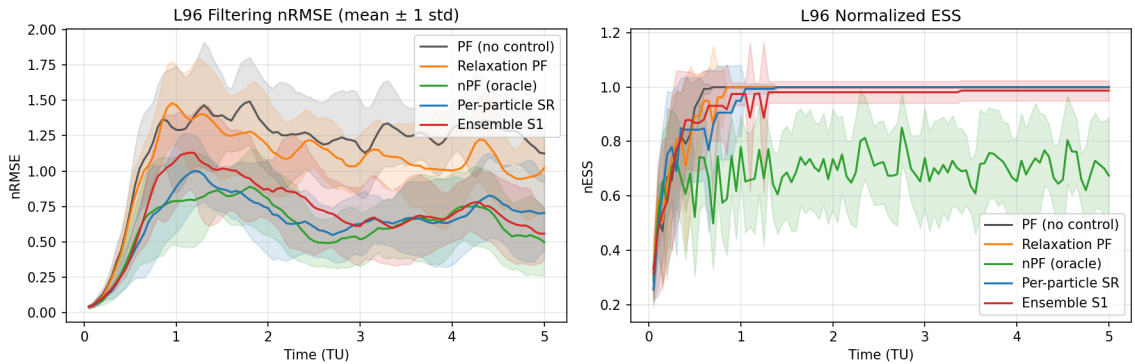


Figure 7.1: Time evolution of nRMSE (left) and nESS (right) for all filter variants on Lorenz-96 ($K = 8$, $F = 8$, $N = 16$ particles). Trial mean with ± 1 std shaded bands over 10 Monte Carlo trials. The oracle nPF achieves the lowest nRMSE but also the lowest nESS; both SR variants maintain near-unity nESS while tracking the true state.

7.3.3 Summary Bar Chart and Runtime

Figures 7.2 and 7.3 show the scalar performance summary and the per-step wall-clock times.

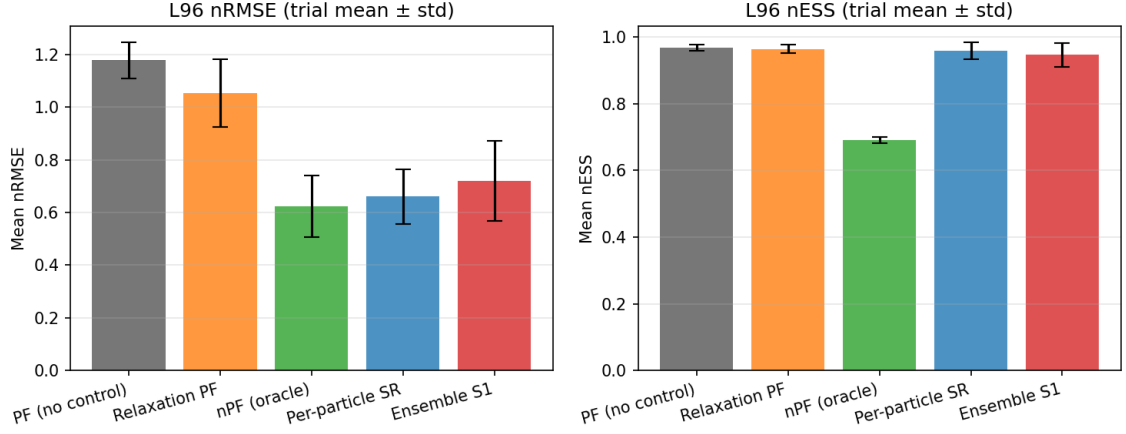


Figure 7.2: Trial-averaged nRMSE (left) and nESS (right) for all filter variants on Lorenz-96. Error bars are one standard deviation over 10 trials.

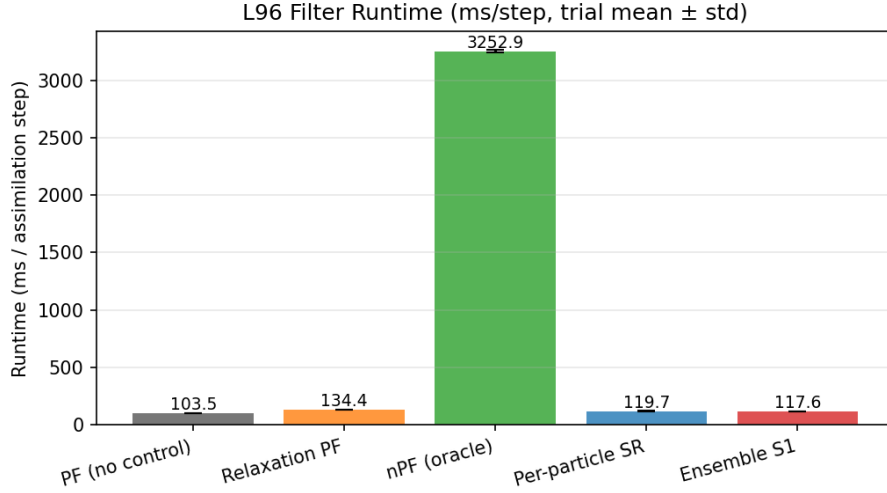


Figure 7.3: Mean wall-clock time per observation step for all filter variants on Lorenz-96 ($K = 8$, $N = 16$). The oracle nPF requires 3,253 ms per step; both learned controllers run at approximately 120 ms, a $27\times$ speedup.

7.4 Discussion

7.4.1 The Four Scaling Questions

Question 1: Oracle tractability. Dataset generation for Lorenz-96 at $K = 8$ is tractable within a single Colab session. The 400-example dataset takes 15 to 30 minutes to generate, demonstrating that the pipeline generalizes to higher-dimensional systems without requiring large-scale cluster resources at this scale.

Question 2: Architecture at $d = 8$. Both learned controllers converge in training, confirming that the 27-dimensional encoder handles the 8-dimensional state adequately. The architecture generalizes to Lorenz-96 with no structural changes beyond input and output layer dimensions.

Question 3: Why SR matches but does not beat the oracle on Lorenz-96.

On Lorenz-63, per-particle SR beats the oracle by $4\times$ in nRMSE; here it merely matches it (0.660 vs 0.623, well within one standard deviation of ± 0.117). This is not a failure of SR on Lorenz-96 but a consequence of the L96 attractor’s structure. As discussed in Section 6.3.4, the quasi-linear advection dynamics of Lorenz-96 at $K = 8$, $F = 8$ mean that the oracle’s Feynman-Kac controls are more reliably aligned with the actual trajectory than on Lorenz-63. The oracle therefore does not overshoot as badly, its nESS degradation is more moderate (0.690 vs 0.750 on L63), and SR’s implicit regularization advantage from regression-to-mean on a heavy-tailed distribution is correspondingly smaller. The result is that SR achieves parity with the oracle on RMSE while maintaining dramatically better nESS (0.958 vs 0.690) - a genuinely useful outcome that demonstrates importance-sampling validity rather than control accuracy as the binding constraint on filtering performance. *Per_particle* SR achieves nRMSE 0.660 ± 0.104 , a 44% reduction relative to the uncontrolled PF (1.178 ± 0.068). Crucially, it nearly matches the oracle nPF (0.623 ± 0.117) on RMSE while maintaining dramatically higher nESS (0.958 vs 0.690). The same pattern observed on Lorenz-63 - the learned controller producing moderate controls that preserve weight diversity - holds at $K = 8$.

The *ensemble_s1* controller (0.721 ± 0.152) is again outperformed by the per-particle baseline. At $N = 16$ particles, the pooled context is even noisier than at $N = 32$, and the result confirms the Lorenz-63 finding: at small ensemble sizes, architectural ensemble-awareness does not provide reliable collective information.

Question 4: Computational cost. The oracle nPF requires 3,253 ms per observation step. Both learned controllers run at approximately 120 ms per step, a $27\times$ speedup, which is smaller than the $\approx 70\times$ speedup on Lorenz-63. The reason is not dimensional: N enters both numerator and denominator of the ratio and cancels. The shrinkage is instead driven by the reduced oracle inner-loop configuration used for L96 data generation, $K_c = 4$ and $n_s = 10$ (versus 8 and 15 on L63), which decreases the oracle’s per-step integration count by a factor of $(8 \cdot 15)/(4 \cdot 10) = 3\times$. Held at matched inner-loop budgets, the L96 speedup would be approximately $80\times$.

The structural scaling runs the opposite direction. The oracle integrates a $(d+d^2)$ -dimensional augmented system per Monte Carlo rollout, growing from 12 dimensions on L63 to 72 on L96 (a $6\times$ per-integration cost increase, or more if dense matrix operations dominate). The MLP forward-pass cost grows only with the encoder input dimension ($12 \rightarrow 27$ for `per_particle`), with hidden widths fixed at 128. Per-integration cost therefore scales as $\mathcal{O}(d^2)$ for the oracle versus $\mathcal{O}(d)$ for the learned controller, so the *structural* advantage of amortization grows with state dimension, thus the reported $27\times$ understates this because the oracle was run in a cheaper configuration, not because scaling hurt the learned controller.

7.4.2 Per-Filter Trajectory Diagnostics

Figures 7.4 through 7.8 show single-trial (trial 0) trajectory diagnostics for all five filter variants on Lorenz-96. Each figure shows three representative state dimensions (X_0, X_2, X_5) in the top row, two attractor projections (X_0-X_1 and X_0-X_4) in the bottom left and center, and either the nRMSE/nESS dual-axis timeseries or the mean control magnitude histogram in the bottom right.

Why all filters perform worse on Lorenz-96 than Lorenz-63. The trajectory diagnostics reveal a qualitatively different failure mode on Lorenz-96 that explains why all variants perform worse than their L63 counterparts, even relative to their respective baselines.

On Lorenz-63, filter failures are episodic and localized: they occur at lobe-switching events where the truth transitions between the two attractor lobes and an uncontrolled ensemble loses the correct lobe. Between switches, all filters track reasonably well. The per-particle SR controller succeeds precisely because it applies targeted corrections at these switch events.

On Lorenz-96 at $K = 8, F = 8$, the failure mode is distributed and persistent.

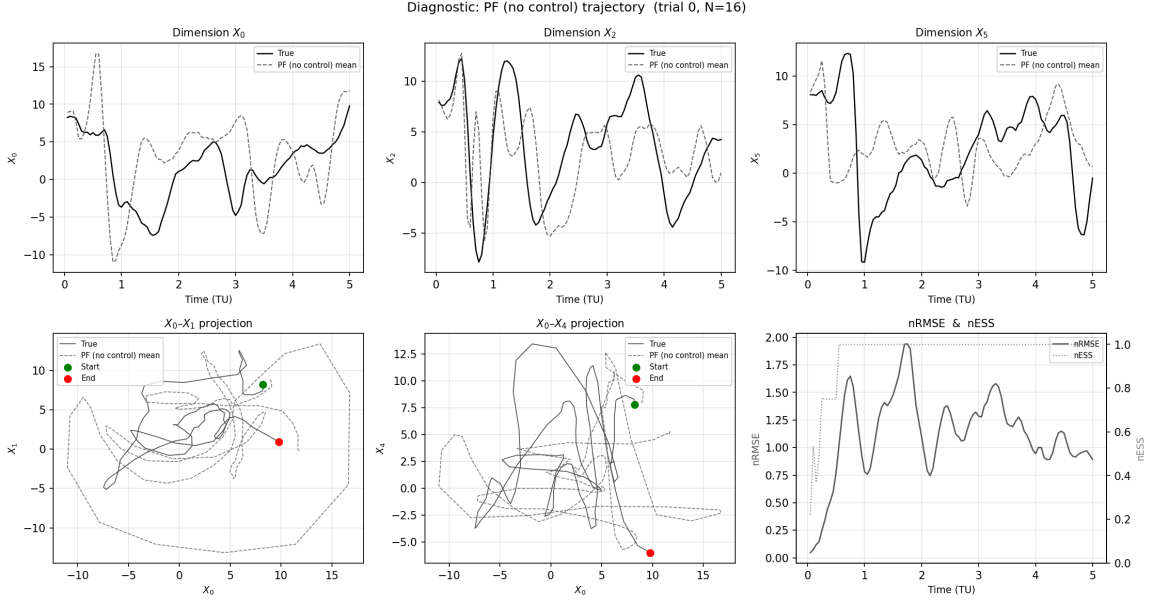


Figure 7.4: Uncontrolled PF diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The posterior mean (dashed) diverges sharply from the truth (solid) in all three shown dimensions by $t \approx 0.5$ TU. The nRMSE rises steeply to ≈ 2.0 within the first TU and never fully recovers. The 8D attractor projections show the mean tracing a completely uncorrelated orbit, confirming near-complete filter divergence. With only $N = 16$ particles and 3 to 4 positive Lyapunov exponents, the uncontrolled PF has insufficient ensemble mass to track the multiple unstable directions simultaneously.

With approximately 3 to 4 positive Lyapunov exponents, error grows simultaneously across multiple state dimensions rather than being concentrated at a single geometric event. The initial transient ($t \approx 0.5$ – 1.5 TU) is particularly damaging: the 16-particle ensemble must track an 8-dimensional chaotic attractor with multiple independent unstable directions, and any phase error in one dimension propagates to adjacent dimensions through the advection coupling. The nRMSE plots for the oracle and SR variants (Figure 7.6, bottom right) both show elevated error during this transient that only partially recovers afterward.

The oracle nPF’s violent nESS oscillations (Figure 7.6, bottom right dotted) are also more severe than on L63, for the same reason. With 3 to 4 positive Lyapunov exponents, the Feynman-Kac Monte Carlo estimate is noisier: $n_s = 10$ rollouts must collectively characterize the unstable subspace in 8 dimensions rather than 3, and the resulting controls overshoot more often. The per-particle SR model’s implicit regularization still helps, as it achieves parity with the oracle’s nRMSE while maintaining nESS 0.958 versus 0.690, however the absolute tracking quality is limited by the 8D degeneracy problem itself.

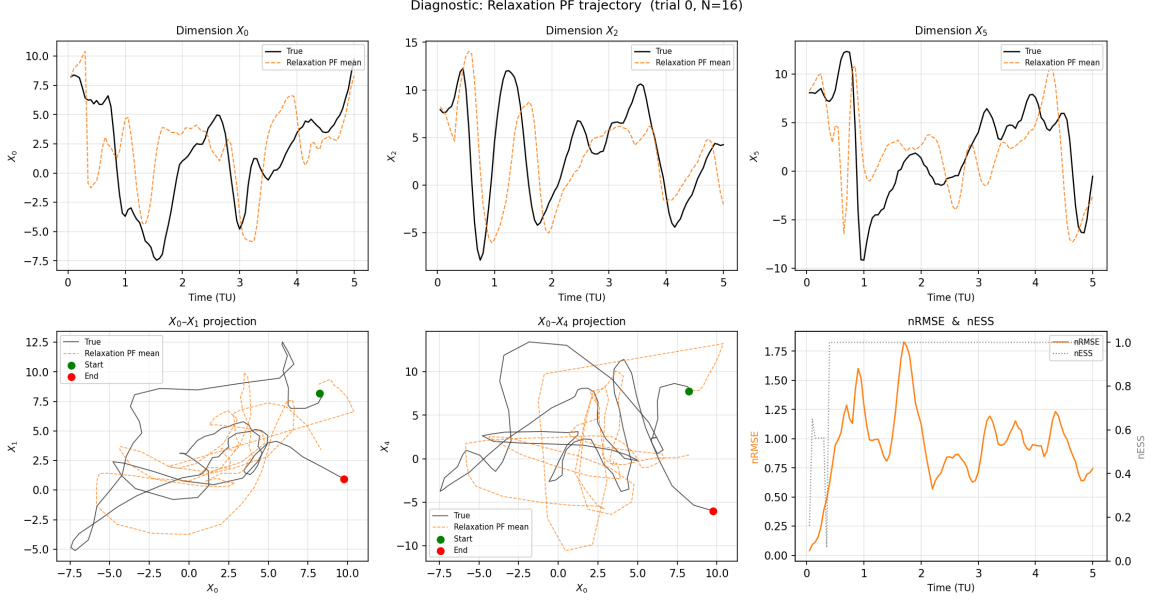


Figure 7.5: Relaxation PF diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The static Kalman gain nudge reduces mean nRMSE from 1.178 to 1.054 relative to the uncontrolled PF but does not prevent phase loss. The posterior mean consistently lags the truth in amplitude and phase across all three shown dimensions, and the nESS (right axis, dotted) drops repeatedly toward zero before resampling events restore it. The attractor projections show the mean following a similar but misaligned orbit, indicative of a persistent phase offset rather than complete divergence.

This difference in failure mode is also why ensemble-awareness matters more on Lorenz-96 in principle, even though it does not help in practice at $N = 16$. On L63 the key collective information is the lobe identity of the ensemble: which lobe is the true state on? On L96 the key collective information is the orientation of the ensemble’s unstable subspace across 8 dimensions: which directions are growing? The latter is harder to estimate from 16 particles than the former from 32, and the pooled context g at $N = 16$ is even noisier than at $N = 32$. Both factors compound to limit all variants on L96, and both point toward larger N , localized architectures exploiting the banded Jacobian structure, and higher-quality oracle labels as the path to improvement at $K = 40$.

7.4.3 Limitations & Future Directions

The Lorenz-96 experiments have three principal limitations. First, the evaluation is restricted to $K = 8$; extending to the standard $K = 40$ benchmark would require either a larger Colab allocation or Della compute budget for dataset generation (the

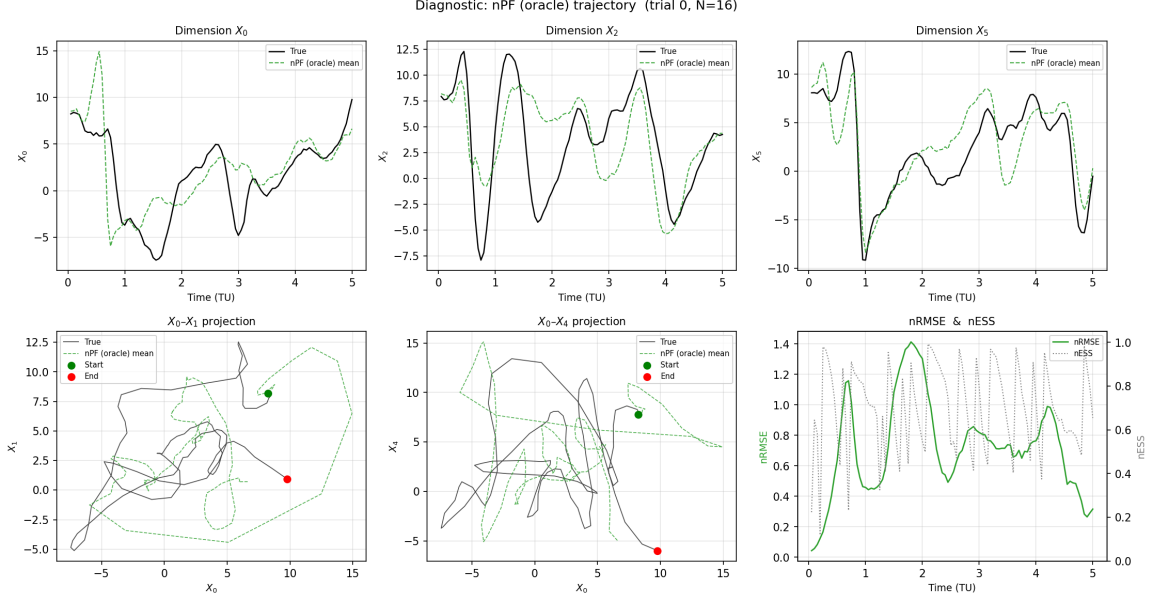


Figure 7.6: Oracle nPF diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The nESS (bottom right, dotted) oscillates violently between near-zero and unity throughout the 5 TU window, far more severe than the L63 oracle behavior. The aggressive Feynman-Kac controls repeatedly collapse and then resample the ensemble. Despite this, tracking remains imperfect: phase errors are visible in X_0 around $t \approx 1$ –2 TU and in X_2 after $t \approx 3$ TU. The projections show the mean tracing a largely correct but occasionally displaced orbit, with the nRMSE timeseries revealing sustained periods of elevated error between resampling events.

augmented system grows to $40 + 1600 = 1640$ dimensions per rollout). Second, no Stage 2 weight-balance penalty or residual ensemble variant is evaluated for Lorenz-96, as computational constraints precluded the additional training runs. Third, the full d -dimensional control vector applied here makes no use of the spatial locality of the Lorenz-96 coupling structure; a localized control formulation exploiting the banded Jacobian would reduce both oracle generation cost and model parameter count. These directions are taken up in Chapter 9.

7.4.4 Summary

The Lorenz-96 experiments demonstrate that the pipeline scales from $d = 3$ to $d = 8$ with no architectural changes. Oracle data generation is tractable in a single Colab session. Per-particle SR matches the oracle nPF on nRMSE (0.660 vs 0.623) at a $27\times$ lower computational cost, while maintaining much higher nESS (0.958 vs 0.690). The ensemble S1 architecture again underperforms the per-particle baseline, consistent

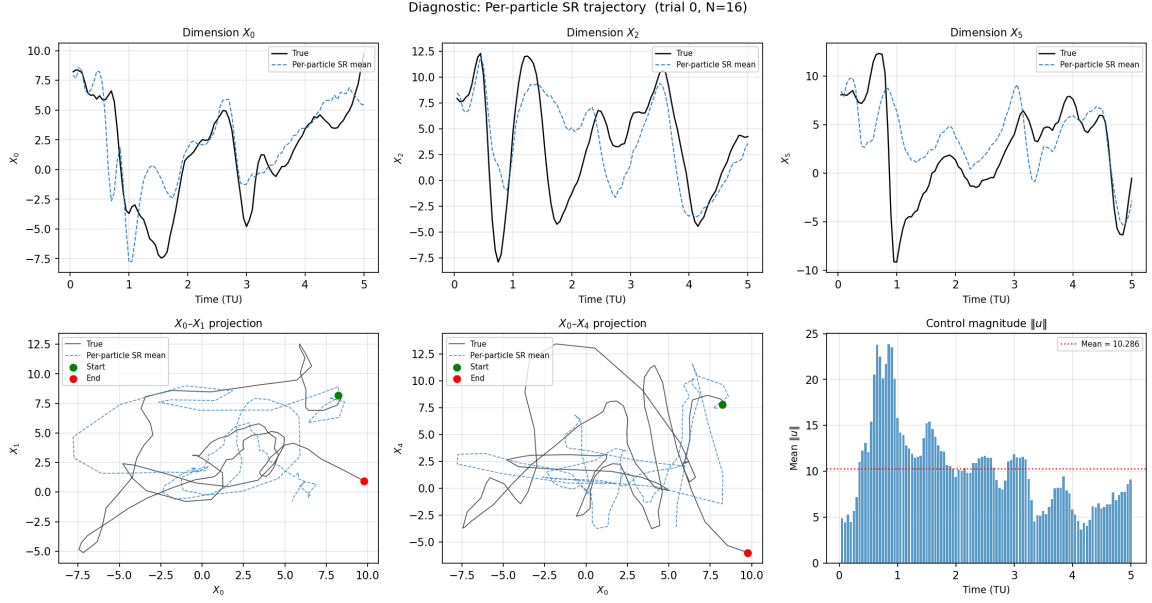


Figure 7.7: *Per-particle SR* diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). The posterior mean (dashed blue) follows the truth more closely than the uncontrolled PF and relaxation PF, but visible phase errors remain in X_0 and X_2 during $t \approx 1$ –2 TU. The mean control magnitude (bottom right histogram) averages ≈ 10.3 , substantially higher than in L63 (≈ 4.8), reflecting the harder 8D tracking problem. The attractor projections show partial alignment: the mean follows the correct qualitative structure but the tighter correspondence seen in L63 is absent.

with the Lorenz-63 finding that architectural ensemble-awareness at small particle counts adds noise rather than useful collective information.

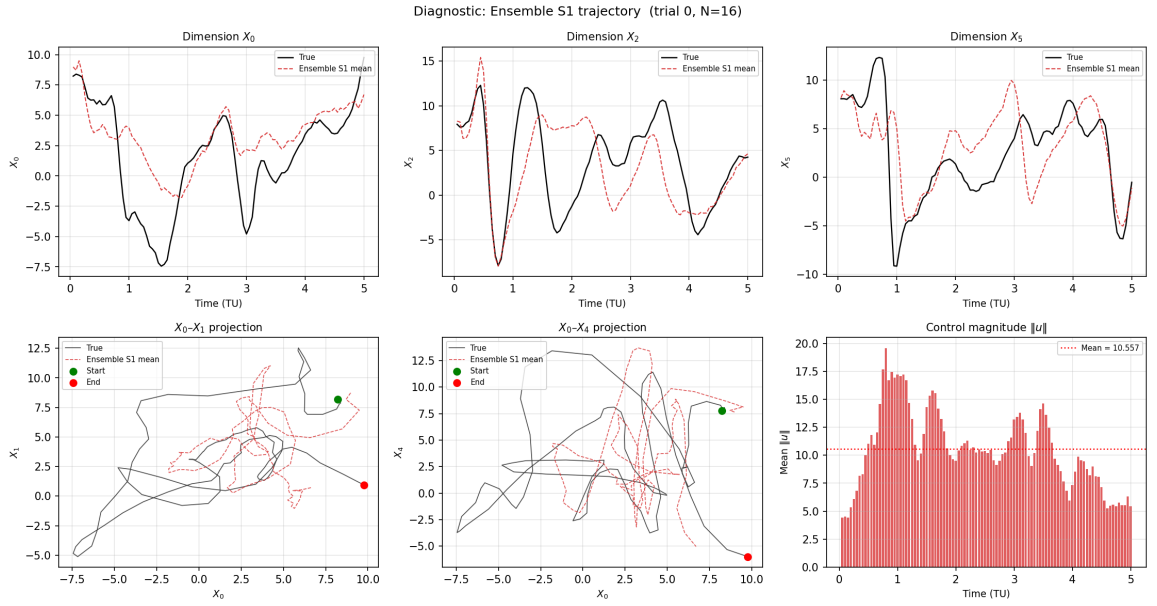


Figure 7.8: *Ensemble S1* diagnostic on Lorenz-96 ($K = 8$, $N = 16$, trial 0). Qualitatively similar to *per_particle* SR with comparable phase errors in X_0 and X_2 . The mean control magnitude (≈ 10.6) is slightly higher than per-particle SR, but the pooled context from $N = 16$ particles provides no discernible improvement in trajectory tracking.

Chapter 8

Ablation & Robustness Studies

Chapters 6 and 7 establish that per-particle SR dramatically improves filtering on both Lorenz-63 and Lorenz-96, but headline results alone cannot explain *why* the improvement occurs or how fragile it is. Three questions motivate the studies in this chapter:

1. Is the 77% nRMSE reduction on Lorenz-63 genuinely attributable to the learned control law, or is it a side effect of deployment-time clipping that would appear for any bounded controller?
2. Does the controller generalize to observation noise levels and particle counts it was not explicitly evaluated on, given that real deployments rarely match training conditions exactly?
3. Does the choice of oracle matter for ensemble-aware learning (model training), and specifically, could ensemble-coordinated labels from the IRnPF allow the pooling architecture to realize its theoretical potential?

Section 8.1 answers the first question through a clipping ablation using the same checkpoint across five thresholds. Sections 8.2 and 8.3 answer the second through noise-level and particle-count robustness evaluations. Section 8.4 addresses the third through a deliberate, compute-constrained hypothesis test comparing nPF and IRnPF oracle labels. Section 8.5 collects all findings.

8.1 Ablation on Control Clipping

What is being tested. The *per_particle* SR controller achieves nRMSE 0.16, 77% below the uncontrolled PF. One candidate explanation is that deployment-time clipping of the network output to $[-U_{\text{clip}}, U_{\text{clip}}]$ is the operative mechanism: by truncating large controls, clipping prevents any single observation step from applying extreme forces that would collapse the ESS. If this were true, the learned control law itself would be incidental, and the advantage could be replicated by clipping an arbitrary controller. The alternative is that the Huber regression training process learns to produce moderate, well-calibrated controls whose magnitudes rarely approach any reasonable clip threshold, making clipping a safety net rather than the mechanism.

Protocol. The same *per_particle_best.pt* checkpoint is used for all runs with no retraining. Five deployment-time clip values are tested: $U_{\text{clip}} \in \{25, 50, 100, 200, \infty\}$ on Lorenz-63 with $N = 32$, $T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, and $N_{\text{trials}} = 10$.

Results. Table 8.1 and Figure 8.1 report the results.

U_{clip}	$\bar{e}$ (nRMSE)	nRMSE std	$\bar{\text{nESS}}$
25 (aggressive)	0.187	0.041	0.860
50	0.158	0.025	0.866
100 (default)	0.158	0.024	0.867
200	0.159	0.025	0.888
∞ (none)	0.158	0.024	0.857

Table 8.1: Effect of deployment-time control clipping threshold on the *per_particle* SR controller. Same checkpoint, no retraining. Lorenz-63, $N = 32$, 10 Monte Carlo trials.

Finding. nRMSE is flat at approximately 0.158 across $U_{\text{clip}} \in \{50, 100, 200, \infty\}$, and degrades only at the aggressive threshold of 25 (nRMSE 0.187, a 19% increase). The mean per-step control magnitude produced by the network is approximately 8.5 in physical units, well below any threshold above 25. The learned controller naturally produces moderate controls through Huber regression on a heavy-tailed oracle: the training process implicitly learns to avoid the large controls that would collapse the ESS, regardless of the deployment-time threshold. Clipping is a safety net that prevents pathological edge cases, not the operative mechanism.

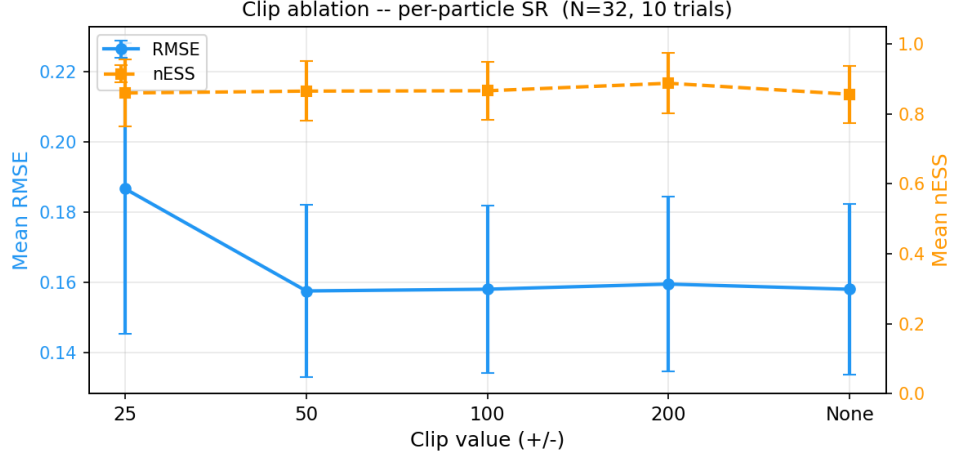


Figure 8.1: nRMSE (blue, left axis) and nESS (orange, right axis) as a function of deployment-time clip threshold for the *per_particle* SR controller. Performance is flat across $U_{\text{clip}} \in \{50, 100, 200, \infty\}$, degrading only at $U_{\text{clip}} = 25$. Error bars are ± 1 std over 10 trials.

8.2 Robustness to Observation Noise

What is being tested. The training dataset covers three noise levels: $\sigma_{\text{obs}} \in \{1.0, 2.0, 4.0\}$. The main closed-loop experiments use $\sigma_{\text{obs}} = 2.0$. This section tests whether the per-particle SR controller generalizes across noise levels, with σ_{obs} as an explicit encoder input so the model has the information needed to adapt its nudging strength. At $\sigma_{\text{obs}} = 1.0$ the observation is as precise as the process noise ($\sigma_{\text{dyn}} = 1.0$) and aggressive nudging is warranted; at $\sigma_{\text{obs}} = 4.0$ the observation is noisier than the process and conservative nudging is appropriate.

Protocol. The *per_particle* controller and uncontrolled PF are evaluated at $\sigma_{\text{obs}} \in \{1.0, 2.0, 4.0\}$ on Lorenz-63 with $N = 32$, $T = 20$ TU, and $N_{\text{trials}} = 10$. The $\sigma_{\text{obs}} = 2.0$ values are taken from the main evaluation. The $\sigma_{\text{obs}} = 4.0$ results are from the noise robustness experiment. The $\sigma_{\text{obs}} = 1.0$ experiment encountered `DeadParticleCloudError` across all 10 trials for the uncontrolled PF: at this noise level, $N = 32$ particles is insufficient for the bootstrap PF to maintain weight support over a 20 TU window, a diagnostic finding discussed below.

Results. Table 8.2 and Figure 8.2 report the results.

Finding. The per-particle SR controller maintains its advantage at $\sigma_{\text{obs}} = 4.0$: nRMSE 0.15 ± 0.02 versus 0.63 ± 0.08 for the uncontrolled PF, a 76% reduction

σ_{obs}	Filter	$\bar{e}$ (nRMSE)	nESS
2.0 (default)	PF (no control)	0.68 ± 0.09	0.97 ± 0.008
	<i>per_particle</i> SR	0.16 ± 0.02	0.87 ± 0.080
4.0 (high)	PF (no control)	0.63 ± 0.08	0.91 ± 0.054
	<i>per_particle</i> SR	0.15 ± 0.02	0.89 ± 0.052

Table 8.2: Closed-loop filtering performance at $\sigma_{\text{obs}} \in \{2.0, 4.0\}$ on Lorenz-63 ($N = 32$, $T = 20$ TU, 10 trials). $\sigma_{\text{obs}} = 1.0$ results are omitted: the uncontrolled PF fails at this noise level with $N = 32$ particles (see text).

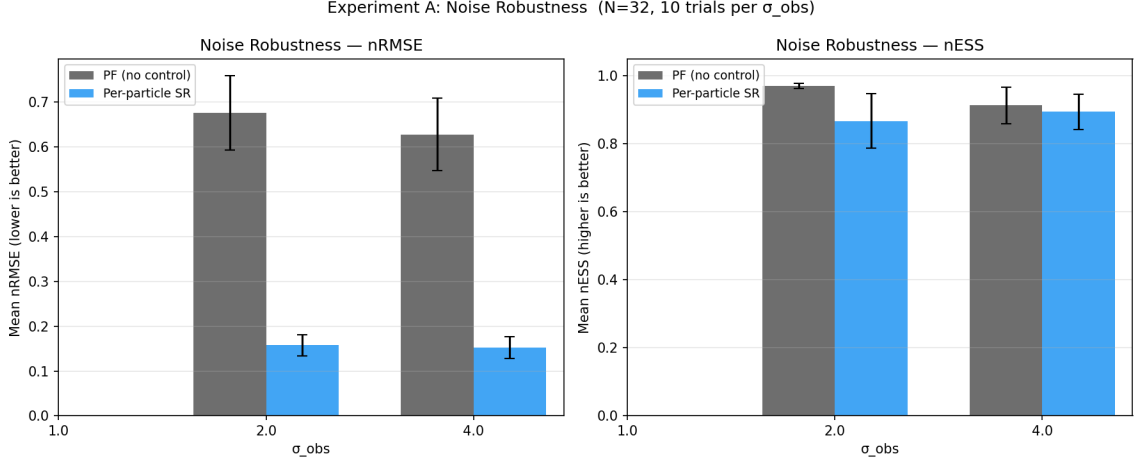


Figure 8.2: Noise robustness: nRMSE (left) and nESS (right) for the uncontrolled PF and *per_particle* SR at $\sigma_{\text{obs}} \in \{2.0, 4.0\}$. Error bars are ± 1 std over 10 trials. The $\sigma_{\text{obs}} = 1.0$ column is absent because the uncontrolled PF fails at this noise level with $N = 32$ particles.

essentially identical to the $\sigma_{\text{obs}} = 2.0$ result (77%). The nESS profiles are also consistent: the SR controller operates at $\text{nESS} \approx 0.87\text{--}0.89$ across both noise levels, indicating stable weight diversity regardless of observation quality.

The $\sigma_{\text{obs}} = 1.0$ failure of the uncontrolled PF is physically meaningful. At low noise, the observation likelihood is sharply peaked relative to the ensemble spread: any particle whose forecast lands outside a small observation-consistent region receives effectively zero weight, causing rapid degeneracy that $N = 32$ particles cannot recover from. The SR controller is not evaluated at $\sigma_{\text{obs}} = 1.0$ in isolation because without a working baseline there is no meaningful relative comparison. This motivates the particle count study in Section 8.3, where larger N would allow the PF to survive at lower noise levels.

8.3 Robustness to Particle Count

What is being tested. The training dataset and main evaluation both use $N = 32$ particles. Here the *per_particle* and *ensemble_s1* controllers, trained at $N = 32$, are deployed at $N \in \{16, 32, 64, 128\}$ alongside the uncontrolled PF and relaxation PF. The permutation-invariant pooling in *ensemble_s1* is well-defined for any N , but its statistical properties change: the weighted mean embedding is estimated from more particles at larger N , and the max-pool statistic is drawn from a larger sample and is systematically more extreme. Robustness to N is therefore an empirical question. For the *per_particle* model, which conditions only on per-particle features with no pooling, generalization is more straightforward since each particle’s input is independent of ensemble size.

Protocol. All four filters are evaluated at $N \in \{16, 32, 64, 128\}$ on Lorenz-63 with $T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, and $N_{\text{trials}} = 10$. The $N = 32$ results are loaded from the main evaluation checkpoint.

Results. Table 8.3 and Figure 8.3 report the results.

N	Filter	$\bar{e}$ (nRMSE)	nESS
16	PF (no control)	0.714 ± 0.099	0.966 ± 0.014
	Relaxation PF	0.700 ± 0.093	0.965 ± 0.014
	<i>per_particle</i>	0.166 ± 0.026	0.912 ± 0.053
	<i>ensemble_s1</i>	0.640 ± 0.094	0.917 ± 0.032
32	PF (no control)	0.676 ± 0.083	0.971 ± 0.008
	Relaxation PF	0.667 ± 0.094	0.957 ± 0.019
	<i>per_particle</i>	0.158 ± 0.024	0.868 ± 0.080
	<i>ensemble_s1</i>	0.625 ± 0.103	0.879 ± 0.063
64	PF (no control)	0.685 ± 0.076	0.952 ± 0.025
	Relaxation PF	0.683 ± 0.111	0.952 ± 0.013
	<i>per_particle</i>	0.153 ± 0.023	0.862 ± 0.066
	<i>ensemble_s1</i>	0.629 ± 0.072	0.925 ± 0.024
128	PF (no control)	0.694 ± 0.080	0.945 ± 0.019
	Relaxation PF	0.639 ± 0.104	0.953 ± 0.013
	<i>per_particle</i>	0.152 ± 0.023	0.819 ± 0.076
	<i>ensemble_s1</i>	0.670 ± 0.097	0.837 ± 0.114

Table 8.3: Closed-loop performance at varying particle counts on Lorenz-63 ($T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, 10 trials). Both ML models are trained at $N = 32$ for all configurations.

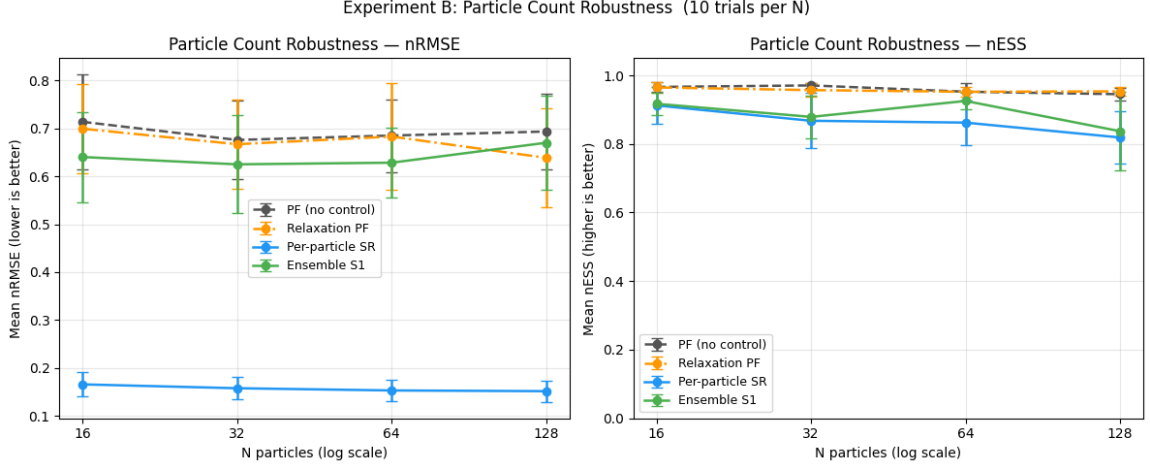


Figure 8.3: nRMSE (left) and nESS (right) as a function of particle count N for all four filter variants on Lorenz-63 ($T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, 10 trials). Both ML models are trained at $N = 32$. The *per_particle* SR controller maintains its advantage across all N , with nRMSE decreasing slightly from 0.166 at $N = 16$ to 0.152 at $N = 128$.

Finding. Three notable results are evaluated. First, the *per_particle* SR controller is highly robust to particle count: nRMSE decreases slightly and monotonically from 0.166 at $N = 16$ to 0.152 at $N = 128$, maintaining a $4\times$ advantage over the uncontrolled PF at every tested N . Since the per-particle model conditions only on individual particle features with no dependence on ensemble size, this robustness is expected theoretically and confirmed empirically.

Second, *ensemble_s1* nRMSE is nearly flat across $N \in \{16, 32, 64\}$ at approximately 0.63–0.64, then degrades slightly at $N = 128$ to 0.670. This is consistent with the out-of-distribution pooling hypothesis: at $N = 128$ the max-pool embedding h_{max} is drawn from a distribution more extreme than seen during training at $N = 32$, introducing a systematic shift in the global context g . The effect is modest but is a design limitation that larger- N training datasets would address.

Third, neither classical baseline improves meaningfully in nRMSE as N increases from 32 to 128. At $\sigma_{\text{obs}} = 2.0$ and $\Delta T = 0.1$ TU, the observation is sufficiently noisy that more particles do not substantially reduce filtering error. The SR advantage is therefore not a particle-count artifact; it persists even as the baselines approach their saturation regime.

8.4 IRnPF Oracle Labels: A Hypothesis Test

This section reports a brief, compute-constrained experiment motivated by the following question: does the choice of oracle matter for ensemble-aware learning? The standard nPF oracle computes each particle’s control independently using n_s uncontrolled rollouts from that particle’s state, so the training labels carry no ensemble-coordination signal. The intermediate-resampling nudged particle filter (IRnPF) applies systematic resampling between control substeps, making the control at substep $k + 1$ a function of the resampled ensemble from substep k and therefore genuinely ensemble-coordinated. The hypothesis is that training on IRnPF labels enables the ensemble-aware architecture to learn controls that are more compatible with weight diversity. This is not a full ablation study. Due to time and compute constraints, training datasets are deliberately small (200 examples for Lorenz-63, 50 for Lorenz-96), the Lorenz-96 evaluation uses a longer window ($T = 10$ TU vs $T = 5$ TU in Chapter 7) to amortize Colab session cost, and the Lorenz-96 nRMSE is normalized by $\sigma_{\text{obs}} = 1.0$ rather than $\|x^{\text{true}}\|$ to match the evaluation script’s output convention. These choices are deliberate: the goal is a rapid hypothesis test, not a definitive comparison. Results are reported transparently with those caveats in place.

8.4.1 Training Configuration

The L63 and L96 IRnPF training pipelines differ from each other and from the standard nPF training in three important ways, each motivated by the properties of IRnPF labels.

Multi-substep supervision (L63 only). The standard nPF training uses only the first substep of the oracle control sequence ($k = 0$), which is the cleanest estimate and precedes any state evolution under the window dynamics. For IRnPF, the key ensemble-coordination signal lives in substeps $k \geq 1$: it is only after intermediate resampling has occurred that the oracle’s control at the next substep reflects the resampled ensemble state. Training on only $k = 0$ would therefore discard the ensemble-coordination information that distinguishes IRnPF from nPF labels. The L63 IRnPF pipeline trains on all K_c substeps simultaneously by expanding the batch: each example of shape (B, N, K_c, d) is reshaped to $(B \cdot K_c, N, d)$, multiplying the effective number of training samples by $K_c = 8$ without requiring additional data generation. The L96 IRnPF pipeline retains the first-substep convention from the standard training, as a deliberate simplification given the 50-example dataset size.

Three additional encoder features (L63 *ensemble_s1* only). The L63 *ensemble_s1* IRnPF variant extends the encoder input from 15 to 18 dimensions by adding three features that carry information about the post-resampling ensemble structure:

- **Nearest-neighbour distance** $d_{\text{nn}}^{(i)}$: the Euclidean distance from particle i to its nearest neighbour in the current ensemble. After IRnPF resampling, the ensemble collapses to clusters of duplicate particles; $d_{\text{nn}}^{(i)} = 0$ signals that particle i is a resampling duplicate, allowing the controller to apply differential corrections.
- **Weight rank** $r^{(i)}$: the normalized rank of particle i 's importance weight within the ensemble, $r^{(i)} \in [0, 1]$. This encodes relative weight ordering without the scale sensitivity of raw weights.
- **Normalized ESS** $\widehat{\text{nESS}}$: the current ensemble nESS, broadcast as a scalar to all particles. This signals how recently resampling occurred and how healthy the ensemble currently is.

These features are motivated specifically by IRnPF's intermediate resampling dynamics and are not present in the standard nPF training. The L96 IRnPF *ensemble_s1* uses the standard 35-dimensional input without these additions, again as a deliberate simplification.

Training schedule. The L63 IRnPF models train for 40 epochs with no early stopping; the best validation checkpoint is saved inline and restored at the end of training. This is appropriate given the small 200-example dataset, where early stopping would terminate training before meaningful generalization can occur. The L96 IRnPF models use the standard 150-epoch schedule with patience-15 early stopping, matching the nPF training configuration. Table 8.4 summarizes the key differences between the IRnPF and nPF training configurations.

8.4.2 Results: Lorenz-63

Table 8.5 and Figures 8.4, 8.5, and 8.6 report the Lorenz-63 results.

The IRnPF-trained per-particle model achieves nRMSE 0.654 and nESS 0.954 — a striking reversal of the nPF trade-off profile. While the nPF per-particle model achieves nRMSE 0.158 and nESS 0.882, the IRnPF variant flips these: RMSE is 4×

	nPF L63	nPF L96	IRnPF L63	IRnPF L96
Training examples	4,800	400	200	50
Supervision target	$k = 0$ only	$k = 0$ only	all K_c substeps	$k = 0$ only
Encoder dim (per-particle)	12	27	12	27
Encoder dim (ensemble S1)	15	35	18	35
Epochs	150	150	40	150
Early stopping	patience 15	patience 15	none	patience 15
Batch size	256	32	32	16

Table 8.4: Training configuration comparison: nPF vs IRnPF, L63 vs L96.

Method	$\bar{e}$ (nRMSE)	nESS
PF (no control)	0.668 ± 0.073	0.969 ± 0.008
Per-particle SR (nPF)	0.158 ± 0.025	0.882 ± 0.092
Ensemble S1 (nPF)	0.653 ± 0.120	0.888 ± 0.078
Per-particle SR (IRnPF)	0.654 ± 0.094	0.954 ± 0.015
Ensemble S1 (IRnPF)	0.665 ± 0.057	0.773 ± 0.024

Table 8.5: Lorenz-63 IRnPF hypothesis test ($N = 32$, $T = 20$ TU, $\sigma_{\text{obs}} = 2.0$, 10 trials). IRnPF models trained on 200 examples with multi-substep supervision; nPF models trained on 4,800 examples with first-substep supervision. Dataset size difference is an intentional consequence of compute constraints.

worse but nESS is the highest of any variant by a substantial margin. The ensemble S1 (IRnPF) result is more sobering: nRMSE 0.665 and nESS 0.773, worse than the nPF ensemble S1 on both metrics. The three additional features (nearest-neighbour distance, weight rank, nESS) did not help at 200 examples; the model is too data-starved to learn from them.

The loss curves confirm underfitting as the root cause. Both IRnPF models plateau at validation loss ≈ 0.250 after 40 epochs. The nPF models, with 4,800 examples and 150 epochs, reach substantially lower validation losses. The RMSE gap between IRnPF and nPF variants is a data-scarcity artifact, not a fundamental incompatibility of IRnPF labels with the architecture. The nESS signal, however, is genuine: even with 200 examples and 40 epochs, the multi-substep supervision on ensemble-coordinated labels produces a controller that maintains dramatically better weight diversity than any comparably-sized alternative.

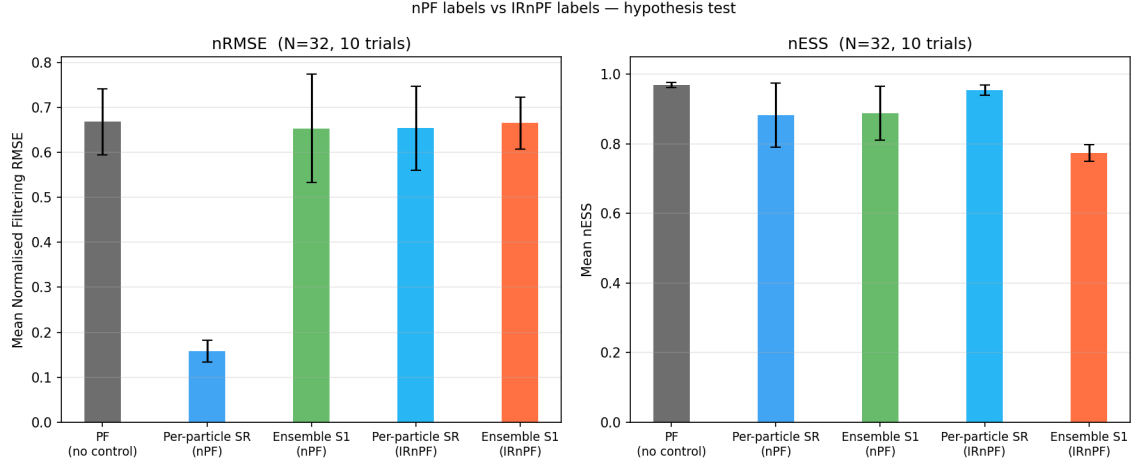


Figure 8.4: nRMSE (left) and nESS (right) for nPF vs IRnPF oracle labels on Lorenz-63 ($N = 32$, 10 trials). Per-particle SR (nPF) dominates on RMSE. Per-particle SR (IRnPF), despite training on $24\times$ fewer examples, achieves the highest nESS of any variant at 0.954.

8.4.3 Results: Lorenz-96

Table 8.6 and Figures 8.7, 8.8, and 8.9 report the Lorenz-96 results.

Method	$\bar{e}$ (nRMSE / σ_{obs})	nESS
PF (no control)	5.179 ± 0.272	0.977 ± 0.010
Per-particle SR (nPF)	2.446 ± 0.448	0.970 ± 0.014
Ensemble S1 (nPF)	3.044 ± 0.344	0.954 ± 0.041
Per-particle SR (IRnPF)	2.688 ± 0.482	0.959 ± 0.037
Ensemble S1 (IRnPF)	3.052 ± 0.520	0.956 ± 0.056

Table 8.6: Lorenz-96 IRnPF hypothesis test ($N = 16$, $T = 10$ TU, $\sigma_{\text{obs}} = 1.0$, 10 trials). nRMSE normalized by σ_{obs} ; values are not directly comparable to Table 7.1. IRnPF models trained on 50 examples with first-substep supervision; nPF models on 400. Evaluation window set to $T = 10$ TU to amortize Colab session cost.

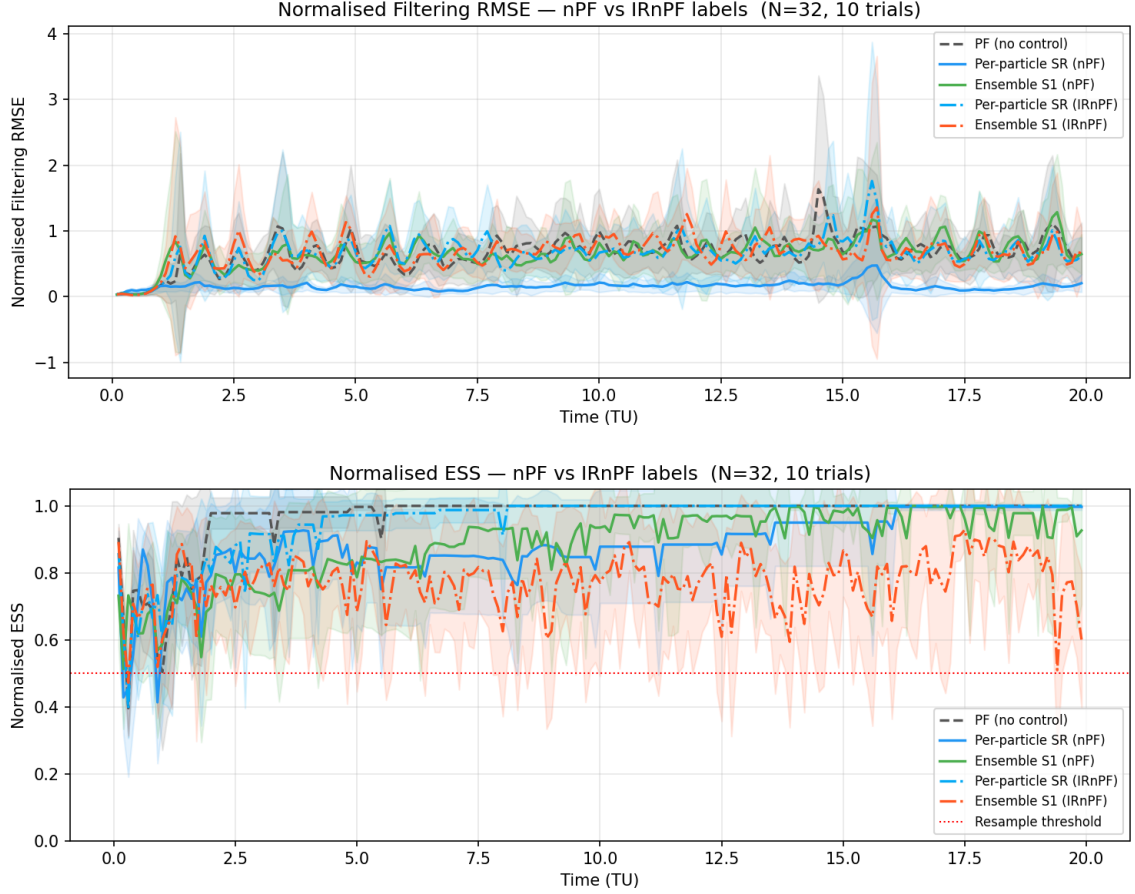


Figure 8.5: Time evolution of nRMSE (top) and nESS (bottom) for nPF vs IRnPF label variants on Lorenz-63. Per-particle SR (nPF) tracks near zero RMSE throughout. IRnPF variants track near the PF baseline on RMSE but the per-particle IRnPF variant sustains markedly higher nESS after the initial transient, staying well above the 0.5 resampling threshold.

On Lorenz-96, the results are inconclusive, mainly due to the reduced nature of the experiment by compute constraints. Per-particle SR (IRnPF) achieves nRMSE 2.688 versus 2.446 for per-particle SR (nPF), a 10% degradation that falls within one standard deviation (± 0.448). Ensemble S1 (IRnPF) at 3.052 is essentially identical to Ensemble S1 (nPF) at 3.044. The L96 IRnPF training used first-substep supervision (not multi-substep) and the standard 35-dimensional encoder (no additional resampling-aware features), so this experiment tests only whether IRnPF labels in the standard training configuration improve over nPF labels at $8\times$ lower data volume. The loss curves show a pronounced train/val gap, confirming severe overfitting to the 50-example training set. No definitive conclusion about the IRnPF oracle’s merit on Lorenz-96 is possible from this experiment.

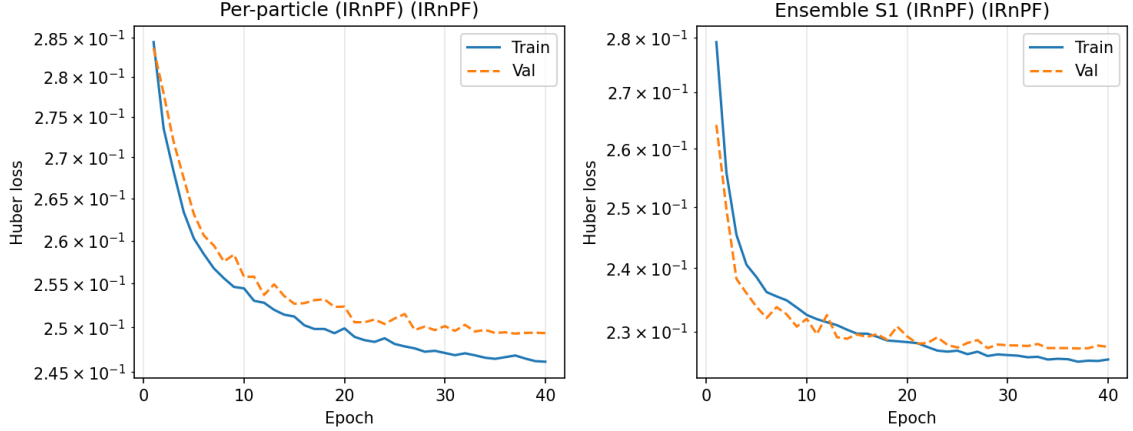


Figure 8.6: Training and validation Huber loss curves for the two IRnPF L63 models (40 epochs, no early stopping). Both converge to a validation plateau around 0.250, consistent with underfitting on the 200-example dataset.

8.4.4 Interpretation

Three conclusions emerge from this hypothesis test. First, multi-substep supervision on IRnPF labels produces qualitatively different learned behavior on Lorenz-63: even at 200 examples, the IRnPF-trained per-particle model achieves nESS 0.954 versus 0.882 for the nPF-trained model, a genuine signal that ensemble-coordinated labels teach controls that preserve weight diversity. Second, the RMSE gap between IRnPF and nPF variants on L63 is a data-scarcity artifact: the 200-example IRnPF models plateau at validation loss ≈ 0.250 after 40 epochs, while the 4,800-example nPF models train to substantially lower losses over 150 epochs. Third, the L96 results are inconclusive due to the 50-example dataset, standard first-substep supervision, and the absence of resampling-aware features, making them an underpowered version of the L63 experiment.

The experiment is best interpreted as a positive signal about the oracle and supervision design space. The hypothesis that ensemble-coordinated multi-substep labels improve ensemble health is supported by the L63 nESS result and not rejected by the L96 RMSE result. Production-scale retraining including 4,800 IRnPF examples with multi-substep supervision and resampling-aware features on L63, and 400 examples with the same on L96 is identified as the natural next step in Chapter 9.

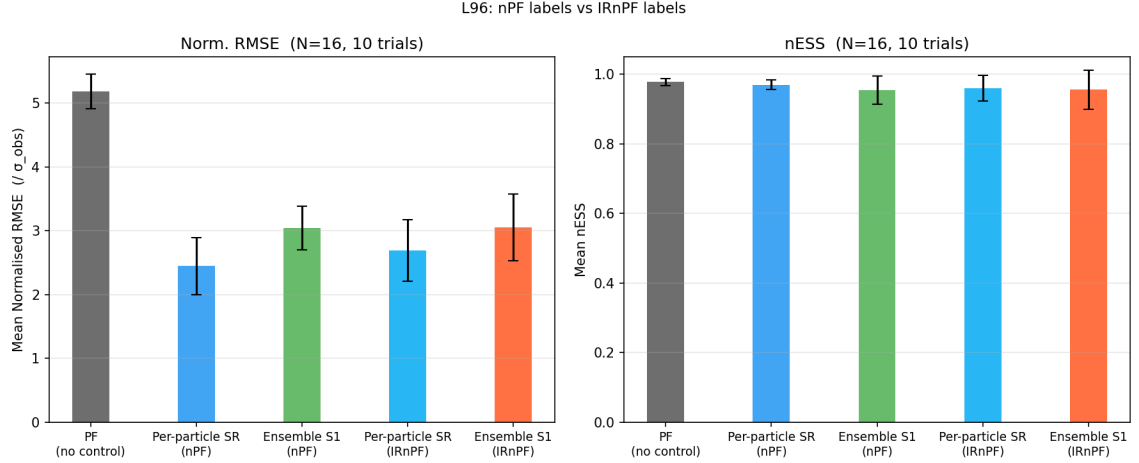


Figure 8.7: nRMSE/ σ_{obs} (left) and nESS (right) for nPF vs IRnPF oracle labels on Lorenz-96 ($N = 16$, $T = 10$ TU, 10 trials). IRnPF and nPF variants are indistinguishable within error bars; per-particle SR (nPF) achieves the lowest nRMSE.

8.5 Summary

Table 8.7 collects the key findings from all four studies in this chapter.

The four studies yield a consistent and practically useful picture. The 77% nRMSE improvement of the per-particle SR controller is attributable to the learned control law, not to deployment-time clipping. The controller generalizes stably across observation noise levels and particle counts spanning a $4\times$ range, without any retraining. The IRnPF experiment, while inconclusive on RMSE due to data constraints, provides a positive signal that ensemble-coordinated oracle labels teach qualitatively different weight-diversity behavior and motivates production-scale follow-up. Together these findings support the practical deployability of the amortized control approach and identify the oracle design space as the most promising direction for future improvement.

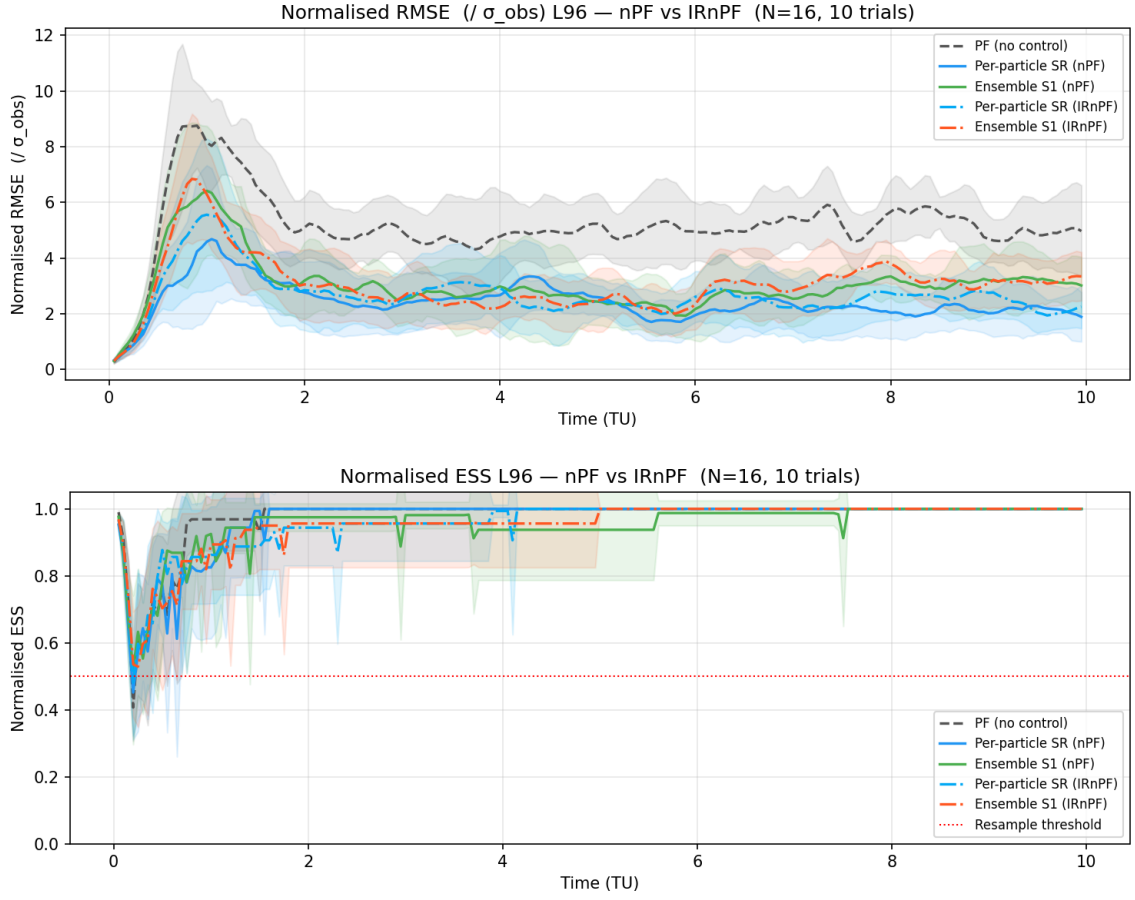


Figure 8.8: Time evolution of $\text{nRMSE}/\sigma_{\text{obs}}$ (top) and nESS (bottom) for nPF vs IRnPF label variants on Lorenz-96. All controlled variants track well below the PF baseline after the initial transient. IRnPF and nPF variants are indistinguishable by $t \approx 3$ TU.

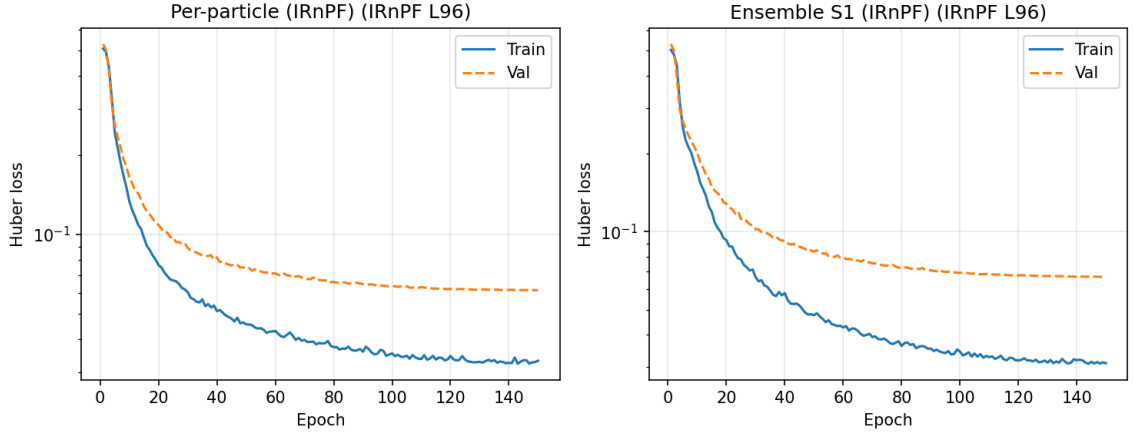


Figure 8.9: Training and validation Huber loss curves for the IRnPF L96 models (log scale, 150 epochs with patience-15 early stopping). The pronounced train/val gap indicates overfitting to the 50-example dataset. Validation loss plateaus at $\approx 6 \times 10^{-2}$ while training continues to $\approx 2 \times 10^{-2}$.

Design decision		Test	Finding	Importance
Control threshold	clipping	Vary $U_{\text{clip}} \in \{25 \dots \infty\}$, checkpoint	Flat nRMSE for $U_{\text{clip}} \geq 50$; degrades only at 25	Low
Noise	generalization	$\sigma_{\text{obs}} = 4.0$ vs 2.0	76% nRMSE reduction at $\sigma = 4.0$; consistent with default result	High
Particle count	generalization	Train $N = 32$, deploy $N \in \{16, 64, 128\}$	nRMSE decreases 0.166 to 0.152; 4 $\times$ advantage preserved at all N	High
IRnPF oracle labels (L63)		200 IRnPF vs 4,800 nPF examples	IRnPF raises nESS to 0.954 but RMSE degrades 4 $\times$; data scarcity (24 $\times$ fewer examples) is the confound, not the labels	Medium
IRnPF oracle labels (L96)		50 IRnPF vs 400 nPF examples	Inconclusive; 10% RMSE degradation within one std; severe overfitting from 50-example training set	Medium

Table 8.7: Summary of all ablation and robustness findings. Reference point is *per_particle* SR at $N = 32$, $\sigma_{\text{obs}} = 2.0$ (nRMSE 0.158, nESS 0.867).

Chapter 9

Conclusion

This thesis designed, implemented, and evaluated an ensemble-aware supervised learning framework for particle nudging control in chaotic data assimilation. The core engineering problem is well-defined: the optimal nudging controller for a particle filter requires thousands of augmented SDE integrations per observation step, making it impractical for real-time deployment. The goal is to amortize that cost offline by learning a neural network that approximates the optimal control in a single forward pass, while preserving the importance-sampling validity that makes particle filtering theoretically sound.

9.1 Summary of Work

Four interconnected components constitute the final design. An offline dataset generation pipeline samples the chaotic attractor with low-discrepancy coverage and runs the Feynman-Kac Monte Carlo oracle to produce labeled control sequences, including the per-particle log Girsanov correction accumulated over each assimilation window. A neural network architecture maps per-particle states and ensemble-level summary statistics to per-particle controls, with permutation equivariance enforced by construction and conditional particle independence preserved by fixing the ensemble summary at the start of each assimilation window. A two-stage training objective combines Huber oracle imitation with a differentiable weight-balance penalty derived from the Girsanov formula. A corrected particle filter deployment pipeline accumulates the Radon-Nikodym correction at every integration substep, restoring importance-sampling validity to all controlled filter variants.

The headline result on Lorenz-63 is that the *per_particle* SR controller achieves nRMSE 0.16 ± 0.03 , a 77% reduction relative to the uncontrolled PF (0.68 ± 0.09) and a $4\times$ improvement over the oracle nPF (0.60 ± 0.10) at $70\times$ lower computational cost. The mechanism is now well-understood: the Feynman-Kac oracle applies aggressive, locally optimal controls that collapse nESS to 0.75, while Huber regression on a heavy-tailed oracle distribution implicitly regularizes the learned controls toward moderate magnitudes that preserve weight diversity (nESS 0.85) while tracking more accurately. The clipping ablation confirms this: varying the deployment-time threshold over $U_{\text{clip}} \in \{50, 100, 200, \infty\}$ produces no change in nRMSE (≈ 0.158 throughout), so the advantage is attributable to the learned control law, not to truncation.

Three ensemble-aware variants were developed to test whether conditioning on collective particle information improves performance. *Ensemble S1* (pooling only), *ensemble S2* (pooling plus weight-balance penalty), and *residual_ensemble* (per-particle base plus trainable ensemble correction) all underperform the per-particle baseline on RMSE at $N = 32$. The pooled context g is too noisy at this ensemble size to contribute reliable collective information, and the ensemble correction pathway introduces training noise that partially corrupts the strong per-particle base. *Ensemble S2* achieves the highest nESS of any variant (0.90) at a modest RMSE cost (0.69), confirming that the differentiable Girsanov-based weight-balance penalty is a meaningful training signal. The central lesson is that ensemble-awareness must enter through the oracle and training data, not through the architecture alone.

The pipeline scales from Lorenz-63 ($d = 3$) to Lorenz-96 ($K = 8, d = 8$) with no structural changes. Per-particle SR achieves nRMSE 0.660 ± 0.104 , a 44% reduction over the uncontrolled PF and statistical parity with the oracle (0.623 ± 0.117), at a $27\times$ lower cost (120 ms vs 3,253 ms per step). The smaller oracle gap on L96 compared to L63 is explained by the quasi-linear advection structure of L96 at $K = 8$: the oracle’s controls are more reliably aligned with the actual trajectory, its nESS degradation is less severe, and SR’s implicit regularization advantage correspondingly shrinks. This is consistent with the broader observation that Lorenz-96 is structurally simpler than it appears [59].

Robustness studies confirm that the per-particle SR controller generalizes across observation noise levels ($\sigma_{\text{obs}} \in \{2.0, 4.0\}$) and particle counts ($N \in \{16, 32, 64, 128\}$) without retraining, maintaining its $4\times$ nRMSE advantage at every tested configuration. The IRnPF hypothesis test provides a complementary signal: training on ensemble-coordinated IRnPF labels raises nESS to 0.954 on Lorenz-63, the highest of any variant, but the RMSE result is confounded by a $24\times$ data-size disparity. The

experiment does not reject the IRnPF hypothesis; it establishes that the oracle design space is worth pursuing at production scale.

9.2 Next Steps & Future Research

Four directions emerge directly from the results, ordered by proximity to the current implementation.

Production-scale IRnPF retraining. The highest-priority next step is to retrain both the per-particle and ensemble-aware architectures on a full-scale IRnPF dataset: 4,800 examples for Lorenz-63 and 400 for Lorenz-96, matching the nPF dataset sizes. The brief hypothesis test in Chapter 8 showed that IRnPF labels teach controls with higher nESS (0.954 vs 0.882) but that the RMSE gap is driven by the $24\times$ data-size disparity, not by a fundamental incompatibility of ensemble-coordinated labels with the architecture. Production retraining would isolate the label effect cleanly and, if successful, would confirm that ensemble-awareness must enter through the training data and that the per-particle architecture is already the right vehicle for that information.

Scaling to Lorenz-96 at $K = 40$. The present L96 experiments use $K = 8$ as a proof-of-concept. The standard benchmark in the data assimilation literature is $K = 40$, where the augmented system per rollout grows to 1,640 dimensions and the attractor lives in a genuinely high-dimensional space. At $K = 40$ the quasi-linear simplicity identified in smaller L96 configurations [59] is expected to diminish, the oracle’s Monte Carlo estimate becomes noisier, and the implicit regularization advantage of the imitation learning approach is expected to be far more pronounced. Generating a $K = 40$ dataset would require Della HPC resources (approximately 10 to 20 hours of Slurm array jobs) and would provide the first test of whether the pipeline generalizes to genuinely high-dimensional chaotic filtering.

Localized and symmetry-equivariant architectures. The current architecture applies a full d -dimensional control to each particle, making no use of the spatial locality of the Lorenz-96 coupling structure. The drift $\dot{x}_k$ depends only on $x_{k-2}, x_{k-1}, x_k, x_{k+1}$, and the translational symmetry under cyclic permutation means each attractor state implicitly represents K symmetry-related states. A symmetry-equivariant encoder sharing weights across spatial positions would have K times fewer

unique parameters and generalize more efficiently from limited datasets. At $K = 40$ with a $3 \times 40 + 3 = 123$ -dimensional encoder input, this structural constraint becomes practically necessary rather than merely desirable.

Particle count robustness at training time. The robustness study shows that *ensemble_s1* degrades modestly at $N = 128$ due to out-of-distribution pooling statistics: the max-pool embedding at $N = 128$ is drawn from a more extreme distribution than seen during training at $N = 32$. Training on datasets with mixed N , sampled from a distribution at data-generation time, would calibrate the pooling module across a range of ensemble sizes and is the most direct fix for ensemble-aware variants intended for deployment at N substantially different from the training count.

9.3 Closing Remarks

The central finding of this thesis is both simple and counterintuitive: a learned controller that deliberately undershoots the optimal control outperforms that optimal control on the downstream filtering metric. The reason is that particle filtering is a distributional problem, not a pointwise tracking problem. A control that is locally optimal for a small Monte Carlo ensemble may be globally suboptimal for the particle filter because it destroys the weight diversity that makes the ensemble a valid approximation of the posterior. The Girsanov correction is the mechanism that makes this tradeoff explicit: any control applied to the particles must be accounted for in the importance weights, and aggressive controls create large Girsanov penalties that collapse the ESS.

The supervised regression approach, trained with a robust loss on oracle labels, learns to navigate this tradeoff implicitly. The model does not minimize ESS collapse or maximize nESS directly; it learns to imitate the oracle. Since the oracle distribution is heavy-tailed and Huber regression penalizes large residuals only linearly, the learned controller regresses toward moderate magnitudes that happen to preserve weight diversity. This implicit regularization bridges the offline training objective and the online filtering objective, and the ablation studies confirm it is robust across clipping thresholds, noise levels, and particle counts. The IRnPF experiment adds a further dimension: ensemble-coordinated labels can shift learned behavior toward even higher nESS, suggesting that the oracle design space itself is a lever worth pulling alongside architectural choices.

The broader implication is that importance-sampling correctness, not model complexity or oracle quality, is the binding constraint on learned particle filter controllers. A structurally simple controller, given proper training labels and a robust loss, can exceed the performance of the oracle it imitates. The path to better filtering lies in careful attention to the statistical structure of the importance-sampling correction that makes the filter valid, and in designing oracles whose labels encode the ensemble-level behavior the filter ultimately needs.

Honor Pledge. This paper represents my own work in accordance with University regulations.

/s/ Sonal Bhatia

Bibliography

- [1] ABET. Criteria for accrediting engineering programs, 2025-2026. <https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-engineering-programs-2025-2026/>, 2025. Accessed: 2026-01-30.
- [2] Melanie Ades and Peter Jan van Leeuwen. An exploration of the equivalent weights particle filter. *Quarterly Journal of the Royal Meteorological Society*, 139(672):820–840, 2013.
- [3] Jeffrey L. Anderson. An ensemble adjustment Kalman filter for data assimilation. *Monthly Weather Review*, 129(12):2884–2903, 2001.
- [4] Alan Bain and Dan Crisan. *Fundamentals of Stochastic Filtering*. Springer, New York, 2009.
- [5] Ryne Beeson. pybeeda, 2023. Available at <https://bitbucket.org/beeson/pybeeda/src/master/>. Reach out to Professor Ryne Beeson at ryne@princeton.edu for access.
- [6] Ryne Beeson. pybeebase, 2024. Available at <https://bitbucket.org/beeson/pybeebase/src/main/>. Reach out to Professor Ryne Beeson at ryne@princeton.edu for access.
- [7] Ryne Beeson and Uwe D. Hanebeck. Adaptive leader-follower control for the intermediate resampling nudged particle filter. Manuscript submitted for publication, 2025.
- [8] Ryne Beeson and N. Sri Namachchivaya. Particle filtering for chaotic dynamical systems using future right-singular vectors. *Nonlinear Dynamics*, 102:679–696, 2020.

- [9] Thomas Bengtsson, Peter Bickel, and Bo Li. Curse-of-dimensionality revisited: Collapse of the particle filter in very large scale systems. In Deborah Nolan and Terry Speed, editors, *Probability and Statistics: Essays in Honor of David A. Freedman*, pages 316–334. Institute of Mathematical Statistics, 2008.
- [10] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*. Athena Scientific, 2017.
- [11] Alexandros Beskos, Dan Crisan, and Ajay Jasra. On the stability of sequential Monte Carlo methods in high dimensions. *The Annals of Applied Probability*, 24(4):1396–1445, 2014.
- [12] Sonal Bhatia. Learning particle-wise control laws for chaotic data assimilation. Junior Independent Work, Department of Mechanical and Aerospace Engineering, Princeton University, 2025.
- [13] N. Botteghi, F. Califano, M. Poel, and C. Brune. Trajectory generation, control, and safety with denoising diffusion probabilistic models. *arXiv preprint arXiv:2306.15512*, 2023.
- [14] John-Joseph Brady, Yuhui Luo, Wenwu Wang, Víctor Elvira, and Yunpeng Li. Regime learning for differentiable particle filters. In *2024 IEEE 13th Sensor Array and Multichannel Signal Processing Workshop (SAM)*, pages 1–5. IEEE, 2024.
- [15] Steven L. Brunton and J. Nathan Kutz. Data-driven science and engineering: Machine learning, dynamical systems, and control. *Cambridge University Press*, 2019.
- [16] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems (NeurIPS)*, 31, 2018.
- [17] Alexandre J. Chorin and Xuemin Tu. Implicit sampling for particle filters. *Proceedings of the National Academy of Sciences*, 106(41):17249–17254, 2009.
- [18] Arnaud Doucet, Nando de Freitas, and Neil Gordon. *Sequential Monte Carlo Methods in Practice*. Springer, 2001.
- [19] Lawrence C. Evans. *Partial Differential Equations*. American Mathematical Society, 2010.

- [20] Geir Evensen. Sequential data assimilation with a nonlinear quasi-geostrophic model using Monte Carlo methods to forecast error statistics. *Journal of Geophysical Research: Oceans*, 99(C5):10143–10162, 1994.
- [21] Geir Evensen, Femke C. Vossepoel, and Peter Jan van Leeuwen. *Data Assimilation Fundamentals: A Unified Formulation of the State and Parameter Estimation Problem*. Springer, Cham, 2022.
- [22] Wendell H. Fleming. Exit probabilities and optimal stochastic control. volume 4, pages 329–346. 1978.
- [23] Wendell H. Fleming. Logarithmic transformations and stochastic control. In William H. Fleming and Luis Gorostiza, editors, *Advances in Filtering and Optimal Stochastic Control*, volume 42 of *Lecture Notes in Control and Information Sciences*, pages 131–141. Springer, Berlin, Heidelberg, 1982.
- [24] D. Gadginmath and F. Pasqualetti. Dynamics-aware diffusion models for planning and control. *arXiv preprint arXiv:2504.00236*, 2025.
- [25] S. Gilpin and M. Herty. A modified particle filter that reduces weight collapse. *arXiv preprint arXiv:2510.23740*, 2025.
- [26] Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. Novel approach to nonlinear/Non-Gaussian Bayesian state estimation. *IEE Proceedings F - Radar and Signal Processing*, 140(2):107–113, 1993.
- [27] Thomas M. Hamill, Jeffrey S. Whitaker, and Chris Snyder. Distance-dependent filtering of background error covariance estimates in an ensemble Kalman filter. *Monthly Weather Review*, 129(11):2776–2790, 2001.
- [28] Uwe D. Hanebeck and Volker Klumpp. Localized cumulative distributions and a multivariate generalization of the Cramér-von Mises distance. In *2008 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*, pages 33–39, 2008.
- [29] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009.
- [30] Andrew H. Jazwinski. *Stochastic Processes and Filtering Theory*. Academic Press, New York, 1970.

- [31] Simon J. Julier and Jeffrey K. Uhlmann. A new extension of the Kalman filter to nonlinear systems. In *Proceedings of AeroSense: The 11th International Symposium on Aerospace/Defense Sensing, Simulation, and Controls*, volume 3068, pages 182–193, 1997.
- [32] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [33] Nikolas Kantas, Arnaud Doucet, Sumeetpal S. Singh, Jan Maciejowski, and Nicolas Chopin. On particle methods for parameter estimation in state-space models. *Statistical Science*, 30(3):328–351, 2015.
- [34] Hilbert J. Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of Statistical Mechanics: Theory and Experiment*, 2005(11):P11011, 2005.
- [35] Theofania Karampela. Optimal proposals from collaborative optimal control: Application to continuous-discrete time particle filtering of chaotic dynamical systems. Master’s thesis, ETH Zurich and Princeton University, 2025.
- [36] Ioannis Karatzas and Steven E. Shreve. *Brownian Motion and Stochastic Calculus*. Springer, New York, 2nd edition, 1991.
- [37] A. Karimi and Mark R. Paul. Extensive chaos in the lorenz-96 model. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 20(4):043105, November 2010.
- [38] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
- [39] Peter E. Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Springer, 1992.
- [40] Harold J. Kushner. On the differential equations satisfied by conditional probability densities of Markov processes. *SIAM Journal on Control*, 2(1):106–119, 1964.
- [41] Kody Law, Andrew Stuart, and Konstantinos Zygalakis. *Data Assimilation: A Mathematical Introduction*. Springer, Cham, 2015.
- [42] Tuan Anh Le, Maximilian Igl, Tom Rainforth, Tom Jin, and Frank Wood. Auto-encoding sequential Monte Carlo. In *International Conference on Learning Representations (ICLR)*, 2018.

- [43] Anjian Li, Amlan Sinha, and Ryne Beeson. Amortized global search for efficient preliminary trajectory design with deep generative models. *arXiv preprint arXiv:2308.03960*, 2023.
- [44] N. Lingala, N. Sri Namachchivaya, Nicolas Perkowski, and Hoong C. Yeong. Particle filtering in high-dimensional chaotic systems. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 22(4):047509, 2012.
- [45] N. Lingala, Nicolas Perkowski, Hoong C. Yeong, N. Sri Namachchivaya, and Z. Rapti. Optimal nudging in particle filters. *Probabilistic Engineering Mechanics*, 37:160–169, 2014.
- [46] Edward N. Lorenz. Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2):130–141, 1963.
- [47] Edward N. Lorenz. Predictability: A problem partly solved. In *Seminar on Predictability*, Shinfield Park, Reading, 1995. ECMWF.
- [48] Chris J. Maddison, Dieterich Lawson, George Tucker, Nicolas Heess, Mohammad Norouzi, Andriy Mnih, Arnaud Doucet, and Yee Whye Teh. Filtering variational objectives. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [49] Luigi Martinelli. Mechanical and aerospace engineering independent work guide, 2025-26, 2025. Prepared August 11, 2025.
- [50] Richard N. Miller, Michael Ghil, and Francois Gauthiez. Advanced data assimilation in strongly nonlinear dynamical systems. *Journal of the Atmospheric Sciences*, 51(8):1037–1056, 1994.
- [51] Pierre Del Moral. *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, New York, 2004.
- [52] Christian A. Naesseth, Scott W. Linderman, Rajesh Ranganath, and David M. Blei. Variational sequential Monte Carlo. In *Proceedings of the 21st International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 84 of *Proceedings of Machine Learning Research*, pages 968–977, 2018.
- [53] Edward Ott. *Chaos in Dynamical Systems*. Cambridge University Press, 2002.

- [54] Edward Ott, Brian R. Hunt, Istvan Szunyogh, Aleksey V. Zimin, Eric J. Kostelich, Matteo Corazza, Eugenia Kalnay, D. J. Patil, and James A. Yorke. A local ensemble Kalman filter for atmospheric data assimilation. *Tellus A*, 56(5):415–428, 2004.
- [55] Michael K. Pitt and Neil Shephard. Filtering via simulation: Auxiliary particle filters. *Journal of the American Statistical Association*, 94(446):590–599, 1999.
- [56] Jonathan Poterjoy. A localized particle filter for high-dimensional nonlinear systems. *Monthly Weather Review*, 144(1):59–76, 2016.
- [57] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [58] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- [59] Stephan Rasp. Lorenz ’96 Is Too Easy! Machine Learning Research Needs a More Realistic Toy Model. <https://raspstephan.github.io/blog/lorenz-96-is-too-easy/>, 2019.
- [60] Sebastian Reich. A nonparametric ensemble transform method for Bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.
- [61] Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *JMLR Workshop and Conference Proceedings*, pages 627–635, 2011.
- [62] Clarence W. Rowley. *Introduction to Feedback Control*. Princeton University, 2024. Course Notes for MAE 433.
- [63] Chris Snyder, Thomas Bengtsson, Peter Bickel, and Jeff Anderson. Obstacles to high-dimensional particle filtering. *Monthly Weather Review*, 136(12):4629–4640, 2008.
- [64] Chris Snyder, Thomas Bengtsson, and Matthias Morzfeld. Performance bounds for particle filters using the optimal proposal. *Monthly Weather Review*, 143(11):4750–4761, 2015.

- [65] Alessio Spantini, Ricardo Baptista, and Youssef Marzouk. Coupling techniques for nonlinear ensemble filtering. *SIAM Review*, 64(4):921–953, 2022.
- [66] Steven H. Strogatz. *Nonlinear Dynamics and Chaos*. Westview Press, 2nd edition, 2018.
- [67] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [68] P. J. van Leeuwen, Hans R. Künsch, Lars Nerger, Roland Potthast, and Sebastian Reich. Particle filters for high-dimensional geoscience applications: A review. *Quarterly Journal of the Royal Meteorological Society*, 145(723):2335–2365, 2019.
- [69] Peter Jan van Leeuwen. Particle filtering in geophysical systems. *Monthly Weather Review*, 137(12):4089–4114, 2009.
- [70] Peter Jan van Leeuwen. Nonlinear data assimilation in geosciences: An extremely efficient particle filter. *Quarterly Journal of the Royal Meteorological Society*, 136(653):1991–1999, 2010.
- [71] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- [72] H. C. Yeong, R. T. Beeson, N. S. Namachchivaya, and N. Perkowski. Particle filters with nudging in multiscale chaotic systems: With application to the lorenz ’96 atmospheric model. *Journal of Nonlinear Science*, 30(4):1519–1552, 2020.
- [73] Manzil Zaheer, Saajan Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [74] Moshe Zakai. On the optimal filtering of diffusion processes. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 11(3):230–243, 1969.

Appendix A

Definitions & Background

Concepts

This appendix provides informal definitions of key terms used throughout the thesis. It is intended for readers with a background in applied mathematics or engineering who have not previously encountered stochastic calculus, Bayesian inference, or particle filtering. The goal is not rigor but accessibility: each definition explains what the concept is, why it is useful, and how it connects to the rest of the thesis. Formal treatments of all topics appear in Chapter 3 and the references cited there; textbooks referred to for these definitions include [30, 4, 41, 21] for filtering theory and [36, 39] for stochastic calculus.

A.1 State Estimation & Data Assimilation

State. The *state* of a dynamical system is the complete description of the system at a given instant: everything needed to determine its future evolution given the governing equations. For a weather model, the state might be the temperature, pressure, and wind velocity at every grid point. For Lorenz-63 [46], the state is the three-dimensional vector (x_1, x_2, x_3) . For Lorenz-96 [47, 37] with K variables, the state is $x \in \mathbb{R}^K$. In this thesis, we never observe the state directly or perfectly; we only see noisy measurements of it at discrete times.

Signal. In the filtering literature, the true evolving state is called the *signal* [4, 30]. The term distinguishes the hidden quantity we care about from the noisy observations

we make of it. In this thesis, the signal follows a stochastic differential equation driven by Brownian motion [36], representing both the deterministic physics of the system and genuine model uncertainty.

Filtering distribution. Given a sequence of noisy observations $Y_1, Y_2, \dots, Y_k$ up to time t_k , the *filtering distribution* $p(x_{t_k} | Y_{1:k})$ is the probability distribution of the true state conditioned on all available observations [30, 4]. It represents the best probabilistic knowledge of where the system currently is, after incorporating all data up to the present moment. The goal of data assimilation is to compute or approximate this distribution sequentially as new observations arrive [41, 21].

Prior and posterior. In the Bayesian framework, the *prior* $p(x_{t_k} | Y_{1:k-1})$ is the predicted state distribution before incorporating the new observation Y_k . The *posterior* $p(x_{t_k} | Y_{1:k})$ is the updated distribution after applying Bayes' rule:

$$p(x_{t_k} | Y_{1:k}) \propto p(Y_k | x_{t_k}) p(x_{t_k} | Y_{1:k-1}).$$

The likelihood $p(Y_k | x_{t_k})$ encodes how probable the observed measurement Y_k is given a hypothetical true state x_{t_k} . The prior encodes what we knew before seeing the new data; the posterior encodes what we know after. In nonlinear chaotic systems like Lorenz-63, the posterior can be multimodal and highly non-Gaussian [50, 70], which is why particle filters [26, 18] rather than Kalman filters [32] are needed.

Data assimilation. *Data assimilation* (DA) is the problem of sequentially computing and updating the filtering distribution as new observations arrive [41, 21]. It originated in geoscience for combining weather model forecasts with observational data [20, 68], but the same framework applies to any system where a computational model and real measurements are available. This thesis is specifically concerned with DA in chaotic nonlinear systems where standard Gaussian approximations like the Kalman filter [32] break down [50, 63].

A.2 Chaos and Lyapunov Exponents

Chaos. A dynamical system is *chaotic* if nearby initial conditions diverge exponentially over time [66, 53]. This *sensitive dependence on initial conditions* means that even a tiny error in the initial state estimate grows rapidly, placing a fundamental

limit on how far ahead accurate predictions can be made. In a chaotic system, the particle filter faces a race: can the ensemble track the true state faster than errors grow? Nudging [72, 8] is the strategy of giving the ensemble a head start by guiding it toward the observation.

Lyapunov exponents. The *maximum Lyapunov exponent* λ_1 is the average rate of exponential divergence between nearby trajectories [66, 53]. Two trajectories starting a distance δ_0 apart grow to a separation of approximately $\delta_0 e^{\lambda_1 t}$ after time t . If $\lambda_1 > 0$, the system is chaotic. Lorenz-63 [46] has $\lambda_1 \approx 0.91$, giving an error doubling time of $\tau_d = \ln 2 / \lambda_1 \approx 0.76$ TU. Lorenz-96 at $K = 8$, $F = 8$ [47, 37] has approximately 3 to 4 positive Lyapunov exponents and $\tau_d \approx 1.7$ TU.

Strange attractor. Chaotic systems often evolve toward a *strange attractor*: a geometric structure in state space that all trajectories approach over time, regardless of where they start [46, 53]. Lorenz-63 has the distinctive butterfly-shaped attractor with two lobes; the system switches irregularly between them. Particles in a particle filter, if not nudged, will spread across this attractor between observations. The switching is the hardest scenario for filtering because a particle on the wrong lobe receives near-zero likelihood weight and cannot easily be corrected by post-hoc reweighting alone [50, 64].

Error doubling time. The *error doubling time* τ_d is the time it takes for a small perturbation to double in magnitude. It provides a natural timescale for the filtering problem. Observation intervals much longer than τ_d lead to severe weight degeneracy: the particle cloud has spread far across the attractor before the next update [63, 64]. In the Lorenz-63 experiments, $\Delta T = 0.1$ TU is about 1/7 of τ_d , which is moderately sparse. Increasing ΔT toward or beyond τ_d would dramatically worsen degeneracy.

A.3 Particle Filters

Particle (ensemble member). A *particle* or *ensemble member* is one simulated realization of the system state [26, 18]. A particle filter maintains N particles simultaneously, treating them collectively as a sample from the filtering distribution [51, 4]. Each particle represents one hypothesis about where the true state might be. More particles provide a richer representation of the posterior, but at linear computational cost.

Importance weights. Because particles are propagated under the model dynamics (or a controlled approximation thereof) rather than sampled directly from the posterior, each particle $x^{(i)}$ is assigned a weight $w^{(i)} \geq 0$ (with $\sum_i w^{(i)} = 1$) that reflects how consistent that particle’s state is with the observations relative to the others [26, 18]. A particle that closely agrees with the data receives a large weight; one that disagrees receives a small weight. The posterior is approximated as: $p(x \mid Y_{1:k}) \approx \sum_i w^{(i)} \delta_{x^{(i)}}(x)$ [51].

Importance sampling. *Importance sampling* is the statistical principle underlying particle filtering [18, 51]. If we want to compute an expectation under a target distribution p but can only sample from a different proposal distribution q , we can correct by weighting each sample $x^{(i)} \sim q$ by the ratio $p(x^{(i)})/q(x^{(i)})$. The particle weights play exactly this role. This is why the Girsanov correction [36, 72] is so important: when we add nudging, we change the proposal distribution from the uncontrolled model dynamics to the controlled dynamics, and the Girsanov factor is the ratio that corrects for this change. Without it, the importance weights are incorrect and the filter is not a valid Bayesian estimator.

Effective sample size. The *effective sample size* (ESS) measures how many equally-weighted particles would give the same statistical efficiency as the current weighted ensemble [18, 33]:

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N (w^{(i)})^2}.$$

When all weights are equal, $N_{\text{eff}} = N$. When one particle has all the weight, $N_{\text{eff}} = 1$. The *normalized ESS* is $\text{nESS} = N_{\text{eff}}/N \in (0, 1]$. An nESS near 1 means the filter is effectively using all its particles. An nESS near $1/N$ means the filter is wasting almost all its computational budget on particles that contribute negligible information. This is why nESS is tracked alongside RMSE: a filter that achieves low RMSE by collapsing its nESS to near zero is not a robust or reliable estimator [25].

Particle degeneracy. *Particle degeneracy* occurs when the ESS collapses to near 1: essentially all the weight concentrates on one or a very few particles [63, 9, 64]. The particle filter then fails to represent the spread of the filtering distribution. In chaotic systems with sparse observations, this is the primary failure mode and the problem this thesis addresses. The key insight is that degeneracy is caused by a mismatch between the proposal distribution (where the particles actually are) and the likelihood-weighted posterior (where they should be) [70, 68]. Nudging reduces

this mismatch by actively steering particles toward observation-consistent regions [72, 8].

Resampling. *Resampling* is the procedure by which particles with very small weights are discarded and high-weight particles are duplicated, so that all particles receive approximately equal weight [26, 18]. This prevents the particle ensemble from collapsing to a single point. The downside is reduced diversity: after resampling, many particles are copies of the same high-weight state and only diverge through subsequent process noise. This thesis uses *systematic resampling* throughout, triggered when $\text{nESS} < 0.5$.

A.4 Stochastic Differential Equations

SDE. A *stochastic differential equation* (SDE) describes the continuous-time evolution of a random process [36, 39]. The general form used in this thesis is:

$$dx_t = b(x_t) dt + \sigma dW_t,$$

where $b(x_t) dt$ is the deterministic *drift* (the average direction of motion, determined by the system’s governing equations) and σdW_t is the stochastic *diffusion* (random perturbations driven by Brownian motion, scaled by the matrix σ). A particle in the filter is a single realization of such a process.

Brownian motion. *Brownian motion* W_t is the canonical model for continuous-time random noise [36]. Its increments $W_{t+\Delta} - W_t$ are independent, Gaussian, with mean zero and variance Δ (proportional to the time step). It models the cumulative effect of many tiny random perturbations and is the continuous-time limit of a random walk. In physical terms, Brownian motion in the signal SDE represents the unresolved small-scale processes and model uncertainty that prevent perfect deterministic prediction.

Itô’s lemma. *Itô’s lemma* is the chain rule of stochastic calculus [36, 39]. For a smooth function $V(t, x)$ and a process x_t satisfying an SDE:

$$dV(t, x_t) = \frac{\partial V}{\partial t} dt + \nabla_x V \cdot dx_t + \frac{1}{2} \text{tr}(\sigma \sigma^\top \nabla_x^2 V) dt.$$

The extra term $\frac{1}{2} \text{tr}(\sigma \sigma^\top \nabla_x^2 V)$ arises because Brownian increments have variance dt rather than $(dt)^2$. This lemma is used in Section 3.4.3 to establish the zero-variance property of the optimal controller [22, 23].

Euler-Maruyama. The *Euler-Maruyama* method is the simplest numerical integration scheme for SDEs, analogous to forward Euler for ODEs [39]:

$$x_{t+\delta t} = x_t + b(x_t) \delta t + \sigma \sqrt{\delta t} \xi, \quad \xi \sim \mathcal{N}(0, I).$$

The actual integration in this thesis uses a Wiener-RK4-Maruyama scheme, which applies fourth-order Runge-Kutta to the deterministic drift and Euler-Maruyama to the diffusion [39]. This provides better accuracy for the chaotic drift at the same step size.

A.5 Particle Nudging & Imitation Learning

Nudging. *Nudging* in the particle filter context refers to adding a deterministic control term $u_t^{(i)}$ to each particle’s SDE [72, 8, 44, 45]:

$$dx_t^{(i)} = f(x_t^{(i)}) dt + u_t^{(i)} dt + \Sigma^{1/2} dW_t^{(i)}.$$

The control steers the particle toward regions of state space consistent with the upcoming observation. The word nudging reflects the fact that the control should be gentle enough not to overwhelm the natural dynamics. A particle that ignores the physics and simply races toward the observation does not represent a physically meaningful sample from any distribution related to the model [70, 2].

Expert policy. In the imitation learning framework [61, 67], the *expert policy* (or oracle) is the entity whose behavior is being imitated. In this thesis, the expert is the Feynman-Kac Monte Carlo optimal nudged particle filter [23, 34]: given a particle state $x^{(i)}$, an upcoming observation y , and a time horizon Δt , it computes the theoretically optimal control u^* using n_s uncontrolled Monte Carlo rollouts. The expert is expensive to query (30,518 ms per observation step on Lorenz-63) but produces high-quality demonstrations. It is queried offline to generate a large training dataset.

Behavioral cloning. *Behavioral cloning* is the simplest form of imitation learning [61]. The agent (neural network) observes the expert’s input-output pairs $(x^{(i)}, y, \Delta t, \sigma_{\text{obs}}) \mapsto u^*$ and learns to replicate them via supervised regression [29]. The key advantage is that no environment interaction is needed at training time: the expert is queried offline, and the neural network is trained on a static dataset. The limitation is that the neural network may encounter situations at deployment that are not well-represented in the training dataset, leading to compounding errors [61]. In this thesis, these distribution shift issues are mitigated by training across a diverse set of attractor states sampled with low-discrepancy coverage.

Amortized inference. *Amortized inference* is the strategy of computing an expensive quantity offline on a representative dataset and training a neural network to approximate it, so that online deployment requires only a cheap forward pass [43, 52, 42]. This is precisely what this thesis does: the oracle control computation (thousands of SDE integrations per step) is amortized offline, and the neural network serves as the amortized approximation. The word amortized reflects the fact that the upfront cost of generating training data and training the network is spread across many future filter deployments.

A.6 The Girsanov Correction

Change of measure. When nudging is added, we change the probability distribution over particle trajectories from what the model predicts to a distribution that is biased toward the observation. A trajectory that was nudged strongly toward the observation would be very unlikely under the uncontrolled model. The *Radon-Nikodym derivative* $d\mathbb{P}/d\mathbb{Q}$ is the ratio of probabilities: it tells us how much more or less likely a given trajectory is under the uncontrolled model $\mathbb{P}$ than under the controlled model $\mathbb{Q}$ [36].

Girsanov’s theorem. *Girsanov’s theorem* provides the formula for the Radon-Nikodym derivative when the drift of an SDE is changed [36]. When the drift changes from $b(x)$ to $b(x) + u$, the log-Radon-Nikodym derivative over $[t_k, t_{k+1}]$ is:

$$\log \frac{d\mathbb{P}}{d\mathbb{Q}} = - \int_{t_k}^{t_{k+1}} (\Sigma^{-1/2} u_s)^\top dW_s - \frac{1}{2} \int_{t_k}^{t_{k+1}} \|\Sigma^{-1/2} u_s\|^2 ds.$$

The first term is a stochastic integral capturing random fluctuations along the controlled path. The second term accumulates the squared control magnitude, penalizing total control effort [22, 23]. A large control produces a large exponent, which means a small Radon-Nikodym factor, which means the trajectory was very unlikely under the uncontrolled model. This is the quantity that corrects for the bias introduced by nudging [72, 8].

Why omitting the correction causes failure. Without the Girsanov correction, the importance weights are computed using only the observation likelihood. A particle that was aggressively nudged toward the observation will score very highly on the likelihood, because it ends up very close to $Y_{t_{k+1}}$. But this high score is not legitimate: the particle was artificially placed there by a strong control, not because it was an independently plausible trajectory under the model. Without the correction, this particle receives an inflated weight, causing the filter to become overconfident and eventually collapse [18, 51]. This is why omitting the Girsanov correction degrades filtering performance even when the controller itself is accurate: the problem is not the control quality, it is the weight validity [72].

A.7 Hamilton-Jacobi-Bellman Equations

Value function. For an optimal control problem, the *value function* $V(t, x)$ gives the minimum achievable expected cost when starting from state x at time t and applying the best possible control from t onward to the end of the window [10, 40]. Once V is known, the optimal control is $u^*(t, x) = -Q\nabla_x V(t, x)$: push in the direction of steepest descent of the value function, scaled by the process noise covariance Q .

HJB equation. The *Hamilton-Jacobi-Bellman* (HJB) equation is the PDE whose solution is the value function [10, 19]. It is derived by requiring that the value function be consistent with the dynamic programming principle: the cost of the optimal policy from state x at time t must equal the cost of taking one step and then acting optimally thereafter. For the nudging cost functional in this thesis, the HJB equation has a nonlinear quadratic gradient term that makes it generally intractable [19].

Log transformation and linearity. The key insight that makes the nudging problem tractable is the Cole-Hopf substitution $V(t, x) = -\log \Phi(t, x)$ [23, 34]. Under this transformation, the quadratic gradient terms in the HJB equation cancel, and Φ

satisfies a linear PDE. Linear PDEs are vastly easier to handle: in particular, their solutions can be represented as expectations over random processes (the Feynman-Kac formula), enabling Monte Carlo approximation [22, 34].

Feynman-Kac formula. The *Feynman-Kac formula* is a classical result that connects linear PDEs to stochastic processes [36, 51]. For the linearized nudging problem, it says that $\Phi(t, x)$ is the expected terminal observation likelihood over all uncontrolled trajectories starting from x at time t [34, 23]. In plain terms: to compute the optimal control at a given particle position, run a bunch of uncontrolled trajectories forward from that position to the next observation time, compute the observation likelihood for each, and weight-average their gradient directions. Paths that naturally arrive near the observation get high weight; paths that arrive far away get low weight. The optimal nudge points in the direction that the high-weight paths are heading. This is exactly what the Monte Carlo oracle in this thesis computes [72, 8].

A.8 Ensemble-Level Concepts

Permutation equivariance. A function mapping an ensemble to per-particle outputs is *permutation equivariant* if permuting the particle labels permutes the outputs by the same permutation [73]. The learned controller must be permutation equivariant: since particle labels are arbitrary and carry no physical meaning, swapping particles 1 and 2 should produce exactly the same controls (just in swapped order) rather than different controls. The architecture in this thesis enforces this by applying the encoder identically to all particles and using only permutation-invariant pooling operations (weighted mean and elementwise maximum) [73].

Permutation invariance. A function of an ensemble is *permutation invariant* if relabeling particles does not change its value [73]. The ensemble mean, ESS, and weight variance are all permutation invariant. Permutation invariance is the right property for ensemble-level summary statistics; permutation equivariance is the right property for per-particle outputs.

Ensemble-aware vs per-particle control. A *per-particle controller* computes each particle’s control using only that particle’s own state and the current observation [72, 8]. It is completely unaware of what other particles are doing. An *ensemble-aware controller* additionally conditions on summary statistics of the full ensemble

(mean, deviation, nESS, weight distribution) to make coordinated decisions [73, 14]. In principle, an ensemble-aware controller can maintain weight diversity by adjusting different particles differently based on their relative positions. In practice, as this thesis demonstrates, the benefit of ensemble context depends on having enough particles for the context to be reliable and on having training labels that encode ensemble-level coordination.

Weight-aware pooling. The *weight-aware pooling* module in this thesis aggregates per-particle embeddings into a global context by computing a weighted mean (using importance weights) and an elementwise maximum [73]. The weighted mean gives more influence to particles that currently carry more posterior probability. The maximum picks out the most extreme embedding in each dimension, providing information about outlier particles. Together, they form a 128-dimensional global context g that is available to each particle’s decoder when computing the final control.

Appendix B

The Lorenz Dynamical Systems

This appendix describes the two dynamical systems used as benchmarks throughout this thesis: the Lorenz-63 model (Section B.1) and the Lorenz-96 model (Section B.2). For each system, the deterministic equations of motion are given, the chaotic behavior and filtering difficulty are characterized, and the stochastic extension used in the filtering experiments is specified. Parameter values used in the experiments are collected in tables at the end of each section.

B.1 Lorenz-63

B.1.1 Historical Context and Motivation

The Lorenz-63 model was introduced by Edward Lorenz in 1963 [46] as a simplified three-variable model of atmospheric convection. Lorenz derived it by truncating the Navier-Stokes equations for a layer of fluid heated from below, retaining only the three most energetically important Fourier modes. Despite its extreme simplicity, the model exhibits deterministic chaos and a strange attractor, and it played a foundational role in establishing that chaotic behavior is generic in nonlinear dynamical systems rather than an artifact of noise or high dimensionality [66].

For data assimilation, Lorenz-63 has become the standard low-dimensional benchmark because it is cheap to simulate, its attractor geometry is well understood, and it captures the essential difficulty of filtering in a chaotic regime: the two-lobe structure means that a particle on the wrong lobe is essentially irrelevant, making weight degeneracy an acute and visible problem even with moderate ensemble sizes.

B.1.2 Equations of Motion

The deterministic Lorenz-63 system is:

$$\begin{aligned}\dot{x}_1 &= \sigma_L(x_2 - x_1), \\ \dot{x}_2 &= x_1(\rho - x_3) - x_2, \\ \dot{x}_3 &= x_1x_2 - \beta x_3,\end{aligned}\tag{B.1.1}$$

with standard parameters $(\sigma_L, \rho, \beta) = (10, 28, 8/3)$. The subscript L on σ_L distinguishes the Lorenz parameter from the diffusion matrix Σ used in the SDE formulation.

The three state variables have physical interpretations in the original convection model: x_1 is proportional to the intensity of convective circulation, x_2 is proportional to the temperature difference between ascending and descending fluid, and x_3 is proportional to the deviation of the vertical temperature profile from linearity. In the filtering context these interpretations are secondary; what matters is the geometric structure of the attractor.

B.1.3 Chaotic Behavior and Attractor Geometry

For the standard parameters, system (B.1.1) has three equilibria: the origin (an unstable saddle) and two symmetric fixed points at $(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$. Neither fixed point is stable; instead, trajectories are attracted to the famous butterfly-shaped strange attractor that winds around both fixed points.

The attractor has the following dynamical properties relevant to filtering:

- **Positive maximum Lyapunov exponent:** $\lambda_1 \approx 0.906$ in standard time units [66], giving an error doubling time of $\tau_d = \ln 2/\lambda_1 \approx 0.765$ TU.
- **Two-lobe structure:** The attractor consists of two lobes corresponding to the two unstable fixed points. Trajectories spiral outward on each lobe before switching to the other. The switching is irregular and unpredictable.
- **Lobe-switching difficulty:** A particle on the left lobe has near-zero likelihood if the true state is on the right lobe. This makes lobe-switching events the hardest filtering scenario: the particle filter must maintain diversity across both lobes at all times to track the true state through a switch.

- **Fractal dimension:** The attractor has a Lyapunov (Kaplan-Yorke) dimension of approximately 2.06, confirming its fractal nature and meaning that it fills a two-dimensional surface embedded in three-dimensional state space.

B.1.4 Stochastic Extension for Filtering

For the filtering experiments in this thesis, the deterministic system (B.1.1) is embedded in the Itô SDE

$$dX_t = f(X_t) dt + \Sigma^{1/2} dW_t, \quad (\text{B.1.2})$$

where f is the deterministic drift from (B.1.1), W_t is a three-dimensional standard Brownian motion, and $\Sigma \succ 0$ is the process noise covariance matrix. The additive noise represents model uncertainty and unresolved dynamics. It also plays an important practical role: without process noise, particles initialized close together would remain close together under the deterministic flow, and the ensemble would not spread appropriately to represent the filtering uncertainty.

The observation model is

$$Y_k = HX_{t_k} + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, R), \quad k = 1, 2, \dots \quad (\text{B.1.3})$$

with observation matrix $H = I_3$ (full state observation) and observation noise covariance $R = \sigma_{\text{obs}}^2 I_3$.

B.1.5 Jacobian

The Jacobian of the Lorenz-63 drift, needed for the sensitivity integrals in the Feynman-Kac optimal control (3.3.10), is:

$$Df(x) = \begin{pmatrix} -\sigma_L & \sigma_L & 0 \\ \rho - x_3 & -1 & -x_1 \\ x_2 & x_1 & -\beta \end{pmatrix}. \quad (\text{B.1.4})$$

This matrix is state-dependent, reflecting the nonlinearity of the system. Along each uncontrolled rollout in the Monte Carlo approximation of the optimal control, the state-transition matrix $\Phi(t) = \exp(\int_0^t Df(x_s) ds)$ is integrated simultaneously with the state, producing a 3×3 matrix that describes how sensitivity to initial conditions evolves along that trajectory.

B.1.6 Experimental Parameters

Table B.1 collects the Lorenz-63 parameters used in all experiments in this thesis.

Table B.1: Lorenz-63 experimental parameters.

Parameter	Value	Description
σ_L	10	Prandtl number
ρ	28	Rayleigh number ratio
β	8/3	Geometric factor
λ_1	≈ 0.906	Maximum Lyapunov exponent
τ_d	≈ 0.765 TU	Error doubling time
σ_{dyn}	1.0	Process noise std (isotropic, $\Sigma = I_3$)
σ_{obs}	2.0 (default)	Observation noise std ($R = 4I_3$)
H	I_3	Observation operator (full state)
ΔT	0.1 TU	Observation interval
δt	0.001 TU	Integration step size
T	20.0 TU	Total assimilation window
N_{steps}	200	Number of observation steps
N	300	Particles (closed-loop eval)
N	32	Particles (training dataset)
x_0	$(-3.139, -2.067, 22.904)^\top$	Reference initial state

B.2 Lorenz-96

B.2.1 Historical Context and Motivation

The Lorenz-96 model was introduced by Edward Lorenz in 1995 [47] as a heuristic model for midlatitude atmospheric dynamics. Unlike Lorenz-63, which was derived from physical principles, Lorenz-96 was constructed to have specific mathematical properties: it is a K -dimensional system with quadratic nonlinearity and energy conservation in the absence of forcing and dissipation, and it exhibits chaotic behavior for sufficiently large forcing. Its primary value is as a benchmark: it is cheap to simulate, its chaotic properties scale with K in a well-understood way, and it has been used in hundreds of data assimilation studies.

The model is particularly useful for studying how filtering methods scale with dimension because the number of positive Lyapunov exponents grows approximately linearly with K , and the structure of the unstable subspace is well characterized. This makes it possible to understand *why* a filter succeeds or fails at a given K , rather than simply observing the outcome.

B.2.2 Equations of Motion

The Lorenz-96 model on a periodic lattice of K variables with cyclic indexing is:

$$\dot{x}_k = (x_{k+1} - x_{k-2})x_{k-1} - x_k + F, \quad k = 1, \dots, K, \quad (\text{B.2.1})$$

with cyclic boundary conditions $x_0 \equiv x_K$, $x_{-1} \equiv x_{K-1}$, and $x_{K+1} \equiv x_1$. The parameter F is a constant external forcing.

Each term in (B.2.1) has a physical interpretation. The quadratic term $(x_{k+1} - x_{k-2})x_{k-1}$ models advection: it conserves energy and is antisymmetric, representing the transport of the quantity x_k (representing a generic meteorological variable such as vorticity or temperature) by the large-scale flow. The linear term $-x_k$ models dissipation. The constant F represents external forcing, analogous to solar heating in the atmosphere. Without forcing, the system decays to the uniform state $x_k = 0$ for all k ; with sufficient forcing, it becomes chaotic.

B.2.3 Chaotic Behavior and Filtering Difficulty

The Lorenz-96 system becomes chaotic for forcing $F \gtrsim 5$ at $K = 8$. This thesis uses $F = 8$ throughout, which corresponds to the strongly chaotic regime. Key dynamical properties at $K = 8$, $F = 8$ are:

- **Positive Lyapunov exponents:** At $K = 8$, $F = 8$, the system has approximately 3 to 4 positive Lyapunov exponents [35], giving an error doubling time of approximately 1.7 TU.
- **Translational symmetry:** The system is invariant under cyclic permutation of the state indices: if x_k is a solution, so is $x_{k+m \bmod K}$ for any integer m . This symmetry means that the attractor has a rotationally symmetric structure and that any state on the attractor has K symmetry-related copies obtained by cyclic shifting. This is exploited in dataset generation to improve attractor coverage.
- **Unstable-neutral subspace:** The dimension of the subspace spanned by directions with non-negative Lyapunov exponents determines the minimum number of particles needed to avoid weight collapse. For $K = 8$, $F = 8$, this subspace has dimension approximately 3 to 4 [35], so $N \approx 8$ to 16 particles is sufficient in principle to span it.

- **Spatial locality:** The drift in (B.2.1) couples only nearest neighbors: $\dot{x}_k$ depends on $x_{k-2}, x_{k-1}, x_k, x_{k+1}$. The Jacobian Df is therefore banded with bandwidth 3, which can be exploited for efficient computation of the state-transition matrix in the Feynman-Kac sensitivity integrals.

B.2.4 Stochastic Extension for Filtering

For the filtering experiments, the deterministic system (B.2.1) is embedded in the Itô SDE

$$dX_t = f(X_t) dt + \Sigma^{1/2} dW_t, \quad (\text{B.2.2})$$

where f is the Lorenz-96 drift, W_t is a K -dimensional standard Brownian motion, and $\Sigma = I_K$ is the isotropic process noise covariance. The observation model is

$$Y_k = X_{t_k} + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, I_K), \quad k = 1, 2, \dots \quad (\text{B.2.3})$$

with identity observation operator (full state observation) and identity observation noise covariance.

B.2.5 Jacobian

The Jacobian of the Lorenz-96 drift is the $K \times K$ matrix with entries:

$$[Df(x)]_{k,j} = \begin{cases} -1 & j = k \\ x_{k-1} & j = k + 1 \\ x_{k+1} - x_{k-2} & j = k - 1 \\ -x_{k-1} & j = k - 2 \\ 0 & \text{otherwise} \end{cases} \quad (\text{B.2.4})$$

with cyclic index arithmetic. The Jacobian is banded with at most 4 nonzero entries per row, making the matrix-vector products required for the sensitivity integral $\mathcal{O}(K)$ rather than $\mathcal{O}(K^2)$. In the augmented system for the Feynman-Kac control, the state-transition matrix $\Phi \in \mathbb{R}^{K \times K}$ satisfies $\dot{\Phi} = Df(x)\Phi$, $\Phi(0) = I_K$, and can be integrated using sparse matrix operations that exploit the banded structure.

B.2.6 Experimental Parameters

Table B.2 collects the Lorenz-96 parameters used in the scaling experiments of Chapter 7.

Table B.2: Lorenz-96 experimental parameters ($K = 8$, $F = 8$).

Parameter	Value	Description
K	8	State dimension
F	8	Forcing parameter (strongly chaotic regime)
λ_1	≈ 0.41	Maximum Lyapunov exponent
τ_d	≈ 1.7 TU	Error doubling time
Positive Lyapunov exponents	3 to 4	Dimension of unstable-neutral subspace
Σ	I_8	Process noise covariance
R	I_8	Observation noise covariance
H	I_8	Observation operator (full state)
ΔT	0.05 TU	Observation interval
δt	0.01 TU	Integration step size
T	5.0 TU	Total assimilation window
N_{steps}	100	Number of observation steps
N	16	Particles (closed-loop eval and training)
$x_0^{(k)}$	$F + 0.01 \mathcal{N}(0, 1)$	Initial condition per variable

B.2.7 Relationship to Full-Scale Lorenz-96

The $K = 8$ experiments in this thesis are a reduced-scale feasibility demonstration. The full Lorenz-96 benchmark used in prior work [72, 35] uses $K = 40$, which has approximately 14 positive Lyapunov exponents and an error doubling time of approximately 0.4 TU, making it substantially harder than the $K = 8$ setting. The $K = 40$ case also requires approximately $137\times$ more computation per Feynman-Kac label (due to the $K + K^2$ dimensional augmented system) and significantly more training data for adequate attractor coverage.

The $K = 8$ experiments establish three things. First, that the dataset generation pipeline produces usable oracle labels for Lorenz-96 dynamics. Second, that the residual ensemble architecture handles an 8-dimensional state and control space. Third, that the learned controller produces competitive filtering performance relative to the uncontrolled PF at this dimension. These three results together support the claim that the pipeline scales beyond Lorenz-63, with full $K = 40$ scaling deferred to future work.

Appendix C

The Girsanov Correction: Derivation & Implications

The Girsanov correction is the single most important structural requirement in this thesis. Every controlled filter evaluated in Chapters 6 and 7 depends on it, and the central lesson of the diagnostic baseline in Section 1.5 is that omitting it causes filter failure regardless of how accurate the learned control is. This appendix explains where the correction comes from, how it is implemented in practice, and what it implies about the theoretical optimum.

The goal here is not to reproduce standard textbook proofs in full generality. The continuous-time Girsanov theorem is stated in any graduate probability text (see [36] for a rigorous treatment); the path-integral optimal control connection is derived in detail in [72, 35]. What this appendix adds is the following: a clear narrative explanation of why the correction has the form it does, the explicit discrete-time approximation that is actually implemented in `pybeeda`, and the connection between the zero-variance theorem and the Stage 2 weight-balance penalty used in training `ensemble_s2`.

C.1 Why the Correction Exists

The particle filter is an importance sampler. Its validity rests on a fundamental premise: the particles are representative samples from the prior predictive distribution, so their importance weights correctly account for how consistent each particle is with the observation. When that premise holds, the weighted empirical measure

converges to the true posterior as $N \rightarrow \infty$.

Nudging breaks that premise. When a control u_t is added to each particle’s SDE, the particles no longer follow the uncontrolled model dynamics. They have been deliberately pushed toward the observation. A particle that ends up close to Y_{k+1} did so partly because of the nudge, not purely because the model dynamics naturally took it there. If the importance weights are computed using only the observation likelihood, they will overcount particles that were artificially placed near the observation, making the filter overconfident. The filter will appear to work well for a few steps and then collapse catastrophically as the overconfident weights cause nearly all probability mass to concentrate on a handful of particles.

The Girsanov correction is the precise mathematical tool for accounting for this artificial placement. It answers the question: given that a particle’s trajectory was governed by the controlled dynamics rather than the uncontrolled model, how much more or less likely would that specific trajectory have been under the model? The answer is the Radon-Nikodym derivative of the uncontrolled path-space measure with respect to the controlled path-space measure, evaluated along the particle’s actual path. Multiplying the observation likelihood by this factor gives a correctly weighted importance weight that accounts for both the observation fit and the trajectory’s plausibility under the model.

A useful physical intuition is this: imagine a particle that was nudged strongly toward the observation and ended up very close to Y_{k+1} . Without the correction, that particle would receive a very high weight because of its proximity to the observation. But if the uncontrolled model would almost never have produced a trajectory that landed that close to the observation from that starting point, the particle is not actually a representative sample from the prior. The Girsanov factor will be small for that particle, partially canceling its high likelihood weight and restoring the correct balance. The net result is a more uniform weight distribution and a healthier ensemble, which is precisely the goal of nudging in the first place.

C.2 The Mathematical Form of the Correction

The derivation below is standard [36, 72] and is included for completeness. Readers who want the full technical detail should consult those references; the key result is equation (C.2.3).

Consider a particle propagating under the uncontrolled SDE

$$dX_t = b(X_t) dt + \sigma dW_t \quad (\text{C.2.1})$$

and under the controlled SDE

$$d\hat{X}_t = b(\hat{X}_t) dt + u_t dt + \sigma dW_t, \quad (\text{C.2.2})$$

both starting from the same initial condition at time t_k . Writing the control as $u_t = \sigma v_t$ and defining $B_t = W_t + \int_{t_k}^t v_s ds$, Girsanov's theorem states that B_t is a Brownian motion under the controlled measure $\mathbb{Q}$, and the Radon-Nikodym derivative of the uncontrolled measure $\mathbb{P}$ with respect to $\mathbb{Q}$ along the controlled path is:

$$\log \frac{d\mathbb{P}}{d\mathbb{Q}} \Big|_{[t_k, t_{k+1}]} = \int_{t_k}^{t_{k+1}} (Q^{-1} u_s)^\top \sigma dW_s + \frac{1}{2} \int_{t_k}^{t_{k+1}} u_s^\top Q^{-1} u_s ds, \quad (\text{C.2.3})$$

where $Q = \sigma \sigma^\top$ is the process noise covariance. This is the log-Girsanov correction that enters the weight update. It consists of two terms. The stochastic integral captures the random fluctuations along the controlled trajectory that distinguish it from uncontrolled paths. The quadratic term accumulates the squared magnitude of the control, penalizing total control effort. Together they measure how unlikely the controlled trajectory would have been under the uncontrolled model.

The corrected weight update is:

$$\tilde{w}_{k+1}^{(i)} \propto w_k^{(i)} \cdot \exp(-g(Y_{k+1}, \hat{X}_{t_{k+1}}^{(i)})) \cdot \exp\left(\log \frac{d\mathbb{P}^{(i)}}{d\mathbb{Q}^{(i)}}\right), \quad (\text{C.2.4})$$

where $g(y, x) = \frac{1}{2}(y-x)^\top R^{-1}(y-x)$ is the negative log-likelihood terminal cost. When $u \equiv 0$, the Girsanov factor is exactly one and the formula reduces to the standard bootstrap weight update.

C.3 The Discrete-Time Implementation

The continuous stochastic integral in (C.2.3) cannot be evaluated exactly in a discrete-time integrator. The implementation in `pybeeda` uses the Euler-Maruyama approximation, accumulating the correction incrementally at each integration substep.

At each substep j with step size δt and Brownian increment $\Delta W_j \sim \mathcal{N}(0, \delta t I)$, the log-correction for particle i is incremented by:

$$\delta \log G_j^{(i)} = (\Sigma^{-1} u^{(i)})^\top \Delta W_j^{(i)} + \frac{\delta t}{2} \|\Sigma^{-1} u^{(i)}\|^2, \quad (\text{C.3.1})$$

where $\Sigma = \sigma_{\text{dyn}}^2 I$ is the isotropic process noise covariance used in all experiments. The total accumulated log-correction over the full assimilation window of N_{sub} substeps is:

$$\log G_k^{(i)} = \sum_{j=0}^{N_{\text{sub}}-1} \left[(\Sigma^{-1} u^{(i)})^\top \Delta W_j^{(i)} + \frac{\delta t}{2} \|\Sigma^{-1} u^{(i)}\|^2 \right]. \quad (\text{C.3.2})$$

There are two things worth noting about this formula. First, the accumulation happens substep-by-substep using the same Brownian increments ΔW_j that are drawn for particle propagation: no additional SDE evaluations are needed. The `NudgedParticleFilter` class in `pybeeda` maintains this running sum internally as particles are propagated, and applies it alongside the log-likelihood at the observation step. Second, the correction is accumulated even when the control is held constant over the window (as in the ML deployment): the stochastic integral varies from particle to particle because each particle draws its own independent Brownian increments, so the Girsanov factor is genuinely random even for a fixed control.

For the hold-constant ML deployment with control $u^{(i)}$ fixed over the entire window, the N_{sub} substep increments can be summed analytically. Since $\sum_j \Delta W_j^{(i)} \sim \mathcal{N}(0, \Delta T I)$, the total correction is:

$$\log G_k^{(i)} = \frac{1}{\sigma_{\text{dyn}}^2} \left[(u^{(i)})^\top \Delta W_{\text{total}}^{(i)} + \frac{\Delta T}{2} \|u^{(i)}\|^2 \right], \quad (\text{C.3.3})$$

where $\Delta W_{\text{total}}^{(i)} \sim \mathcal{N}(0, \Delta T I)$ and $\Delta T = N_{\text{sub}} \cdot \delta t$ is the observation interval. The quadratic term $\Delta T \|u\|^2 / (2\sigma_{\text{dyn}}^2)$ is the deterministic penalty for applying a constant push of magnitude $\|u\|$ over the full window. Large controls produce large deterministic penalties, which partially cancel the high observation likelihood those large controls produce.

This cancellation is the mechanism behind the implicit Girsanov regularization described in Section 6.3. The oracle applies large controls that score well on the likelihood but incur large Girsanov penalties, collapsing nESS. The learned per-particle SR controller, having been trained with Huber regression on the oracle’s heavy-tailed control distribution, learns moderate controls whose quadratic Girsanov penalties are

smaller. The result is a better balance between tracking accuracy and weight diversity.

C.4 The Zero-Variance Property & What It Means for Training

The discrete Girsanov formula also reveals what the theoretically optimal controller would achieve. This result, originally due to Lingala et al. [45] and used extensively in [72, 35], has a direct implication for the Stage 2 weight-balance penalty used to train `ensemble_s2`.

The result. Consider N particles all starting from the same location $x^\star$ at time t_k . The optimal controller $u^\star(t, x) = -Q \nabla_x V(t, x)$, derived from the HJB equation in Section 3.3, has a remarkable property: the combined effect of the observation likelihood and the Girsanov correction produces the same unnormalized weight for every particle, regardless of the random Brownian noise along each trajectory. After normalization, all particles receive weight $1/N$ and the weight variance is exactly zero.

Why this is true. The proof uses Itô’s lemma applied to the value function $V(t, \hat{X}_t)$ along the optimally controlled trajectory. The HJB equation for V is designed so that the drift of $V(t, \hat{X}_t)$ under the optimal control is exactly cancelled by the quadratic term in the Girsanov exponent. What remains after integrating from t_k to t_{k+1} is:

$$g(Y_{k+1}, \hat{X}_{t_{k+1}}^{(i)}) + \int_{t_k}^{t_{k+1}} (v_s^\star)^\top dW_s + \frac{1}{2} \int_{t_k}^{t_{k+1}} \|v_s^\star\|^2 ds = V(t_k, x^\star), \quad (\text{C.4.1})$$

where the left side is exactly the negative log-unnormalized weight of particle i and the right side $V(t_k, x^\star)$ is a deterministic constant that depends only on the fixed starting point and the fixed upcoming observation. Every particle starting from $x^\star$ produces the same left side, so every particle receives the same weight. This is what makes the optimal controller the gold standard: it eliminates weight variance entirely, in the limit where all particles share the same starting location.

Why the result matters for training. In practice, particles start from different locations (the resampled ensemble), so the weight variance is positive but substantially reduced relative to the uncontrolled filter. The Stage 2 weight-balance penalty directly targets this quantity: it computes the variance of predicted post-control

weights via the discrete Girsanov formula (C.3.2) and penalizes it during training. The zero-variance theorem tells us that minimizing this penalty is not an arbitrary regularization choice but a direct approximation of the objective that the optimal controller achieves exactly. The penalty is a soft, differentiable proxy for the same quantity the oracle minimizes implicitly through its HJB construction.

Connection to the oracle gap. The zero-variance property also explains why the oracle gap exists on Lorenz-63. The oracle achieves low weight variance in theory, but in practice it uses only $n_s = 15$ Monte Carlo rollouts per particle to estimate the optimal control. Near attractor lobe boundaries, where the true optimal control direction changes rapidly, this small sample produces a noisy estimate that overshoots. The resulting controls are large, the Girsanov penalties are large, and nESS collapses to 0.75. The learned SR controller, by regressing toward moderate controls through Huber loss training, avoids these overshoots and achieves both better nRMSE and better nESS. The zero-variance theorem tells us the direction both controllers are trying to move in; the difference is how aggressively they move and how much Girsanov penalty they incur along the way.

Appendix D

Dataset Generation Details & Pseudocode

This appendix documents the dataset generation pipeline at the implementation level, complementing the conceptual description in Chapter 4. All pseudocode is derived directly from `init_data_gen.py` (Lorenz-63) and `datagen_196.ipynb` (Lorenz-96) in `pybeebase` [6].

All of the code in this thesis can be found in the `sonal_ml_for_da` branch within the [pybeebase repository linked here](#).¹

Section D.1 specifies the HDF5 dataset schema. Section D.2 states the modeling assumptions. Sections D.3 and D.4 give pseudocode for the Lorenz-63 and Lorenz-96 pipelines. Section D.5 collects verified dataset statistics.

D.1 Dataset Schema

Each training example corresponds to one multi-particle ensemble snapshot. Datasets are stored in HDF5 format with gzip compression. Table D.1 specifies the full schema; only the state dimension d , particle count N , and oracle substep count K_c differ between Lorenz-63 and Lorenz-96.

Normalization note. `ness_w0` and `ness_wf` store raw ESS in $[1, N]$, not normalized nESS. All training and evaluation code divides by N before reporting. The

¹For access to the repository, please reach out to Professor Beeson directly at ryne@princeton.edu.

Table D.1: HDF5 dataset schema. All shapes verified against production files. L63: $N = 32$, $d = 3$, $K_c = 8$. L96 ($K = 8$): $N = 16$, $d = 8$, $K_c = 4$.

Key	Shape	Description
X0	(n, N, d)	Particle states at window start
w0	(n, N)	Normalized importance weights; each row sums to 1
y	(n, d)	Observation vector
dt	$(n,)$	Time horizon Δt (TU)
sigma_obs	$(n,)$	Observation noise std
U_seq	(n, K_c, N, d)	Oracle control sequence; $[b, k, i, :]$ is control for example b , substep k , particle i
X_final	(n, N, d)	Post-control particle states
w_final	(n, N)	Post-control weights with Girsanov correction
log_rn	(n, N)	Per-particle log Radon-Nikodym correction
ness_w0	$(n,)$	Pre-control raw ESS $\in [1, N]$; divide by N for nESS
ness_wf	$(n,)$	Post-control raw ESS $\in [1, N]$; divide by N for nESS
x_true	(n, d)	True attractor state used to construct y
mu_w	(n, d)	Importance-weighted ensemble mean at window start
diag_cov_w	(n, d)	Diagonal of importance-weighted ensemble covariance
oracle_retries	$(n,)$	Oracle retry count ($< 2\%$ of examples require any)

L63 dataset contains a spurious $dt = 0.0$ entry (a null row artifact from HDF5 pre-allocation); the metadata DT_GRID confirms $\{0.15, 0.30, 0.45, 0.60\}$ as the intended grid and 0.0 is not used.

D.2 Modeling Assumptions

Deterministic attractor sampling. All attractor trajectory integration uses deterministic RK4 with no stochastic forcing. This ensures reproducible geometry and avoids correlation between the sampling distribution and the oracle rollouts.

Observation from deterministic forecast. For each true state x_{true} , the observation is constructed as $y = \Phi_{\Delta t}^{\text{det}}(x_{\text{true}}) + \sigma_{\text{obs}} \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$. Using the deterministic forecast as the observation center keeps the observation reachable by particles near x_{true} without requiring extreme oracle controls.

Initial weights from deterministic forecast likelihood.

$$w_0^{(i)} \propto \exp\left(-\frac{\|\Phi_{\Delta t}^{\text{det}}(x_0^{(i)}) - y\|^2}{2\sigma_{\text{obs}}^2}\right). \quad (\text{D.2.1})$$

Training label is first substep only. $U_{\text{target}} = \mathbf{U_seq}[:, :, 0, :]$: the control at the first of K_c oracle substeps. This is the cleanest estimate, before any state evolution under the controlled dynamics.

No clipping at generation time. Oracle controls are stored unclipped. Clipping to $[-U_{\text{clip}}, U_{\text{clip}}]$ is applied at training time.

Oracle robustness. If the oracle raises a floating-point overflow ($< 2\%$ of examples, near attractor lobe boundaries), the ensemble is resampled and the oracle retried up to 25 times.

D.3 Lorenz-63 Generation Pseudocode

D.3.1 Attractor Sampling and Farthest-Point Pruning

Algorithm 1 Greedy farthest-point low-discrepancy pruning (shared by L63 and L96)

Require: Drift f ; initial state x_{init} ; step δt_{att} ; total steps S ; burn-in B ; stride s ; target M

Ensure: $\mathcal{X}_{\text{ld}} \in \mathbb{R}^{M \times d}$

- 1: Integrate deterministic RK4 for S steps; record every s -th state after burn-in B :
pool $\mathcal{X}_{\text{pool}}$
 - 2: Pick random $x_0 \in \mathcal{X}_{\text{pool}}$; $\mathcal{X}_{\text{ld}} \leftarrow \{x_0\}$; $d^2[i] \leftarrow \|p_i - x_0\|^2$
 - 3: **while** $|\mathcal{X}_{\text{ld}}| < M$ **do**
 - 4: $x^* \leftarrow \arg \max_i d^2[i]$; $\mathcal{X}_{\text{ld}} \leftarrow \mathcal{X}_{\text{ld}} \cup \{x^*\}$
 - 5: $d^2[i] \leftarrow \min(d^2[i], \|p_i - x^*\|^2)$
 - 6: **end while**
 - 7: **return** $\mathcal{X}_{\text{ld}}$
-

L63 parameters. $x_{\text{init}} = (-3.139, -2.067, 22.904)^\top$, $\delta t_{\text{att}} = 0.001$ TU, $S = 200,000$, $B = 30,000$, $s = 10$, $M = 5,000$.

D.3.2 Ensemble Regime Sampling

Algorithm 2 Ensemble initialization by degeneracy regime

Require: $\mathcal{X}_{\text{ld}}$; deterministic forecasts to time Δt ; observation y ; regime; N

Ensure: Particle ensemble $X_0 \in \mathbb{R}^{N \times d}$

```

1:  $d^2[i] \leftarrow \|\hat{x}_i - y\|^2$  for all  $i$ 
2: if regime = uniform then
3:   pool  $\leftarrow$  top-min( $3N, 64$ ) closest indices
4:    $X_0 \leftarrow \mathcal{X}_{\text{ld}}[\text{CHOICE}(\text{pool}, N)]$ 
5: else if regime = severe_collapse then
6:   near  $\leftarrow$  top-max( $N, 40$ ) closest; far  $\leftarrow$  bottom- $\lfloor |\mathcal{X}_{\text{ld}}|/10 \rfloor$ 
7:    $n_{\text{near}} \leftarrow \max(2, \lfloor N/3 \rfloor)$ ;  $n_{\text{far}} \leftarrow N - n_{\text{near}}$ 
8:    $X_0 \leftarrow \mathcal{X}_{\text{ld}}[\text{CHOICE}(\text{near}, n_{\text{near}}) \cup \text{CHOICE}(\text{far}, n_{\text{far}})]$ 
9: end if
10: return  $X_0$  (shuffled)

```

Physical interpretation. The *uniform* regime produces initial nESS roughly 40 to 80% of N . The *severe_collapse* regime produces initial nESS roughly 5 to 20% of N , simulating near-degenerate conditions. The mean pre-control nESS across the L63 dataset is 0.224 ± 0.311 (L96: 0.177 ± 0.133), confirming broad coverage of both healthy and degenerate ensemble states.

D.3.3 Oracle Control and Girsanov Accumulation

Algorithm 3 nPF oracle: control sequence and Girsanov correction (L63)

Require: $X_0 \in \mathbb{R}^{N \times 3}$; y ; Δt ; $K_c = 8$; $n_s = 15$; filter $\mathcal{F}$

Ensure: $U_{\text{seq}} \in \mathbb{R}^{K_c \times N \times 3}$; $X_f \in \mathbb{R}^{N \times 3}$; $\log G \in \mathbb{R}^N$

```

1:  $\delta t_{\text{sub}} \leftarrow \Delta t / K_c$ ;  $\log G \leftarrow \mathbf{0}_N$ 
2: for  $k = 0, \dots, K_c - 1$  do
3:    $x_{\text{before}}^{(i)} \leftarrow x^{(i)}$  for all  $i$ 
4:    $u_k^{(i)} \leftarrow \mathcal{F}.\text{calculate\_nudging}(y, k\delta t_{\text{sub}}, \Delta t)$   $\triangleright n_s$  MC rollouts per particle
5:    $U_{\text{seq}}[k, i] \leftarrow u_k^{(i)}$ 
6:   Advect:  $x^{(i)} \leftarrow \mathcal{F}.\text{advect}(x^{(i)}, k\delta t_{\text{sub}}, (k+1)\delta t_{\text{sub}})$ 
7:   for  $i = 1, \dots, N$  do
8:      $\Delta x \leftarrow x_{\text{after}}^{(i)} - x_{\text{before}}^{(i)}$ 
9:      $\log G[i] += -(u_k^{(i)})^\top (\Delta x - f(x_{\text{before}}^{(i)}) \delta t_{\text{sub}}) + \frac{1}{2} \|u_k^{(i)}\|^2 \delta t_{\text{sub}}$ 
10:  end for
11: end for
12: return  $U_{\text{seq}}, \{x^{(i)}\}, \log G$ 

```

Post-control weights are computed as:

$$\log \tilde{w}_f^{(i)} = \log w_0^{(i)} - \frac{\|X_f^{(i)} - y\|^2}{2\sigma_{\text{obs}}^2} + \log G^{(i)}, \quad (\text{D.3.1})$$

followed by softmax normalization.

Algorithm 4 Lorenz-63 dataset generation main loop

Require: $|\mathcal{X}_{\text{id}}| = 5,000$; $N_Y = 200$; $\mathcal{T} = \{0.15, 0.30, 0.45, 0.60\}$ TU; $\Sigma_{\text{obs}} = \{1.0, 2.0, 4.0\}$; Regimes = $\{\text{uniform}, \text{severe_collapse}\}$; $E_{\text{pair}} = 2$; $N = 32$; $K_c = 8$; $n_s = 15$

Ensure: HDF5 with 9,600 allocated rows (4,800 valid)

- 1: Draw $\mathcal{X}_{\text{true}} \subset \mathcal{X}_{\text{id}}$, $|\mathcal{X}_{\text{true}}| = 200$, seed 42
 - 2: **for** each $x_{\text{true}}, \Delta t, \sigma_{\text{obs}}, \text{regime}, j \in E_{\text{pair}}$ **do**
 - 3: $y \leftarrow \Phi_{\Delta t}^{\text{det}}(x_{\text{true}}) + \sigma_{\text{obs}} \varepsilon$
 - 4: $X_0 \leftarrow \text{SAMPLEENSEMBLE}(\mathcal{X}_{\text{id}}, y, N, \text{regime})$
 - 5: $w_0 \leftarrow \text{GAUSSIANWEIGHTS}(X_0, y, \sigma_{\text{obs}})$
 - 6: **Try (up to 25):** $U_{\text{seq}}, X_f, \log G \leftarrow \text{ORACLE}(X_0, y, \Delta t)$
 - 7: $w_f \leftarrow \text{SOFTMAX}(\log w_0 - \frac{\|X_f - y\|^2}{2\sigma_{\text{obs}}^2} + \log G)$
 - 8: Write example to HDF5
 - 9: **end for**
-

D.4 Lorenz-96 Generation Pseudocode

The L96 pipeline follows Algorithm 4 with four modifications: $N = 16$, $K_c = 4$, $n_s = 10$, and the shorter time and noise grids below.

L96 attractor sampling parameters. $x_{\text{init},k} = F + 0.01 \mathcal{N}(0, 1)$, $\delta t_{\text{att}} = 0.01$ TU, $S = 300,000$, $B = 50,000$, $s = 10$, $M = 5,000$. The longer burn-in (50k vs 30k) reflects the slower decorrelation time of L96.

Algorithm 5 Lorenz-96 ($K = 8$) dataset generation

Require: $|\mathcal{X}_{\text{id}}| = 5,000$; $N_Y = 50$; $\mathcal{T} = \{0.05, 0.10\}$ TU; $\Sigma_{\text{obs}} = \{1.0, 2.0\}$; Regimes = $\{\text{uniform}, \text{severe_collapse}\}$; $E_{\text{pair}} = 1$; $N = 16$; $K_c = 4$; $n_s = 10$

Ensure: HDF5 with $50 \times 2 \times 2 \times 2 \times 1 = 400$ examples

- 1: Attractor sampling: Algorithm 1 with L96 drift
 - 2: Run Algorithm 4 with L96 parameters above
-

When run on Della HPC, the job is a Slurm array of 50 tasks. Each processes a disjoint subset of $\mathcal{X}_{\text{true}}$ and writes a shard; shards are merged by concatenating all

keys along axis 0.

D.5 Verified Dataset Statistics

Table D.2: Verified dataset statistics extracted from production HDF5 files. L63 has 9,600 allocated rows and 4,800 valid examples (generation was interrupted; the HDF5 pre-allocates the full expected count and the remaining rows are zero-padded).

Property	Lorenz-63	Lorenz-96 ($K = 8$)
State dimension d	3	8
$ \mathcal{X}_{\text{id}} $	5,000	5,000
Attractor integration steps	200,000	300,000
Burn-in steps	30,000	50,000
Stride s	10	10
Observation centers N_Y	200	50
Time horizons $\mathcal{T}$ (TU)	$\{0.15, 0.30, 0.45, 0.60\}$	$\{0.05, 0.10\}$
Noise levels Σ_{obs}	$\{1.0, 2.0, 4.0\}$	$\{1.0, 2.0\}$
Regimes	uniform, severe collapse	uniform, severe collapse
Ensembles per pair E_{pair}	2	1
Particles per example N	32	16
Oracle substeps K_c	8	4
Oracle rollouts n_s	15	10
Total expected examples	9,600	400
Valid examples	4,800	400
Training examples (85%)	4,080	340
Validation examples (15%)	720	60
Pre-control mean nESS	0.224 ± 0.311	0.177 ± 0.133
Storage format	HDF5 (gzip)	HDF5 (gzip)
Generation platform	Local workstation	Google Colab / Della HPC

Pre-control nESS interpretation. The mean pre-control nESS of 0.224 on L63 (approximately 7 effective particles out of $N = 32$) confirms that the dataset covers genuinely degenerate scenarios. The high standard deviation (0.311) reflects the bi-modal distribution across the two regimes: uniform produces nESS near 0.5, while severe-collapse produces values near 0.05. The L96 mean of 0.177 is lower because spanning 8 unstable dimensions simultaneously leads to faster pre-control weight collapse.

Appendix E

Filter Algorithm Pseudocode

This appendix gives self-contained algorithmic descriptions of all five filter variants evaluated in Chapters 6 and 7, together with the narrative context needed to understand what each algorithm is doing and why it differs from the others. The pseudocode complements the source code in `pybeeda` [5] and `pybeebase` [6]; readers who want to trace a specific implementation detail should look there.

All five filters share the same outer loop: propagate particles forward from one observation time to the next, update importance weights using the observation likelihood, and resample if the ensemble has collapsed below the nESS threshold. What distinguishes them is how the propagation step is augmented. The uncontrolled PF propagates freely under the signal dynamics. The relaxation PF adds a static linear push toward the observation. The oracle nPF computes the theoretically optimal push online using Monte Carlo rollouts. The two learned controllers replace that expensive computation with a single neural network forward pass, while preserving the Girsanov weight correction that makes importance sampling valid.

A key implementation point applies to all controlled filters: the Girsanov log-correction $\log G^{(i)}$ is accumulated incrementally at every SDE integration substep during the assimilation window, not just at observation times. This is because the Radon-Nikodym derivative (3.4.2) integrates the control signal continuously over the window. In the `pybeeda` framework, this accumulation is handled automatically by any filter that registers a nudging controller via the `_calculate_nudging` interface; the learned controllers in this thesis override exactly that method.

Notation. N : particles ($N = 32$ L63, $N = 16$ L96). ΔT : observation interval (0.1 TU L63, 0.05 TU L96). δt : integration step (0.001 TU both systems). $N_{\text{sub}} = \Delta T / \delta t$

(100 for L63, 50 for L96). f : system drift. $\Sigma = \sigma_{\text{dyn}}^2 I$: process noise covariance. $g(y, x) = \frac{1}{2}(y - x)^\top R^{-1}(y - x)$: negative log-likelihood terminal cost. RESAMPLE: systematic (universal) resampling, returning equal output weights.

E.1 Bootstrap Particle Filter

The bootstrap PF is the foundational algorithm and the primary baseline in this thesis. It makes no attempt to steer particles toward the upcoming observation: each particle simply follows the signal dynamics (3.1.1) with independent Brownian increments. When the observation Y_{k+1} arrives, weights are updated by evaluating the Gaussian likelihood at each particle’s final position. Particles that ended up close to the observation receive high weight; those that drifted far away receive exponentially suppressed weight.

The critical weakness of this approach in chaotic systems is that the prior predictive distribution (where the particles actually are after propagating under the dynamics) can become very spread out, with most of its mass far from the observation-consistent region. The likelihood then concentrates almost all weight on the one or two particles that happened to land nearby, collapsing the ESS. This is the degeneracy problem that nudging is designed to prevent. No Girsanov correction is needed here because the proposal distribution (the signal dynamics) is exactly the model prior. There is no change of measure to correct for.

Algorithm 6 Bootstrap Particle Filter (uncontrolled baseline)

Require: Initial particles $\{x_0^{(i)}\}$; weights $w_0^{(i)} = 1/N$; observations $\{Y_k\}_{k=1}^K$

Ensure: Filtered estimates $\{\hat{x}_k\}_{k=1}^K$; per-step nESS

```
1: for  $k = 0, \dots, K - 1$  do
2:   for  $i = 1, \dots, N$  do                                 $\triangleright$  Propagate all particles: no control
3:     for  $j = 1, \dots, N_{\text{sub}}$  do
4:        $\Delta W \sim \mathcal{N}(0, \delta t I)$ 
5:        $x^{(i)} \leftarrow x^{(i)} + f(x^{(i)}) \delta t + \Sigma^{1/2} \Delta W$ 
6:     end for
7:   end for
8:   for  $i = 1, \dots, N$  do                                 $\triangleright$  Weight update: observation likelihood only
9:      $\tilde{w}^{(i)} \leftarrow w_k^{(i)} \cdot \exp(-g(Y_{k+1}, x^{(i)}))$ 
10:  end for
11:  Normalize:  $w^{(i)} \leftarrow \tilde{w}^{(i)} / \sum_j \tilde{w}^{(j)}$ 
12:   $N_{\text{eff}} \leftarrow 1 / \sum_i (w^{(i)})^2$ ; record nESS  $\leftarrow N_{\text{eff}}/N$            $\triangleright$  Captured before any
    resampling
13:  if  $N_{\text{eff}} < N/2$  then
14:     $\{x^{(i)}, w^{(i)}\} \leftarrow \text{RESAMPLE}(w, x)$ 
15:  end if
16:   $\hat{x}_{k+1} \leftarrow \sum_i w^{(i)} x^{(i)}$ 
17: end for
```

E.2 Relaxation Nudged Particle Filter

The relaxation PF [70] is the classical nudging baseline. Before propagating, each particle receives a nudge computed from the static Kalman gain K_{relax} : it is pushed toward the upcoming observation Y_{k+1} by an amount proportional to its current distance from that observation. The gain $K_{\text{relax}} = \sigma_{\text{dyn}}^2 / (\sigma_{\text{dyn}}^2 + \sigma_{\text{obs}}^2)$ is the same Kalman update coefficient that would be used in a linear-Gaussian system. It balances between trusting the model (small gain when σ_{obs} is large) and trusting the observation (large gain when σ_{obs} is small).

The key limitation is that this gain is time-invariant and particle-independent: every particle receives the same push magnitude regardless of its current position on the attractor, and the gain does not account for how far away the next observation is in time. Near an attractor lobe switch, where the optimal push direction changes rapidly, a fixed linear nudge is inadequate.

Crucially, because the relaxation PF changes the particle proposal from the prior dynamics to a controlled SDE, the Girsanov correction must be accumulated throughout the window to maintain importance-sampling validity. The log-correction $\log G^{(i)}$ is incremented at every substep using the discrete approximation of the Girsanov stochastic integral.

Algorithm 7 Relaxation Particle Filter [70]

Require: Static gain $K_{\text{relax}} = \sigma_{\text{dyn}}^2 / (\sigma_{\text{dyn}}^2 + \sigma_{\text{obs}}^2) \cdot I$; all other inputs as Algorithm 6

Ensure: Filtered estimates; per-step nESS

```

1: for  $k = 0, \dots, K - 1$  do
2:   for  $i = 1, \dots, N$  do                                     ▷ Compute hold-constant nudge
3:      $u^{(i)} \leftarrow K_{\text{relax}} (Y_{k+1} - x_k^{(i)})$ 
4:   end for
5:   for  $i = 1, \dots, N$  do                                     ▷ Propagate with nudge; accumulate Girsanov
6:      $\log G^{(i)} \leftarrow 0$ 
7:     for  $j = 1, \dots, N_{\text{sub}}$  do
8:        $\Delta W \sim \mathcal{N}(0, \delta t I)$ 
9:        $x^{(i)} \leftarrow x^{(i)} + (f(x^{(i)}) + u^{(i)}) \delta t + \Sigma^{1/2} \Delta W$ 
10:       $\log G^{(i)} += (\Sigma^{-1} u^{(i)})^\top \Delta W + \frac{\delta t}{2} \|\Sigma^{-1} u^{(i)}\|^2$ 
11:    end for
12:  end for
13:  for  $i = 1, \dots, N$  do                                     ▷ Corrected weight update
14:     $\tilde{w}^{(i)} \leftarrow w_k^{(i)} \cdot \exp(-g(Y_{k+1}, x^{(i)}) + \log G^{(i)})$ 
15:  end for
16:  Normalize, record nESS, resample if  $N_{\text{eff}} < N/2$ , compute  $\hat{x}_{k+1}$ 
17: end for

```

Gain values. For L63 ($\sigma_{\text{dyn}} = 1.0$, $\sigma_{\text{obs}} = 2.0$): $K_{\text{relax}} = 1/(1+4) \cdot I_3 = 0.2 \cdot I_3$. For L96 ($\sigma_{\text{dyn}} = 1.0$, $\sigma_{\text{obs}} = 1.0$): $K_{\text{relax}} = 1/(1+1) \cdot I_8 = 0.5 \cdot I_8$. The L96 gain is larger because the observation noise equals the process noise, so the observation provides relatively more information per unit time.

E.3 Oracle Nudged Particle Filter

The oracle nPF [72, 35, 7] is the performance ceiling for all learned controllers. Rather than applying a fixed linear gain, it computes the theoretically optimal control at each of K_c substeps during the assimilation window using the Feynman-Kac Monte Carlo approximation derived in Section 3.3.5. The optimal control at time t is proportional to the negative gradient of the value function, which is approximated by running n_s uncontrolled rollouts forward from the current particle position and computing a likelihood-weighted average of their gradient directions toward the observation.

The oracle has two important properties. First, it is adaptive: the control direction is recomputed K_c times per window, allowing it to respond to how the state evolves under the controlled dynamics. Second, it is expensive: each rollout must integrate the full $(d + d^2)$ -dimensional augmented system that tracks both the state and the state-transition matrix (the Jacobian integral in Equation (3.3.9)). For $d = 3$ (L63), this is a 12-dimensional system; for $d = 8$ (L96), it is a 72-dimensional system.

The Girsanov correction accumulates here exactly as in the relaxation PF, except that the control u^* changes at each substep boundary rather than remaining constant over the window.

Algorithm 8 Oracle Nudged Particle Filter (Feynman-Kac Monte Carlo) [72, 7]

Require: Oracle rollouts n_s (15 for L63, 10 for L96); control substeps K_c (8 for L63, 4 for L96); all other inputs as Algorithm 6

Ensure: Filtered estimates; per-step nESS

```

1: for  $k = 0, \dots, K - 1$  do
2:   for  $i = 1, \dots, N$  do
3:      $\log G^{(i)} \leftarrow 0$ 
4:     for  $c = 1, \dots, K_c$  do ▷ Recompute control at each substep
5:        $t_{\text{rem}} \leftarrow \Delta T(1 - (c - 1)/K_c)$  ▷ Remaining time to observation
6:        $u^* \leftarrow \text{FEYNMANKAC}(x^{(i)}, t_{\text{rem}}, Y_{k+1}, n_s)$  ▷  $n_s$  rollouts of  $(d + d^2)$ -dim
augmented system
7:       for  $j = 1, \dots, N_{\text{sub}}/K_c$  do ▷ Propagate for one substep interval
8:          $\Delta W \sim \mathcal{N}(0, \delta t I)$ 
9:          $x^{(i)} \leftarrow x^{(i)} + (f(x^{(i)}) + u^*) \delta t + \Sigma^{1/2} \Delta W$ 
10:         $\log G^{(i)} += (\Sigma^{-1} u^*)^\top \Delta W + \frac{\delta t}{2} \|\Sigma^{-1} u^*\|^2$ 
11:      end for
12:    end for
13:  end for
14:  Corrected weight update, normalize, resample if  $N_{\text{eff}} < N/2$ , compute  $\hat{x}_{k+1}$ 
15: end for

```

Computational cost. The dominant cost is $N \times K_c \times n_s$ augmented SDE integrations per observation step. For L63 ($N = 32$, $K_c = 8$, $n_s = 15$, 200 observation steps): $200 \times 8 \times 32 \times 15 = 768,000$ integrations of a 12-dimensional system per trial, measured at 30,518 ms per observation step. For L96 ($N = 16$, $K_c = 4$, $n_s = 10$, 100 steps): $100 \times 4 \times 16 \times 10 = 64,000$ integrations of a 72-dimensional system per trial, measured at 3,253 ms per step. The learned controllers replace this entire computation with a single MLP forward pass.

Oracle as expert for imitation learning. Beyond serving as a closed-loop baseline, the oracle plays a second role: it is the expert whose behavior is imitated during training. For each attractor state and observation pair in the dataset, the oracle is

queried offline to produce a labeled control u^* , and the neural network is trained via behavioral cloning to reproduce those controls. At deployment, the network replaces the oracle entirely.

E.4 Per-Particle Learned Controller

The per-particle SR controller is the core result of this thesis. It replaces the oracle’s online Feynman-Kac computation with a single MLP forward pass, processing all N particles in one batched call. The controller is entirely per-particle: it sees no information about other particles’ states or weights, only its own state $x_k^{(i)}$, the observation Y_{k+1} , the innovation $\iota^{(i)} = x_k^{(i)} - Y_{k+1}$, its own weight $w_k^{(i)}$, the time horizon ΔT , and the observation noise σ_{obs} .

A critical design choice is that the control is held constant over the entire assimilation window. The oracle recomputes its control at each of $K_c = 8$ substeps; the learned controller computes one control at the start and applies it for all $N_{\text{sub}} = 100$ integration steps. This reduces the number of model evaluations from K_c to 1 per particle per observation step, which is what makes the learned controller as fast as the uncontrolled PF.

The Girsanov correction is accumulated at every integration substep, exactly as in the oracle and relaxation filters. This is the key structural requirement that makes the controlled filter importance-consistent. The control magnitude from the learned controller averages $\|\bar{u}\| \approx 4.8$ on L63, which is moderate enough that the Girsanov penalties remain small and the ensemble stays healthy.

Algorithm 9 Per-Particle Learned Nudged PF (`per_particle`)

Require: Trained `PerParticleBaseline` μ_θ (43,203 parameters for L63; 45,448 for

L96); normalizer statistics $(\mu_\xi, \sigma_\xi, \mu_u, \sigma_u)$ fit on the training split; deployment clip

$U_{\text{clip}} = 100$

Ensure: Filtered estimates; per-step nESS

```
1: for  $k = 0, \dots, K - 1$  do
2:   Compute ML control (one forward pass for all particles):
3:     Compute innovations:  $\iota^{(i)} \leftarrow x_k^{(i)} - Y_{k+1}$ 
4:     Assemble and normalize inputs per particle
5:      $U_n \leftarrow \mu_\theta(X_n, w_n, Y_n, \Delta T_n, \sigma_{\text{obs},n}) \quad \triangleright \text{Shape } (N, d)$ ; all particles batched
6:     Denormalize and clip:
7:      $u^{(i)} \leftarrow \text{clip}(U_n[i] \cdot \sigma_u + \mu_u, -U_{\text{clip}}, U_{\text{clip}})$  for each  $i$ 
8:   Propagate with hold-constant ML control; accumulate Girsanov:
9:   for  $i = 1, \dots, N$  do
10:     $\log G^{(i)} \leftarrow 0$ 
11:    for  $j = 1, \dots, N_{\text{sub}}$  do
12:       $\Delta W \sim \mathcal{N}(0, \delta t I)$ 
13:       $x^{(i)} \leftarrow x^{(i)} + (f(x^{(i)}) + u^{(i)}) \delta t + \Sigma^{1/2} \Delta W$ 
14:       $\log G^{(i)} += (\Sigma^{-1} u^{(i)})^\top \Delta W + \frac{\delta t}{2} \|\Sigma^{-1} u^{(i)}\|^2$ 
15:    end for
16:  end for
17:  Corrected weight update, normalize, resample if  $N_{\text{eff}} < N/2$ , compute  $\hat{x}_{k+1}$ 
18: end for
```

Why the learned controller can outperform its teacher. The oracle is trained to minimize weight variance for a given particle position, and does so by applying aggressive controls that compress the ensemble toward the observation. This collapses the nESS to 0.75 on L63. The learned controller, trained with the Huber loss ($\delta = 1.0$) on the oracle’s demonstrated controls, regresses toward the median of the oracle control distribution rather than the mean. Because the oracle’s distribution is heavy-tailed (large controls occur at lobe switches), the learned policy is systematically more conservative, preserving nESS at 0.87. The result is better closed-loop performance

(nRMSE 0.16 vs 0.61) despite lower per-step control fidelity. This is the implicit Girsanov regularization described in Section 6.3.

E.5 Ensemble-Aware Learned Controller

The ensemble-aware variants (`ensemble_s1` and `ensemble_s2`) extend the per-particle baseline by conditioning each particle’s control on a global summary of the entire ensemble. The motivation is that in a real filtering setting, the optimal control for particle i depends not just on its own state but on where the other particles are: if the ensemble is already well-concentrated near the observation, a strong nudge is unnecessary and would only collapse the ESS. If the ensemble is widely spread, a stronger coordinated push is warranted.

The global context $g \in \mathbb{R}^{128}$ is computed once at the start of each assimilation window and held fixed throughout propagation. This is a deliberate design choice: if g were recomputed at each substep using the evolving particle states, the particle SDEs would become coupled and the Girsanov correction would no longer apply independently per particle. By fixing g at the window boundary, each particle’s controlled SDE remains conditionally independent of the others given g , and the Girsanov accumulation proceeds identically to the per-particle case.

The architecture follows the DeepSets framework [73]: the encoder is applied identically to all particles (permutation equivariance), and the pooling uses only permutation-invariant operations (weighted mean and elementwise maximum). The final control is a residual correction on top of the per-particle baseline, initialized with a small learnable scale $s = 0.1$ to guarantee that the ensemble variant performs at least as well as the per-particle checkpoint at the start of training.

In practice on L63 with $N = 32$ particles, the pooled context g is too noisy to provide reliable collective information. The ensemble variants underperform the per-particle baseline because the noise in g corrupts the strong per-particle base rather than enriching it. The weight-balance penalty variant (`ensemble_s2`) achieves the highest nESS of any variant (0.93) by learning more conservative controls, but at the cost of tracking accuracy.

Algorithm 10 Ensemble-Aware Learned Nudged PF (`ensemble_s1` and `ensemble_s2`)

Require: Trained `ResidualEnsembleController` μ_θ (59,971 parameters for L63; 62,856 for L96); normalizer $(\mu_\xi, \sigma_\xi, \mu_u, \sigma_u)$; $U_{\text{clip}} = 100$

Ensure: Filtered estimates; per-step nESS

- 1: **for** $k = 0, \dots, K - 1$ **do**
 - 2: **Compute ensemble summary** (once per window; held fixed):
 - 3: $\bar{x}_k \leftarrow \sum_i w_k^{(i)} x_k^{(i)}$ ▷ Importance-weighted mean
 - 4: $\delta^{(i)} \leftarrow x_k^{(i)} - \bar{x}_k$ for all i ▷ Per-particle deviation from mean
 - 5: $\iota^{(i)} \leftarrow x_k^{(i)} - Y_{k+1}$ for all i ▷ Per-particle innovation
 - 6: **Encoder** (applied identically to all particles):
 - 7: Per-particle input: $\xi^{(i)} = (x_k^{(i)}, Y_{k+1}, \Delta T, \sigma_{\text{obs}}, \iota^{(i)}, \delta^{(i)}, w_k^{(i)})$ ▷ 15-dim for L63; 35-dim for L96
 - 8: $h^{(i)} \leftarrow \phi_{\text{enc}}(\xi_n^{(i)}) \in \mathbb{R}^{64}$ for all i ▷ Batched; permutation equivariant
 - 9: **Pooling** (permutation invariant; computed once):
 - 10: $\bar{h} \leftarrow \sum_i w_k^{(i)} h^{(i)}$ ▷ Importance-weighted mean embedding
 - 11: $h_{\text{max}} \leftarrow \max_i h^{(i)}$ ▷ Elementwise max over particles
 - 12: $g \leftarrow [\bar{h}; h_{\text{max}}] \in \mathbb{R}^{128}$ ▷ Global context; fixed for this window
 - 13: **Decoder** (residual architecture):
 - 14: $u_n^{(i)} \leftarrow \phi_{\text{base}}(h^{(i)}) + s \phi_{\text{corr}}(h^{(i)}, g)$ for all i
 - 15: Denormalize and clip: $u^{(i)} \leftarrow \text{clip}(u_n^{(i)} \cdot \sigma_u + \mu_u, -U_{\text{clip}}, U_{\text{clip}})$
 - 16: **Propagate and accumulate Girsanov:** ▷ Identical to Algorithm 9, Steps 7-14
 - 17: Corrected weight update, normalize, resample if $N_{\text{eff}} < N/2$, compute $\hat{x}_{k+1}$
 - 18: **end for**
-

Conditional particle independence. Because g is computed from the particle states at t_k and held fixed during propagation from t_k to t_{k+1} , each particle’s controlled SDE is conditionally independent of the others given g . This means the Girsanov correction applies per-particle in exactly the same way as in Algorithm 9: there is no cross-particle coupling to account for in the weight update. This independence

is what allows the ensemble-aware filter to be a valid importance sampler despite conditioning on collective information.

ensemble_s1 vs ensemble_s2. These two variants are identical at deployment time and run the same algorithm. The difference is entirely in how μ_θ was trained. **Ensemble_s1** was trained with the Huber imitation loss only (Stage 1). **Ensemble_s2** was fine-tuned from the **ensemble_s1** checkpoint with an additional Stage 2 penalty on the predicted post-control weight variance, computed differentially via the Gir-sanov formula with $\lambda^* = 0.1$. The penalty explicitly penalizes control patterns that would collapse the ensemble, teaching the controller to trade some tracking accuracy for better weight diversity. This is reflected in the closed-loop results: **ensemble_s2** achieves the highest nESS of any variant (0.93) at the cost of a higher nRMSE (0.68 vs 0.63 for **ensemble_s1**).

E.6 Systematic Resampling

Resampling is the mechanism that prevents weight degeneracy from becoming permanent. When the nESS falls below 0.5, it is triggered: particles are redrawn with replacement from the current weighted ensemble, with probabilities proportional to their weights. High-weight particles are duplicated; low-weight particles are discarded. All weights are then reset to $1/N$.

The method used throughout this thesis is systematic (universal) resampling, which draws a single uniform random variable and spaces the N sample points uniformly from it. This is preferred over multinomial resampling because it introduces less variance in the number of times each particle is selected: a particle with weight w is selected either $\lfloor Nw \rfloor$ or $\lceil Nw \rceil$ times, rather than having count variance $Nw(1-w)$ as in multinomial resampling. This reduced variance means fewer spurious duplications of high-weight particles and fewer wasteful discards of moderate-weight ones.

One subtlety that matters for the evaluation in this thesis: nESS is recorded *before* resampling at every observation step. This means the nESS timeseries plots in Chapter 6 show the true pre-resample degeneracy level, including dips below the 0.5 threshold that trigger resampling. If nESS were recorded after resampling it would always appear close to 1, hiding the degeneracy information entirely.

Algorithm 11 Systematic (Universal) Resampling [18]

Require: Normalized weights $\{w^{(i)}\}_{i=1}^N$; particle states $\{x^{(i)}\}$

Ensure: Resampled states $\{\tilde{x}^{(i)}\}_{i=1}^N$; equal weights $1/N$

```
1: Compute cumulative weights:  $C_i \leftarrow \sum_{j=1}^i w^{(j)}$ ;  $C_0 = 0$ 
2: Draw  $u_1 \sim \mathcal{U}(0, 1/N)$ ;  $u_i \leftarrow u_1 + (i - 1)/N$  for  $i = 1, \dots, N$   $\triangleright$  One random
   draw;  $N$  evenly spaced sample points
3:  $j \leftarrow 1$ 
4: for  $i = 1, \dots, N$  do
5:   while  $u_i > C_j$  do
6:      $j \leftarrow j + 1$ 
7:   end while
8:    $\tilde{x}^{(i)} \leftarrow x^{(j)}$   $\triangleright$  Select particle  $j$ ; may be selected multiple times
9: end for
10:  $\tilde{w}^{(i)} \leftarrow 1/N$  for all  $i$ 
11: return  $\{\tilde{x}^{(i)}, \tilde{w}^{(i)}\}_{i=1}^N$ 
```

In `pybeeda`, resampling is handled by the `Universal` sampler with `effective_particle_number_upper_bound =  $\lfloor 0.5 \times N \rfloor$` . This threshold is checked after every weight normalization, before the posterior mean estimate is computed.

E.7 Filter Comparison Summary

Table E.1 summarizes the key structural properties of each filter, and provides the concrete cost comparison that motivates the imitation learning approach.

The oracle cost scales as $\mathcal{O}(d^2)$ per rollout because each augmented rollout integrates the $(d + d^2)$ -dimensional sensitivity system that computes the gradient of the value function. The learned controllers replace this entirely: the offline training process has already extracted whatever the oracle knows into the network weights, so deployment requires only $\mathcal{O}(dH)$ per particle per step at fixed $H = 128$.

The measured speedups confirm this scaling. On L63, where $d = 3$ and the oracle integrates a 12-dimensional system, the speedup is $70\times$ (438 ms vs 30,518 ms per step). On L96, where $d = 8$ and the oracle integrates a 72-dimensional system, the speedup is $27\times$ (120 ms vs 3,253 ms per step). The speedup is smaller on L96 despite

Table E.1: Summary of filter algorithm properties. All controlled filters include the Girsanov correction. $H = 128$ is the encoder hidden width; d_{in} is the encoder input dimension (12 or 15 for L63; 27 or 35 for L96).

Filter	Control	Girsanov	Ensemble	Cost	Algorithm
Bootstrap PF	None	No	No	$\mathcal{O}(N \cdot N_{\text{sub}} \cdot d)$	6
Relaxation PF	Linear	Yes	No	$\mathcal{O}(N \cdot N_{\text{sub}} \cdot d)$	7
Oracle nPF	Feynman-Kac	Yes	No	$\mathcal{O}(N \cdot K_c \cdot n_s \cdot N_{\text{sub}} \cdot d^2)$	8
<i>per_particle</i>	MLP (imitation)	Yes	No	$\mathcal{O}(N \cdot d_{\text{in}} \cdot H)$	9
<i>ensemble_s1/s2</i>	MLP (imitation)	Yes	Yes	$\mathcal{O}(N \cdot d_{\text{in}} \cdot H)$	10

the larger augmented system dimension because the L96 oracle uses fewer rollouts ($n_s = 10$ vs 15) and fewer substeps ($K_c = 4$ vs 8), reducing the total oracle cost per step. The learned controller’s cost, by contrast, grows only modestly with d because the input dimension increases from 12 to 27 (per-particle) or 15 to 35 (ensemble variants) while H remains fixed.

Appendix F

Hyperparameters & Implementation Details

This appendix collects all verified hyperparameters, architecture specifications, and implementation choices in one place for reproducibility. All values are taken directly from the generation scripts and evaluation notebooks in `pybeebase` [6]. Hyperparameters here are sourced from the training notebook source code, which can be found within the `pybeebase` repository.

The appendix is organized by pipeline stage. Section F.1 covers dataset generation. Section F.2 documents the model architectures with verified parameter counts extracted directly from the saved checkpoints. Section F.3 specifies the two-stage training protocol. Section F.4 records the closed-loop evaluation setup, including a note on a trial count discrepancy between the L96 script header and the actual saved results. Section F.6 lists software versions and hardware.

F.1 Dataset Generation Hyperparameters

Dataset generation is one of the most computationally intensive stages in the pipeline (along with filtering evaluation), but also the one most likely to need modification if this work is extended to new systems. The two most consequential choices are the attractor sampling parameters and the oracle configuration.

For the attractor sampling, the burn-in period must be long enough for the trajectory to fully forget its initial condition and settle onto the invariant measure. For Lorenz-63, 30,000 steps at $\delta t = 0.001$ TU gives 30 TU of burn-in, roughly 40 error

doubling times, which is conservative. For Lorenz-96, the burn-in is 50,000 steps at $\delta t = 0.01$ TU, giving 500 TU, which is similarly conservative given the ≈ 1.7 TU error doubling time. The stride of 10 reduces correlation between successive pool points.

For the oracle configuration, the key tradeoff is between n_s (number of Monte Carlo rollouts per particle per substep, which determines label quality) and K_c (number of control substeps per assimilation window, which determines how adaptively the oracle responds to state evolution). Both are reduced for Lorenz-96 relative to Lorenz-63 to keep per-label generation time tractable: each L96 rollout integrates a 72-dimensional augmented system, compared to 12 dimensions for L63.

The time horizon grid $\mathcal{T}$ covers the range of observation intervals likely to be encountered in the closed-loop filter. For L63, the grid extends from 0.15 to 0.60 TU, spanning roughly one-fifth to three-quarters of an error doubling time. For L96, the grid is compressed to 0.05 to 0.10 TU to match the shorter timescales of that system.

The two degeneracy regimes (*uniform* and *severe_collapse*) are designed to cover the range of initial ensemble states the filter encounters in practice. In the uniform regime, all particles start as plausible hypotheses for the true state. In the severe-collapse regime, two-thirds of particles are seeded far from the observation-consistent region, mimicking a post-lobe-switch scenario where the ensemble has already partially degenerated. Training on both regimes ensures the network sees controls corresponding to both healthy and struggling filter states.

Table F.1: Verified dataset generation hyperparameters. Values extracted from the `production` configuration blocks in `init_data_gen.py` (L63) and `datagen_196.ipynb` (L96).

Parameter	Lorenz-63	Lorenz-96 ($K = 8$)
<i>Attractor sampling</i>		
Integration steps S	200,000	300,000
Burn-in steps B	30,000	50,000
Stride s	10	10
FPS target $ \mathcal{K}_{\text{id}} $	5,000	5,000
Attractor step δt_{att} (TU)	0.001	0.01
Integrator	Deterministic RK4	Deterministic RK4
<i>Dataset construction</i>		
Observation centers N_Y	200	50
Time horizon grid $\mathcal{T}$ (TU)	$\{0.15, 0.30, 0.45, 0.60\}$	$\{0.05, 0.10\}$
Noise grid Σ_{obs}	$\{1.0, 2.0, 4.0\}$	$\{1.0, 2.0\}$
Regimes	uniform, severe collapse	uniform, severe collapse
E_{pair}	2	1
Particles per example N	32	16
Valid examples	4,800	400
<i>Oracle configuration</i>		
Rollouts n_s	15	10
Substeps K_c	8	4
Oracle integration step δt (TU)	0.001	0.01
Integrator	Wiener-RK4-Maruyama	Wiener-RK4-Maruyama
Augmented system dimension	12 ($= d + d^2$)	72 ($= d + d^2$)
Clip at generation time	None	None
Retry limit	25	25
<i>Normalization</i>		
Train/val split	85% / 15%	85% / 15%
Split seed	42	42
Method	Z-score per feature	Z-score per feature
Fit on	Training split only	Training split only

F.2 Model Architecture Hyperparameters

All four deployed models share the same encoder-decoder skeleton: a `PerParticleEncoder` MLP that maps each particle’s input to a 64-dimensional embedding, and a decoder MLP that maps that embedding (plus, for the ensemble variants, a 128-dimensional global context) to a d -dimensional control. The encoder is applied identically and independently to every particle in a single batched forward pass, which is what enforces permutation equivariance by construction.

The parameter count difference between Lorenz-63 and Lorenz-96 is almost entirely due to the larger input and output dimensions: the same hidden widths (128, 64) are used for both systems. For the per-particle variant, the encoder input grows from 12 to 27 dimensions because the state, observation, and innovation features all scale with d , and the decoder output grows from 3 to 8 dimensions. For the ensemble variant, the additional deviation feature $\delta^{(i)} = x^{(i)} - \bar{x}$ adds another d dimensions, bringing the input to 15 (L63) or 35 (L96).

The residual scale s , initialized to 0.1, serves an important purpose during training: it ensures that at initialization the ensemble correction adds only a small perturbation on top of the transferred per-particle base, so the ensemble variant starts from a point of guaranteed good performance rather than random noise. As training proceeds, s is learned jointly with all other parameters and can grow if the ensemble correction is useful.

All parameter counts below are verified by summing the shapes of all tensors in the saved checkpoint state dicts.

Table F.2: Verified architecture hyperparameters. Parameter counts are extracted directly from saved checkpoints by summing all tensor element counts.

Parameter	Lorenz-63	Lorenz-96
<i>PerParticleBaseline</i>		
Encoder input dim	12	27
Input features	$(x^{(i)}, y, \Delta t, \sigma_{\text{obs}}, \iota^{(i)}, w^{(i)})$	same ($d = 8$)
Total parameters	43,203	45,448
<i>ResidualEnsembleController</i>		
Encoder input dim	15	35
Additional feature vs baseline	$\delta^{(i)} = x^{(i)} - \bar{x}$ (3-dim)	$\delta^{(i)}$ (8-dim)
Total parameters	59,971	62,856
<i>Shared architecture (both variants)</i>		
Encoder layers	$[d_{\text{in}} \rightarrow 128 \rightarrow 128 \rightarrow 64]$	same
Base decoder	$[64 \rightarrow 128 \rightarrow 64 \rightarrow d]$	same
Correction decoder	$[192 \rightarrow 128 \rightarrow 64 \rightarrow d]$	same ($192 = 64 + 128$)
Pooling output	$[\bar{h}; h_{\text{max}}] \in \mathbb{R}^{128}$	same
Learnable scale s	initialized to 0.1	same
Activation	ReLU (no activation at final layer)	same

F.3 Training Protocol Hyperparameters

Training proceeds in two stages. Stage 1 is oracle imitation via behavioral cloning: the network is trained to minimize the Huber loss between its predicted controls and the oracle’s demonstrated controls on the training split. The Huber loss with $\delta = 1.0$ is chosen over MSE specifically because the oracle control distribution is heavy-tailed: near attractor lobe boundaries, the oracle applies very large controls that would dominate an MSE loss and cause the network to overfit to rare extreme cases. The Huber loss transitions to linear behavior for residuals above $\delta = 1.0$, reducing the influence of these outliers and producing a learned policy that regresses toward more moderate controls. This is the mechanism behind the implicit Girsanov regularization described in Section 6.3.

Controls are clipped to $[-100, 100]$ before normalization at training time. This removes roughly 2% of oracle labels that have extremely large magnitude typically

from near-singular attractor regions and prevents those rare examples from dominating the loss surface. The clip value matches the deployment-time clip so that the network never encounters inputs that fall outside its training distribution.

The cosine annealing learning rate schedule is used rather than a step schedule because the training loss landscape for imitation learning is relatively smooth: the cosine decay allows the optimizer to take large steps early in training and gradually reduce the step size as the model approaches a good solution, without requiring manual tuning of a decay epoch. Early stopping with patience 15 prevents overfitting on the relatively small L96 dataset of 340 training examples.

Stage 2 applies only to `ensemble_s2` and is the weight-balance fine-tuning step. Starting from the best Stage 1 checkpoint, the model is fine-tuned with an augmented loss that adds a penalty on the predicted post-control weight variance, computed differentiably via the Girsanov formula in physical denormalized units. The learning rate is reduced to one-tenth of Stage 1 to preserve the imitation behavior learned in Stage 1 while making targeted adjustments to improve ensemble health. The λ sweep runs four independent Stage 2 runs from the same Stage 1 checkpoint; $\lambda^* = 0.1$ is selected as the Pareto-optimal value on the offline validation set’s RMSE-nESS frontier.

Table F.3: Stage 1 training hyperparameters (behavioral cloning from oracle). These apply to all four model variants; ensemble variants additionally use the transfer initialization described below.

Parameter	Value	Notes
Optimizer	Adam [38]	
Initial learning rate η_0	3×10^{-4}	
LR schedule	Cosine annealing	Decays to 10^{-6} over max epochs
Max epochs	150	
Batch size	256	
Gradient clip (L2 norm)	1.0	Applied before parameter update
Loss function	Huber, $\delta = 1.0$	Quadratic below δ ; linear above; reduces influence of outlier labels
Loss space	Normalized (z-scored)	Both predictions and oracle targets in normalized space
Training label	<code>U_seq[:, :, 0, :]</code>	First oracle substep only; cleanest estimate before state evolution
Control clip U_{clip}	100.0	Applied to oracle labels before normalization; removes $\approx 2\%$ of tails
Early stopping monitor	Validation Huber loss	
Early stopping patience	15 epochs	
Minimum improvement Δ	10^{-6}	
Best weights restored	Yes	Checkpoint saved at best validation loss
Transfer init (ensemble only)	Source: <code>per_particle_best.pt</code>	Base decoder weights copied from best per-particle checkpoint
Frozen epochs (ensemble only)	50	Base decoder frozen; encoder, pooling, correction decoder, and s trained
Unfreeze LR (ensemble only)	10^{-4}	Reduced rate after base decoder is unfrozen
PyTorch seed	42	
Data split seed	42	

Table F.4: Stage 2 training hyperparameters (weight-balance fine-tuning, `ensemble_s2` only). Begins from the best Stage 1 checkpoint of `ensemble_s1`.

Parameter	Value	Notes
Starting checkpoint	<code>ensemble_s1_best.pt</code>	Best Stage 1 weights for <code>ensemble_s1</code>
Initial learning rate	3×10^{-5}	One-tenth of Stage 1; preserves Stage 1 representations
LR schedule	Cosine annealing to 10^{-6}	
Max epochs	100	
Early stopping patience	15 epochs	
Gradient clip	1.0	
Total loss	$\mathcal{L}_{\text{Huber}} + \lambda \mathcal{L}_{\text{penalty}}$	Imitation loss plus weight-balance penalty
Huber loss space	Normalized	Same as Stage 1
Penalty space	Raw (denormalized)	Controls denormalized before computing predicted weights
Penalty formula	$\text{Var}_i[\tilde{w}_{\text{pred}}^{(i)}]$	Variance of post-control weights under Girsanov-plus-likelihood update
λ sweep	$\{0.1, 1.0, 10.0, 100.0\}$	Four independent Stage 2 runs from the same Stage 1 checkpoint
λ^* selected	0.1	Pareto-optimal on offline validation RMSE-nESS frontier

F.4 Closed-Loop Evaluation Hyperparameters

The closed-loop evaluation is the final arbiter of filter performance. Several design decisions in the evaluation setup are worth explaining.

All filters within a trial share the same truth trajectory, observation sequence, and initial particle ensemble. This is essential for fair comparison: differences in nRMSE or nESS between methods within a trial reflect the nudging strategy alone, not different realizations of the chaotic truth or different lucky starting positions. Only the particle integration noise differs between methods, so each filter’s particles explore state space independently while being evaluated against a common truth.

The seed structure uses a large prime step of 997 between trials to avoid accidental collisions between the signal, observation, and particle seeds. Within each trial, fixed offsets separate the four independent random streams signal integrator, observation sensor, initial ensemble, and particle integrators. This ensures that adding a new filter variant to the evaluation does not change the seeds of any existing variant.

The L96 evaluation has a confirmed trial count discrepancy. The script header states `N_TRIALS = 5`, but the saved `l96_closed_loop_trials.csv` contains 10 completed rows per method. The script was evidently re-run or extended; the CSV and the aggregated `l96_closed_loop_results.json` reflect all 10 trials and are the authoritative source for the results in Chapter 7.

Both systems normalize RMSE by the true state magnitude at each step: $\|x^{\text{true}} - \hat{x}\|_2 / \|x^{\text{true}}\|_2$. This makes the metric dimensionless and comparable across different attractor regions, where the true state magnitude varies considerably. All timing measurements use `time.perf_counter()` on a single CPU core with no parallelism; this includes the full inference pipeline for the ML controllers normalization, forward pass, denormalization, clipping, and all Monte Carlo rollouts for the oracle.

Table F.5: Verified closed-loop evaluation parameters.

Parameter	Lorenz-63	Lorenz-96
<i>System parameters</i>		
State dimension d	3	8
σ_{dyn}	1.0	1.0
σ_{obs}	2.0	1.0
<i>Filtering setup</i>		
Total time T (TU)	20.0	5.0
Observation interval ΔT (TU)	0.1	0.05
Integration step δt (TU)	0.001	0.001
Substeps N_{sub}	100	50
Observation steps N_{steps}	200	100
Particles N	32	16
Resampling threshold	nESS < 0.5	nESS < 0.5
Resampling method	Systematic (Univer- sal)	Systematic (Univer- sal)
nESS captured	Before resampling	Before resampling
<i>Trial structure</i>		

Parameter	Lorenz-63	Lorenz-96
Number of trials	10	10 (see note above)
Seed step	$997 \times \text{trial index}$	$997 \times \text{trial index}$
Signal seed	base +1	base +1
Observation seed	base +2	base +2
Initial ensemble seed	base +3	base +3
Particle integration seeds	base +10 + 7 $\times$ method index	base +10 + 7 $\times$ method index
Reference initial state	$(-3.139, -2.067, 22.904)^\top \cdot \mathbf{1}_8 + 0.01 \mathcal{N}(0, I_8)$, seed 7	
<i>ML controller deployment</i>		
Control clip U_{clip}	100.0	100.0
Inference mode	<code>torch.no_grad()</code>	<code>torch.no_grad()</code>
Device	CPU	CPU
Batch	All N particles in one forward pass	
<i>Oracle nPF (eval)</i>		
Rollouts n_s	15	10
Control substeps K_c	8	4
<i>Metrics and output</i>		
nRMSE normalization	$\ e\ _2 / \ x^{\text{true}}\ _2$	same
Timing method	<code>time.perf_counter()</code>	same
Results format	Scalar means/stds	Per-timestep time-series + scalars
Trial CSV written	No	Yes (196_closed_loop_trials.csv)

F.5 Repository Structure

All source code for this thesis lives inside the `pybeebase` repository, hosted on Bitbucket and versioned on the branch `feature/ensemble-nudging-reorganize`. The experiment codebase is organized under a single subfolder whose name mirrors the thesis title, keeping all thesis-specific files cleanly separated from the upstream `pybeebase` library code.

Design rationale. The two packages, `pybeebase` and `pybeeda`, are sibling directories that must both be importable simultaneously. The critical constraint is that the Python interpreter’s working root must be the parent directory that contains both packages. Setting `sys.path` to either package’s own directory, rather than their shared parent, is the single most common source of `ModuleNotFoundError` in this project. This layout is intentional: placing the experiment code inside `pybeebase/` means the Bitbucket repository is the single source of truth for both the library and the thesis experiments, while `pybeeda` remains a separately installed sibling.

The complete repository structure is shown below. Files in `outputs/` that were generated automatically (debug CSVs, smoke-test datasets) are omitted for clarity; only files that appear in the thesis results are listed.

<code>pybeebase/</code>	[Bitbucket repo root]
<code> </code>	
<code>+-- README.md</code>	entry point and path troubleshooting
<code> </code>	
<code>+-- ensemble-nudging-pf/</code>	thesis experiment root
<code> +-- README.md</code>	title, abstract, dependencies, init
<code> </code>	
<code> +-- l63_sr/</code>	Lorenz-63 pipeline
<code> +-- README.md</code>	
<code> +-- scripts/</code>	
<code> +-- init_data_gen.py</code>	nPF oracle dataset generation
<code> +-- closed_loop_eval.py</code>	closed-loop eval (script)
<code> +-- closed_loop_eval_l63.ipynb</code>	closed-loop eval (notebook)
<code> +-- diagnose_controls.py</code>	visualize learned control fields
<code> +-- diagnose_pf.py</code>	inspect PF trajectories
<code> +-- eval_two_n.py</code>	particle-count ablation
<code> \-- plot_checkpoint.py</code>	plot training loss curves
<code> </code>	
<code> +-- notebooks/</code>	
<code> +-- training.ipynb</code>	train all model variants
<code> \-- training_explainer.md</code>	annotated walkthrough
<code> </code>	
<code> \-- outputs/</code>	
<code> +-- checkpoints/</code>	
<code> +-- per_particle_best.pt</code>	
<code> +-- ensemble_s1_best.pt</code>	
<code> +-- residual_ensemble_best.pt</code>	
<code> +-- ensemble_s2_lam0.1_lam0.100_best.pt</code>	
<code> +-- ensemble_s2_lam1.0_lam1.000_best.pt</code>	
<code> +-- ensemble_s2_lam10.0_lam10.000_best.pt</code>	
<code> \-- ensemble_s2_lam100.0_lam100.000_best.pt</code>	
<code> +-- l63_mp_sr_production.h5</code>	
<code> +-- l63_mp_sr_smoke.h5</code>	
<code> +-- closed_loop_results.json</code>	
<code> +-- eval_two_n.json</code>	
<code> +-- filtering_summary_bar.png</code>	

```

| |         +--- filtering_{rmse,ness}_timeseries.png
| |         +--- comparison_{all,oracle,pf,relaxation}_{rmse,ness}.png
| |         +--- clip_ablation.png
| |         \-- diag_pf_{rmse,trajectory}_trial0.png
|
| +--- 196_sr/
| |   +--- README.md
| |   +--- scripts/
| | |   +--- init_data_gen_196.py
| | |   +--- merge_shards_196.py
| | |   +--- training_196.py
| | |   +--- closed_loop_eval_196.py
| | |   +--- diagnose_traj_196.py
| | |   \-- run_datagen_196.slurm
| |
| |   +--- notebooks/
| | |   +--- datagen_196.ipynb
| | |   +--- training_196.ipynb
| | |   \-- closed_loop_eval_196.ipynb
| |
| |   \-- outputs/
| | |   +--- checkpoints/
| | | |   +--- per_particle_best.pt
| | | |   \-- ensemble_s1_best.pt
| | |   +--- 196_mp_sr_production.h5
| | |   +--- 196_closed_loop_results.json
| | |   +--- 196_closed_loop_trials.csv
| | |   +--- 196_filtering_{summary,runtime,timeseries}.png
| | |   \-- diag_{PF,Relaxation,nPF,SR,S1,traj}*.png
|

```

Dependency and path setup. The critical rule for working with this codebase is that both `pybeebase` and `pybeeda` must be on `sys.path` simultaneously, and both are added by pointing to their *shared parent directory*, not to either package itself. In a local script, this is done as:

```

import sys

from pathlib import Path

# Add the parent of both pybeebase/ and pybeeda/ to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parents[N]))

```

where `N` is the number of directory levels from the script to the shared parent. In the Colab notebooks, the equivalent is to mount Google Drive and set `PROJECT` to the `163_sr/` or `196_sr/` folder; the notebooks then walk upward to find the shared parent automatically using a ranked candidate list. The most common failure mode

encountered during this project was setting `sys.path` to `pybeebase/` itself rather than its parent, which causes `ModuleNotFoundError: No module named 'pybeebase'` on the very first import line. Both `pybeebase/README.md` and `pybeeda/README.md` contain a troubleshooting table covering this and five other common import errors with their fixes.

Checkpoint naming convention. The `163_sr/outputs/checkpoints/` directory contains one checkpoint file per trained variant. The naming convention is `{variant}_lam{lambda}_best.pt` for Stage 2 variants and `{variant}_best.pt` for Stage 1 and per-particle variants. Each checkpoint stores the model state dict, the training history, and the normalizer statistics together in a single file, so no separate normalizer file is needed for evaluation.

IRnPF output locations. The `IRnPF_test/` subfolders have the following output paths:

`IRnPF_test/163/outputs/irnpf_163_production.h5` and
`IRnPF_test/196/outputs/irnpf_196_production.h5`.

F.6 Software & Hardware Specifications

Table F.6: Software versions and hardware used at each pipeline stage.

Component	Specification
<i>Core software</i>	
Python	3.10+
PyTorch	2.x
NumPy	1.24+
h5py	3.x (HDF5 I/O)
matplotlib	3.7+
Particle filter framework	pybeeda [5]
Data and model utilities	pybeebase [6]
<i>Hardware by pipeline stage</i>	
L63 dataset generation	Local workstation (Apple M-series)
L96 dataset generation	Google Colab (A100 or T4 GPU, 15–30 min per run)
L96 generation (Della option)	Princeton Della HPC, Slurm array, 50 tasks
Model training	Google Colab (H100 or A100 GPU)
Closed-loop evaluation	Google Colab, single CPU core (all timing measurements)
<i>Checkpoint format</i>	
Keys stored	model_state, history, normalizer
Loading pattern	torch.load(path, map_location='cpu', weights_only=False)
Portability guarantee	Normalizer statistics embedded in checkpoint; evaluation requires no access to original training data

Checkpoint portability. Every checkpoint stores the model state dict and the normalizer statistics together in a single file. The normalizer must be stored alongside the weights because the network operates in a normalized input/output space: the same checkpoint evaluated with different normalizer statistics would produce incorrect controls. The loading pattern used in all evaluation notebooks is:

```
ckpt = torch.load(path, map_location='cpu', weights_only=False)
```



```
model.load_state_dict(ckpt['model_state'])
nm = nm_to_numpy(ckpt['normalizer'])
```

The `weights_only=False` flag is required because the checkpoint contains the normalizer dictionary, a Python dict of `torch.Tensor` values, in addition to the model state. The `nm_to_numpy` helper converts all tensor values to NumPy arrays for compatibility with the NumPy-based `pybeeda` filter code that runs particle propagation and Girsanov accumulation.

Timing methodology. All wall-clock times reported in Tables 6.1 and 7.1 are measured on an H100 GPU via Google CoLaboratory using `time.perf_counter()` with no parallelism across particles or substeps. For the ML controllers, timing includes the full per-step inference pipeline: ensemble summary computation, input normalization, MLP forward pass, output denormalization and clipping, and injection into the particle integrator. For the oracle nPF, timing includes all $K_c \times N \times n_s$ Monte Carlo rollouts, sensitivity integral accumulation, and Girsanov correction computation at each substep. Timing values are from a single representative trial, trial 9, rather than averages across trials, because wall-clock time is dominated by fixed computational costs that vary negligibly across trials.